UNIVERSIDADE FEDERAL DO PARÁ
INSTITUTO DE CIÊNCIAS EXATAS E NATURAIS
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

BLENO WILSON FRANKLIN VALE DA SILVA

REACT-M: UMA EVOLUÇÃO DA ABORDAGEM REACT PARA APOIO AO PROCESSO DE GERÊNCIA DE REQUISITOS DE SOFTWARE BASEADO EM EVIDÊNCIAS EMPÍRICAS

Belém
2020

BLENO WILSON FRANKLIN VALE DA SILVA

REACT-M: UMA EVOLUÇÃO DA ABORDAGEM REACT PARA APOIO AO PROCESSO DE GERÊNCIA DE REQUISITOS DE SOFTWARE BASEADO EM EVIDÊNCIAS EMPÍRICAS

Dissertação de Mestrado apresentada ao Programa de Pós-Graduação em Ciência da Computação da Universidade Federal do Pará, como requisito para obtenção do grau de Mestre em Ciência da Computação.
Área de Concentração: Engenharia de Software.
Orientador: Prof. Dr. Sandro Ronaldo Bezerra Oliveira.

Belém
2020

Bleno Wilson Franklin Vale da Silva

REACT-M: Uma evolução da abordagem REACT para apoio ao processo de gerência de requisitos de software baseado em evidências empíricas / Bleno Wilson Franklin Vale da Silva; orientador, Sandro Ronaldo Bezerra Oliveira - 2020.

Dissertação (Mestrado) - Universidade Federal do Pará, Instituto de Ciências Exatas e Naturais, Programa de Pós-Graduação em Ciência da Computação, Belém, 2020.

1 Engenharia de Requisitos Ágil. 2. Gerência de Requisitos. 3. Métodos Ágeis. 4. Melhoria do Processo de Software 5. Revisão Sistemática da Literatura. I. Oliveira, Sandro R. B orientador. II. Universidade Federal do Pará, Instituto de Ciências Exatas e Naturais, Programa de Pós-Graduação em Ciência da Computação. III. Título.

BLENO WILSON FRANKLIN VALE DA SILVA

REACT-M: UMA EVOLUÇÃO DA ABORDAGEM REACT PARA APOIO AO PROCESSO DE GERÊNCIA DE REQUISITOS DE SOFTWARE BASEADO EM EVIDÊNCIAS EMPÍRICAS

Dissertação de Mestrado apresentada ao Programa de Pós-Graduação em Ciência da Computação da Universidade Federal do Pará, como requisito para obtenção do grau de Mestre em Ciência da Computação.
Área de Concentração: Engenharia de Software.

Data da aprovação: Belém, ___/___/_____.

Banca Examinadora

_____
Prof. Dr. Sandro Ronaldo Bezerra Oliveira
Programa de Pós-Graduação em Ciência da Computação - UFPA – Orientador

_____
Prof. Dr. Gustavo Henrique Lima Pinto
Programa de Pós-Graduação em Ciência da Computação – UFPA – Membro Interno

_____
Prof. Dr. Carlos dos Santos Portela
Faculdade de Sistemas de Informação – Campus Universitário do Tocantins-Cametá – UFPA – Membro Externo

AGRADECIMENTOS

Agradeço a Deus, pois independente da maneira que eu exerço minha fé, sei que sempre esteve e está do meu lado me dando forças e sabedoria para alcançar meus objetivos.

Agradeço aos meus pais, Wilson Franklin e Eliete Barbosa e ao meu irmão, Bruno Wilson, por terem me dado oportunidade de concretizar e encerrar está caminhada da minha vida. Sei que eles não mediram esforços para que este sonho se realizasse, sem a compreensão, ajuda e confiança deles nada disso seria possível.

Agradeço ao Prof. Dr. Sandro Bezerra, por ser um grande educador e ter compartilhado seu conhecimento e experiência comigo, durante toda minha caminhada na UFPA: graduação e mestrado.

Agradeço aos meus amigos, Aline Lima, Evelyn Rodrigues, Pamella Melo, Phillipe Santos, Gessica Pinheiro, Arthur Xavier, Leonardo Afonso e Diego Abreu por estarem presentes de algum modo durante esta jornada, me dando força e acreditando em mim, o que me ajudou na superação dos momentos difíceis.

Agradeço aos membros da banca avaliadora deste trabalho, Prof. Dr. Gustavo Henrique Lima Pinto e Prof. Dr. Carlos dos Santos Portela, que aceitaram participar desse momento especial na minha vida profissional e acadêmica

Por fim, agradeço a todos os professores, com diploma ou sem, que tive durante a minha vida e que contribuíram para o meu crescimento. Com certeza este momento só se tornou realidade graças à ajuda de todos que um dia me ensinaram algo.

RESUMO

A adoção de abordagens ágeis para o desenvolvimento de software vem crescendo na indústria desde o início da década de 2000, devido propor soluções alternativas para minimizar alguns problemas das abordagens tradicionais, bem como alguns desafios dos próprios métodos ágeis atuais. Uma área que está ganhando destaque é a Agile-RE, que busca aplicar os princípios e os valores dos métodos ágeis nas atividades de Engenharia de Requisitos, ou seja, permitir desenvolver e gerenciar os requisitos de forma mais ágil. O método REACT (*Requirements Evolution in Agile ContexT*) é uma proposta de método ágil para o desenvolvimento de requisitos ágeis, no entanto não possui cobertura às atividades de Gerência de Requisitos que são fundamentais para controlar a evolução dos requisitos durante a vida do software. Neste contexto, o presente trabalho tem como objetivo apresentar o método REACT-M (REACT-*Management*), uma proposta de método ágil para apoio às atividades de gerência de requisitos que esteja fortemente fundamentado em evidências empíricas da indústria de software, nas práticas da Engenharia de Requisitos Ágil e nos resultados esperados do processo de Gerência de Requisitos constantes nos modelos de qualidade MR-MPS-SW e CMMI-DEV. As evidências empíricas foram extraídas de outras abordagens ágeis, as quais foram identificadas e analisadas a partir de uma Revisão Sistemática da Literatura. Com os resultados obtidos do estudo secundário foi criado o método REACT-M. Embora este método possa ser considerado uma extensão do método REACT para desenvolvimento de software, o método pode ser usado de forma independente, o que facilita a adoção por interessados. O método REACT-M foi avaliada por meio de Estudo de Caso, conduzido em um projeto real para o desenvolvimento de um produto destinado à aprendizagem de programação. Em síntese dos resultados, observou-se que o método proporcionou mais simplicidade, flexibilidade e maior comunicação entre os participantes ao longo do projeto. Desta forma, o estudo pretende ser uma contribuição à comunidade científica e indústria de software por fornecer uma abordagem ágil para uma área especifica da engenharia de requisitos, que também está alinhada a modelos de qualidade voltados a Melhoria do Processo de Software.

PALAVRAS-CHAVE: Engenharia de Requisitos Ágil, Gerência de Requisitos, Métodos Ágeis, Melhoria do Processo de Software, Revisão Sistemática da Literatura.

ABSTRACT

The adoption of agile approaches to software development has been growing in industry since early 2000's, due to propose alternative solutions to minimize some problems of traditional approaches, as well as some challenges of the current agile methods. A field that is gaining prominence is Agile-RE, which aims to apply the principles and values of the agile methods in the Requirements Engineering activities, in other words, to allow the development and management of requirements in a more agile way. The REACT Method (Requirements Evolution in Agile ContexT) is an agile method proposal to the development of agile requirements, however, it doesn't cover in the activities of Requirements Management, which are critical to control the evolution of the requirements during the software lifecycle. In this context, this work aims to present the REACT-M (REACT-Management) method, an agile method proposal to support the requirements management activities which are strongly grounded on empirical evidences in the software industry, in the Agile Requirements Engineering and in the expected results of the Requirements Management process, constant in the MR-MPS-SW and CMMI-DEV quality models. The empirical evidences were extracted from other agile approaches, which were identified and analysed through a Systematic Literature Review. From the results obtained in the secondary study, it was created the REACT-M method, which aims to cover all the activities related to software requirements management in an agile environment. Although this method may be considered an extension of the REACT method to software development, the method can be independently used, which facilitates the adoption by those who are interested. The REACT-M method was rated, by Study Case, conducted in a real project to the development of a product intended to the programming learning. In summary of the results, it was observed that the method provided more simplicity, flexibility, and more communication among the participants during the project. Thus, the study intends to be a contribution to the scientific community and the software industry, because it brings an agile approach to a specific field of the requirements engineering, which is also aligned with quality models which aims Software Process Improvement.

KEYWORDS: Agile Requirements Engineering, Requirements Management, Agile Methods, Software Process Improvement, Systematic Literature Review.

LISTA DE FIGURAS

Figura 2.1 Níveis de Maturidade do CMMI-DEV (Adaptado de SEI, 2010) 29
Figura 2.2 Componentes do Modelo MPS.BR (Adaptado de SOFTEX, 2016a). 31
Figura 2.3 Relação entre Estudos Primários e Estudos Secundários (Barcelos e Travassos, 2006). 37
Figura 2.4 Atividades da Engenharia de Requisitos 45
Figura 2.5 Representação do modelo cascata (Royce, 1970) 47
Figura 2.6 Fundamentação dos métodos ágeis 50
Figura 2.7 Ciclo de Vida do REACT (Correa e Oliveira, 2018) 53
Figura 3.1 Processo para condução de RSL. 57
Figura 3.2 Processo de extração e síntese dos dados 71
Figura 3.3 Qualidade dos estudos primários 73
Figura 3.4 Países que mais publicaram sobre Agile-RE 74
Figura 4.1 Fases para elaboração do método ágil REACT-M 84
Figura 4.2 Fluxo de tarefas do contemplando os processos DRE e GRE 86
Figura 4.3 Mapeamento de Atividades do Processo de GRE com abordagens ágeis selecionadas 89
Figura 4.4 Ciclo de Vida REACT-M 92
Figura 4.5 Template de Persona 94
Figura 4.6 Template de Domain Expert Kanban 96
Figura 4.7 Template de Team Kanban 97
Figura 4.8 Template de Validação utilizando Critérios INVEST 99
Figura 4.9 Exemplos de Rastreabilidade Entre Users Stories e Artefatos derivados ... 101
Figura 4.10 Formulário para controle de mudanças 103
Figura 5.1 Desenho do Estudo de Caso 105
Figura 5.2 Personas Mapeadas do Produto 108
Figura 5.3 Domain Expert Kanban 109
Figura 5.4 Team Kanban 110
Figura 5.5 Critérios de Aceite 110
Figura 5.6 Formulário para controle de mudanças 112

LISTA DE QUADROS

Quadro 1.1 Níveis de Maturidade e Processos do MR-MPS-SW (Adaptado de SOFTEX, 2016a). 34
Quadro 1.2 Correlação de atividades que compõem a Engenharia de Requisitos 44
Quadro 1.3 Manifesto para desenvolvimento ágil de software (Agile Manifesto, 2001) 48
Quadro 1.4 Práticas da Agile-RE 51
Quadro 1.5 Exemplos de Desafios da Agile-RE 52
Quadro 3.1 Quadro metodológico da RSL 57
Quadro 3.2 Fontes de busca selecionadas 62
Quadro 3.3 String de Busca Genérica. 65
Quadro 3.4Critérios de inclusão e Exclusão 66
Quadro 3.5Critérios de qualidade 67
Quadro 3.6 Escala de Likert-5 68
Quadro 3.7 Escala para cada critério de qualidade 68
Quadro 3.8 Níveis de qualidade 70
Quadro 3.9 - Mapeamento de abordagens por estudos primários. 75
Quadro 4.1 Critérios Objetivos para a seleção de abordagens ágeis da RSL 87
Quadro 4.2 Escala Likert-3 para pontuação das abordagens ágeis 87
Quadro 4.3 Abordagens ágeis selecionadas 88
Quadro 5.1 Quadro metodológico do Estudo de Caso 106
Quadro 5.2 Forças do REACT-M 113
Quadro 5.3 Fraquezas do REACT-M 113
Quadro 5.4 Oportunidades de Melhoria do REACT-M 114
Quadro 5.5 Ameaças ao REACT-M. 114
Quadro 5.6 Melhorias no REACT-M 115

LISTA DE TABELAS

Tabela 3.1 Número de estudos por fonte de busca 72

LISTA DE ABREVIATURAS E SIGLAS

Agile-RE	*Agile Requirements Engineering*
ASD	*Agile Software Development*
CMMI	*Capability Maturity Model Integration*
CMMI-ACQ	CMMI *for Acquisition*
CMMI-DEV	CMMI *for Development*
CMMI-SVC	CMMI *for Services*
DRE	Desenvolvimento de Requisitos de Software
DSDM	*Dynamic systems development method*
FDD	*Feature Driven Development*
IEEE	*Institute of Electrical and Electronics Engineers*
ISO	*International Organization for Standardization*
IEC	*International Electrotechnical Commission*
MA-MPS	Método de Avaliação MPS.BR
MN-MPS	Modelo de Negócio MPS.BR
MPS.BR	Melhoria do Processo de Software Brasileiro
MR-MPS-RH	Modelo de Referência MPS para Gestão de Pessoas
MR-MPS-SV	Modelo de Referência MPS para Serviços
MR-MPS-SW	Modelo de Referência MPS para Software
PBIs	*Product Backlog Itens*
PNQ	Prêmio Nacional da Qualidade
REACT	Requirements Evolution in Agile ContexT
REACT-M	REACT- *Management*
RD	*Requirements Engineering*
REQM	*Requirements Management*
RSL	Revisão Sistemática da Literatura
SEI	*Software Engineering Institute*
SG	*Specific Goals*
SOFTEX	Associação para Promoção da Excelência do Software Brasileiro
SP	*Specific Practices*
SPIDER	*Software Process Improvement: Development and Research*
SWEBoK	*Software Engineering Body of Knowledge*
SWOT	*Strengths, Weaknesses, Opportunities and Threats*

TI	Tecnologia da Informação
TDD	*Test Driven Development*
UFPA	Universidade Federal do Pará
XP	*eXtreme Programming*

SUMÁRIO

1 INTRODUÇÃO ... 18
1.1 Contexto do Trabalho ... 18
1.2 Motivação ... 20
1.3 Objetivos ... 21
1.4 Metodologia do Trabalho ... 22
1.5 Estrutura do Trabalho ... 24
2 FUNDAMENTAÇÃO TEÓRICA ... 26
2.1 Melhoria do Processo de Software ... 26
2.1.1 O Modelo CMMI-DEV ... 27
2.1.2 O Modelo MR-MPS-SW ... 30
2.2 Engenharia de Software Baseada em Evidências ... 35
2.2.1 Método de Revisão Sistemática da Literatura ... 38
2.3 Engenharia de Requisitos ... 42
2.3.1 Processo de Gerência de Requisitos ... 45
2.4 Métodos Ágeis ... 46
2.5 Engenharia de Requisitos Ágil ... 50
2.6 Método REACT para Desenvolvimento de Requisitos ... 52
2.7 Trabalhos Relacionados ... 54
3 UMA REVISÃO SISTEMÁTICA DA LITERATURA SOBRE ENGENHARIA DE REQUISITOS E MÉTODOS ÁGEIS ... 56
3.1 Metodologia da Revisão Sistemática ... 56
3.2 Estratégias da Revisão Sistemática ... 58
3.2.1 Objetivo da Revisão ... 58
3.2.2 Questão de Pesquisa Principal ... 59
3.2.3 Questões Secundárias ... 60
3.2.4 Escopo e Recursos da Pesquisa ... 60
3.2.5 Busca Primária ... 61
3.2.6 Seleção de Fontes de Busca ... 62
3.2.7 Identificação de Palavras-Chaves e Sinônimos ... 63
3.2.8 Geração de String de Busca ... 64
3.2.9 Estratégia de Seleção dos Estudos Primários ... 65
3.2.10 Estratégia de Avaliação da Qualidade dos Estudos Primários ... 67

3.2.11 Estratégia de Extração e Síntese dos dados 71
3.3 Sumarização dos Resultados da Revisão Sistemática 71
4 REACT-M: UM MÉTODO ÁGIL PARA A GERÊNCIA DE REQUISITOS DE SOFTWARE 83
4.1 Metodologia para Criação do REACT-M 84
4.1.1 Fase de Planejamento 85
4.1.2 Fase de Seleção 87
4.1.3 Fase de Composição 90
4.2 Guia de utilização do REACT-M 91
4.2.1 Atividade de Identificar Fornecedores de Requisitos 93
4.2.2 Atividade de Elaborar Backlog de Tarefas 95
4.2.3 Atividade de Avaliar e Dimensionar 0 Backlog com a Equipe Técnica 97
4.2.4 Atividade de Rastreabilidade de Requisito 100
4.2.5 Atividade de Revisar e Registrar Inconsistências 101
4.2.6 Tratar Mudanças no Backlog 102
5 AVALIAÇÃO DA ABORDAGEM PROPOSTA 104
5.1 Metodologia do Estudo de Caso 105
5.2 Fase de Planejamento 106
5.3 Fase de Execução 107
5.3.1 Primeira Iteração de Desenvolvimento 107
5.3.2 Segunda Iteração de Desenvolvimento 111
5.4 Fase de Avaliação 112
6 CONCLUSÕES 116
6.1 Sumarização dos Resultados deste Estudo 116
6.2 Contribuições 118
6.3 Limitações 119
6.4 Trabalhos Futuros 120
6.4.1 Evolução da Revisão Sistemática 120
6.4.2 Evolução da Abordagem Ágil proposta 120
6.4.3 Utilização da abordagem em empresas que buscam maturidade 121
REFERÊNCIAS BIBLIOGRÁFICAS 122
APÊNDICE A – PROTOCOLO DA REVISÃO SISTEMÁTICA DA LITERATURA 126
APÊNDICE B – ESTUDOS PRIMÁRIOS SELECIONADOS 138
APÊNDICE C – FORMULÁRIO DE AVALIAÇÃO SWOT 145

1 INTRODUÇÃO

Neste capítulo a contextualização, a motivação, os objetivos, a metodologia e, finalmente, a estrutura desta dissertação serão apresentados. O objetivo deste capítulo é prover uma visão geral deste estudo e estabelecer o roteiro a ser seguido durante o desenvolvimento deste.

1.1 Contexto do Trabalho

Os métodos ágeis de desenvolvimento de software vêm ganhando crescente popularidade desde o início da década de 2000. Este fato fundamenta-se pelos diversos benefícios que os mesmos proporcionam à indústria de software, como melhor gestão de mudanças, aumento na produtividade da equipe, melhoria na qualidade de software e maior ênfase na comunicação com o cliente (Versione, 2015). Além disso, estes métodos surgiram como uma alternativa para minimizar alguns dos problemas das abordagens tradicionais, enfrentados por um mercado que exige cada vez mais rapidez na entrega de produtos de qualidade (Qusef e De Lucia., 2010). Alguns destes problemas estão relacionados aos requisitos, como por exemplo pouco envolvimento dos clientes, alta mudança nos requisitos, requisitos mal especificados, dentre outros (Weigers *et al.*, 2013). Entretanto, segundo Inayat *et al.* (2014), Dyba e Dingsoyr (2008) e Parsons *et al.* (2007), ainda são escassas as evidências empíricas sobre as metodologias ágeis que estão sendo utilizadas em ambientes reais de desenvolvimento de software.

Dentre as áreas de conhecimento da engenharia de software, a engenharia de requisitos possui extrema importância no ciclo de vida de projetos de software (Pressman e Maxin, 2016; Sommerville, 2010). Segundo o *Chaos Report* publicado por Standish Group (2014), cerca de 21% dos principais fatores de falhas em projetos de

software estão relacionados com os requisitos, que possuem impacto direto e na qualidade do produto final. Já em outro estudo realizado por Schwaber (2007), o qual analisou dados de mais de 100 projetos de desenvolvimento de software, o autor notou que 35% dos requisitos mudam, 65% das funcionalidades descritas pelos requisitos nunca ou raramente são utilizadas e cerca de 50% do tempo desprendido da equipe é gasto com requisitos, arquitetura e especificação do produto. Com base nesta afirmativa, diversos modelos de qualidade para processos de software como o MR-MPS-SW (Modelo de Referência MPS para Software) (SOFTEX, 2016a) e o CMMI-DEV (*Capability Maturity Model Integration for Development*) (SEI, 2010) adotam guias para o desenvolvimento e gerência de requisitos.

Neste cenário, vem crescendo consideravelmente o número de abordagens ágeis para apoio ao desenvolvimento de produtos de software, surgindo diversos métodos, técnicas e práticas alinhadas aos princípios e valores do manifesto ágil que buscam resolver ou minimizar alguns dos problemas ou lacunas ainda existentes nos processors usados na indústria de software (Medeiros, 2017). Neste contexto, a engenharia de requisitos ágil vem ganhando destaque, por sugerir o uso de práticas, princípios e valores dos métodos ágeis em suas atividades. Esta abordagem permite desenvolver e gerenciar os requisitos de forma mais ágil, utilizando documentação mais simples e concisa, eliminando alguns artefatos e documentos de controle que com frequência são identificados nas metodologias tradicionais (Santos e Oliveira, 2018). Logo, torna-se significativo e justificável realizar uma pesquisa científica a fim de relacionar a área de Engenharia de Requisitos com os métodos ágeis.

Além disso, é notável o aumento da preocupação com a qualidade do produto por parte da indústria de software, buscando melhor atender a necessidade dos usuários e também tornando este ponto um diferencial no mercado para as organizações. Para isto, tem-se investido bastante na melhoria do processo de desenvolvimento de software (Oliveira e Vasconcelos, 2016). Desta forma, diversos modelos de qualidade para processos de software como o MR-MPS-SW e o CMMI-DEV surgem com o objetivo de orientar o caminho da melhoria. Os modelos citados possuem preocupação com as atividades de Engenharia de Requisitos e fornecem boas práticas que devem ser adotadas para o Desenvolvimento de Requisitos (DRE), que abrange as atividades de

elicitação, análise, especificação e validação de requisitos, e também para a Gerência de Requisitos (GRE), que envolve atividades de gerenciar o *Backlog*, rastreabilidade, gerência de mudança, revisão de requisitos (SEI, 2010; SOFTEX, 2016b; Weigers *et al.*, 2013). É importante ressaltar que o presente estudo está inserido no contexto do processo de Gerência de Requisitos, por ser uma extensão do estudo conduzido por Santo e Oliveira (2018), que possuía foco nas atividades de desenvolvimento de requisitos. Assim, este trabalho pretende tratar de uma lacuna do trabalho anterior, fornecendo um controle e evolução dos requisitos em ambiente ágil

Portanto, pretende-se realizar este estudo com o objetivo de propor uma abordagem ágil de apoio às atividades do processo de Gerência de Requisitos, o método REACT-M (REACT-*Management*). Esta abordagem estará fortemente fundamentada em evidências empíricas, coletada de estudos publicados na comunidade científica, dos quais foram extraídos diversos métodos, práticas ou técnicas ágeis identificados na literatura especializada (Santos e Oliveira, 2018). Desta forma, contribuindo à indústria de software e à comunidade científica ao fornecer uma nova proposta de método ágil específico para a área de requisitos e que atenda a todas as atividades de Gerência de Requisitos de forma completa, organizada e eficiente. Além disso, servir como uma alternativa na área de requisitos para organizações ágeis que buscam implementar modelos de maturidade de processos de software, como CMMI-DEV e MR-MPS-SW, por meio de métodos ágeis, uma vez que o processo de Gerência de Requisitos é um dos processos contidos nestes modelos. Para esta dissertação, por abordagem entende-se como um método que possui artefatos, papéis envolvidos e um passo-a-passo de instrução de uso com a finalidade de atingir um objetivo específico.

1.2 Motivação

Com o crescimento dos métodos ágeis na indústria de software, a Engenharia de Requisitos (ER) vem quebrando o paradigma de que todos os requisitos do produto devem ser levantados em fases iniciais do projeto, para uma perspectiva mais real, compreendendo que requisitos vão mudar durante o projeto, de acordo com as necessidades dos clientes. Assim, o uso dos valores e princípios ágeis em atividades de ER pode ajudar a minimizar problemas comumente deparados nos projetos de

desenvolvimento de software, como por exemplo pouco envolvimento dos clientes, alta mudança nos requisitos, requisitos mal especificados, dentre outros (Weigers *et al.*, 2013).

Pode-se considerar também que a ER é uma área de grande importância para o sucesso de projetos de desenvolvimento de software (Pressman e Maxin, 2016). As atividades de DRE estão focadas na construção de requisitos. No entanto, para uma ER efetiva é importante que não sejam negligenciadas as atividades de GRE, preocupadas em controlar a evolução dos requisitos ao longo do ciclo de vida de um projeto de software (SOFTEX, 2016b).

Além disso, muitos dos principais métodos ágeis difundidos na indústria de software possuem foco nas áreas como gestão, modelos de ciclo de vida ou testes, por exemplo Scrum, ASD, TDD, BDD, XP, dentre outros; fornecendo poucas práticas endereçadas para as atividades da ER. O trabalho de Santos e Oliveira (2018) propõe a adoção do método ágil REACT para as atividades de DRE, tentando dessa forma fornecer um *framework* específico pra ER Ágil. No entanto, é importante salientar que o objetivo do REACT é de elicitar, analisar, especificar e validar os requisitos de software, não tendo como objetivo as atividades de GRE.

Nesse cenário, este estudo motiva-se pelo fato da área de requisitos ser tratada apenas como parte de muitos dos principais métodos ágeis existentes na indústria, os quais possuem foco em outras áreas como gestão e testes. Existem trabalhos voltados especificamente *para* ER Ágil, porém ainda se faz necessário estudos que estressem o tema e tentem cobrir lacunas ainda não identificadas. Assim, este estudo pode ser considerado uma extensão do trabalho produzido por Santos e Oliveira (2017) e tem como objetivo propor um método ágil que contemple as atividades de GRE responsáveis pela evolução criteriosa dos requisitos.

1.3 Objetivos

O principal objetivo deste estudo é desenvolver um método ágil voltado à área de requisitos para apoio às atividades de GRE, que esteja aderente ao método REACT proposto por Santos e Oliveira (2018) e fundamentado em evidências empíricas presentes na literatura, fornecendo uma solução ágil para profissionais da indústria de

software que almejam adotar uma abordagem ágil para a evolução dos requisitos.

Para atender ao objetivo geral, os seguintes objetivos específicos devem ser contemplados:

- Aprofundar o conhecimento sobre Engenharia de Requisitos e Gerência de Requisitos no contexto de métodos ágeis, por meio de cursos de capacitação e por pesquisa bibliográfica informal, a fim de obter um melhor embasamento teórico;
- Identificar abordagens ágeis de apoio às atividades de Gerência de Requisitos, por meio de uma Revisão Sistemática da Literatura;
- Elaborar uma proposta de método ágil de apoio às atividades de Gerência de Requisitos com base nas abordagens ágeis empíricas identificadas na Revisão Sistemática da Literatura;
- Avaliar a aplicabilidade desta proposta de método ágil a partir de um Estudo de Caso em um ambiente de desenvolvimento de software a fim de identificar suas forças, fraquezas, oportunidades de melhoria e ameaças em relação ao seu uso;
- Publicar resultados de pesquisa em congressos científicos da área de informática.

1.4 Metodologia do Trabalho

A realização deste trabalho ocorreu a partir das etapas descritas a seguir:

a) Etapa do Estudo Inicial

- Estudo dos modelos de referência CMMI-DEV e MR-MPS-SW, com foco nos processos referentes a Gerência de Requisitos;
- Estudo sobre métodos ágeis aplicados ao contexto da Engenharia de Software e Engenharia de Requisitos;
- Estudo aprofundado de trabalhos na área da Engenharia de Requisitos Ágil;
- Estudo teórico sobre conduções de Revisões Sistemáticas da Literatura aplicadas na área da Engenharia de Software.

b) Etapa da Revisão Sistemática da Literatura

- Definição de um protocolo para a revisão sistemática (Apêndice A);
- Execução do protocolo da revisão sistemática;
- Aplicação da técnica de Síntese e Análise Temática (Cruzes e Dyba, 2011) para realizar a análise qualitativa dos dados obtidos pela execução da revisão sistemática;
- Apresentação dos resultados obtidos a partir da revisão sistemática;
- Identificação de trabalhos relacionados a esta pesquisa obtidos a partir da revisão sistemática.

c) Etapa da Elaboração da Abordagem Ágil de apoio à Gerência de Requisitos de Software

- Modelagem de um fluxo de atividades para o processo de Gerência de Requisitos de software com base nos modelos de qualidade CMMI-DEV e MR-MPS-SW;
- Definição de critérios objetivos para a seleção de um conjunto menor das evidências empíricas identificadas pela Revisão Sistemática da Literatura;
- Realização da seleção das evidências empíricas para fundamentar a concepção da abordagem ágil para o Gerência de Requisitos;
- Análise das evidências empíricas selecionadas a fim de definir um conjunto de cerimônias, artefatos, papéis e o ciclo de vida que irão compor a abordagem ágil para o Gerência de Requisitos.

d) Etapa do Estudo de Caso para Avaliação da Abordagem Ágil de apoio à Gerência de Requisitos de Software

- Planejamento e definição do tipo de projeto de software, equipe de desenvolvimento e recursos necessários para aplicar a abordagem ágil;
- Definição da técnica de avaliação desta proposta junto aos participantes do estudo de caso;

- Execução do Estudo de Caso, onde deve ser aplicado o método ágil proposto;
- Condução de *Brainstormings* com os participantes do Estudo de Caso para avaliação da aplicabilidade da abordagem ágil proposta por meio da Técnica SWOT (*Strengths, Weaknesses, Opportunities and Threats*).

e) Etapa da Documentação

- Elaboração dos documentos gerados ao longo desta pesquisa (Protocolo da RSL, Artigos Científicos, Guia Prático para uso da abordagem proposta; Processo de avaliação do Estudo de Caso; e redação final da dissertação).

A pesquisa realizada pode ser caracterizada de várias formas, utilizando a definição de Silva e Menezes (2005) é possível classificá-la da seguinte maneira:

- **Quanto à natureza:** pesquisa aplicada, pois tem o objetivo de gerar conhecimentos para a aplicação prática focando na solução de problemas específicos;
- **Quanto à abordagem do problema:** pesquisa qualitativa são utilizadas. Porém, também quantitativa devido a necessidade de se traduzir em números algumas informações obtidas com a condução da Revisão Sistemática da Literatura;
- **Quanto aos objetivos:** trata-se de uma pesquisa exploratória e descritiva, pois proporciona um maior entendimento do problema, relacionando levantamentos bibliográficos e por utilizar questionários para verificar as características e percepções de uma população;
- **Quanto aos procedimentos técnicos:** classifica-se como uma pesquisa Bibliográfica e Experimental, pois a mesma foi elaborada a partir de materiais publicados como artigos de periódicos e de eventos, livros e materiais disponibilizados na Internet, igualmente, avaliada a partir da condução de um Estudo de Caso.

1.5 Estrutura do Trabalho

Este capítulo introdutório abordou a visão geral a respeito do trabalho desenvolvido nesta dissertação, identificando o contexto, a motivação, os objetivos e a metodologia utilizada para sua execução.

O Capítulo 2 apresenta a fundamentação teórica deste estudo, que inclui uma descrição detalhada dos tópicos relevantes à pesquisa e dos trabalhos relacionados.

O Capítulo 3 apresenta a Revisão Sistemática da Literatura, incluindo a estrutura de execução e os seus resultados preliminares.

O Capítulo 4 apresenta o REACT-M, uma proposta de abordagem ágil para a gerência de requisitos de software, o qual foi fundamentado a partir de um conjunto de evidências empíricas, identificadas por meio da Revisão Sistemática discutida no Capítulo 3.

O Capítulo 5 apresenta o Estudo de Caso conduzido em um projeto real de desenvolvimento de software destinado ao aprendizado de programação dentro da Universidade Federal do Pará. O Estudo de Caso tem o intuito de avaliar a aplicabilidade da abordagem ágil proposta.

Finalmente, o Capítulo 6 apresenta as considerações finais, contribuições, limitações e trabalhos futuros desta dissertação.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta a fundamentação teórica desta dissertação, esclarecendo alguns conceitos e termologias relacionados à engenharia de software, que embasam o trabalho desenvolvido. Os seguintes tópicos serão discutidos:

- Melhoria do processo de Software;
  - Modelo CMMI-DEV;
  - Modelo MR-MPS-S;
- Engenharia de Software Baseada em Evidências;
  - Método de Revisão Sistemática da Literatura;
- Engenharia de Requisitos;
  - Processo de Gerência de Requisitos;
- Métodos Ágeis para Desenvolvimento de Software;
- Engenharia de Requisitos Ágeis;
- Método REACT para Desenvolvimento de Requisitos;
- Trabalhos Relacionados.

2.1 Melhoria do Processo de Software

Um processo de software é explicado por Pressman (2016), como um conjunto organizado de atividades, ações e tarefas, que formam um roteiro para o desenvolvimento de um produto de alta qualidade. O trabalho de Sommerville (2019) define processo de software como um conjunto de atividades relacionadas que levam à produção de um software.

De acordo com Falbo (2005), a qualidade de um produto está fortemente ligada à qualidade do processo utilizado para desenvolvê-lo. A definição de um processo de software possibilita um melhor entendimento das atividades necessárias para a fabricação de um produto de qualidade.

Para a IEEE (2014), a norma ISO/IEC 25010:2011 descreve a qualidade de software de acordo com o nível que um sistema, componente ou processo consegue atender aos requisitos do produto e as expectativas ou necessidades dos clientes ou usuários, o que evidencia que a área da qualidade de software está fortemente ligada ao processo utilizado para desenvolvê-lo (Oliveira e Vasconcelos, 2016).

Devido esta forte conexão entre a qualidade do produto e processo, a comunidade de engenharia de software desenvolveu diversos guias de boas práticas, normas e modelos para auxiliar as organizações a melhorarem seu desempenho quanto a custo, prazo e qualidade do produto (Oliveira e Vasconcelos, 2016). Modelos como MR-MPS-SW (Modelo de Referência do MPS para Software) (SOFTEX, 2016a) e CMMI-DEV (*Capability Maturity Model Integration for Development*) (SEI, 2010) definem boas práticas que podem ser usadas como base para a melhoria de um processo de software. As próximas subseções irão apresentar estes modelos com mais detalhes, dando foco no processo Gerência de Requisitos, tema central deste trabalho.

2.1.1 O Modelo CMMI-DEV

O modelo de maturidade para melhoria do processo de software CMMI (*Capability Maturity Model Integration*) foi desenvolvido pela SEI (*Software Engineering Institute*) e oferece um conjunto de boas práticas às atividades de desenvolvimento e manutenção de produtos e serviços. As práticas deste modelo abrangem todo o ciclo de vida do produto, que vão desde a concepção até a entrega e manutenção (SEI, 2010). O CMMI é formado por três modelos:

- *CMMI for Development* (CMMI-DEV): foca no processo de desenvolvimento de produtos de software;
- *CMMI for Acquisition* (CMMI-ACQ): foca nos processos de aquisição e terceirização de bens e serviços;

- *CMMI for Services* (CMMI-SVC): foca nos processos de empresas prestadoras de serviços.

O modelo CMMI-DEV contém os elementos essenciais para a definição de um processo efetivo e descreve um caminho para as organizações evoluírem seus processos de um nível imaturo para um nível disciplinado (Berttollo, 2006).

O CMMI-DEV possui duas formas de representação: a contínua e por estágios. A representação contínua possibilita uma maior flexibilidade na implementação do modelo, permitindo que a empresa escolha a área de processo ou grupo de áreas do processo que deseja melhorar. Esta representação é recomendada para organizações que buscam a melhoria em pontos problemáticos associados a um processo isolado, ou que querem trabalhar em cima de várias áreas que estão fortemente ligadas ao objetivo estratégico da empresa. Cada área de processo pode atingir os seguintes níveis de capacidade (SEI, 2010):

- Nível 0 ou Incompleto (caótico);
- Nível 1 ou Executado: o processo é executado de modo a completar o trabalho necessário para produzir o trabalho necessário;
- Nível 2 ou Gerenciado: planejar a execução do processo e analisar a aderência entre o planejado e o executado;
- Nível 3 ou Definido: o processo é construído sobre as diretrizes do processo existente, e é mantido uma descrição do processo.

A representação por estágios define conjuntos de áreas de processos que constituem um caminho de melhoria a ser seguido pela organização. Esse caminho, definido pelo modelo, é retratado por níveis de maturidade, no qual cada nível possui um conjunto de áreas do processo que caracterizam diferentes componentes da organização. O uso da representação por estágios é recomendado para organizações que não possuem definido quais áreas do processo desejam melhorar e que o seu principal objetivo é o ganho de maturidade. A representação por estágios oferece uma estrutura de melhoria gradativa das áreas do processo, onde cada estágio definido no modelo indica um nível de maturidade alcançado pela organização (SEI, 2010).

O modelo CMMI-DEV propõe a adoção da melhoria em cinco estágios. Cada estágio representa a base para as atividades de melhoria dos processos, a Figura 2.1 representa os níveis existentes no modelo.

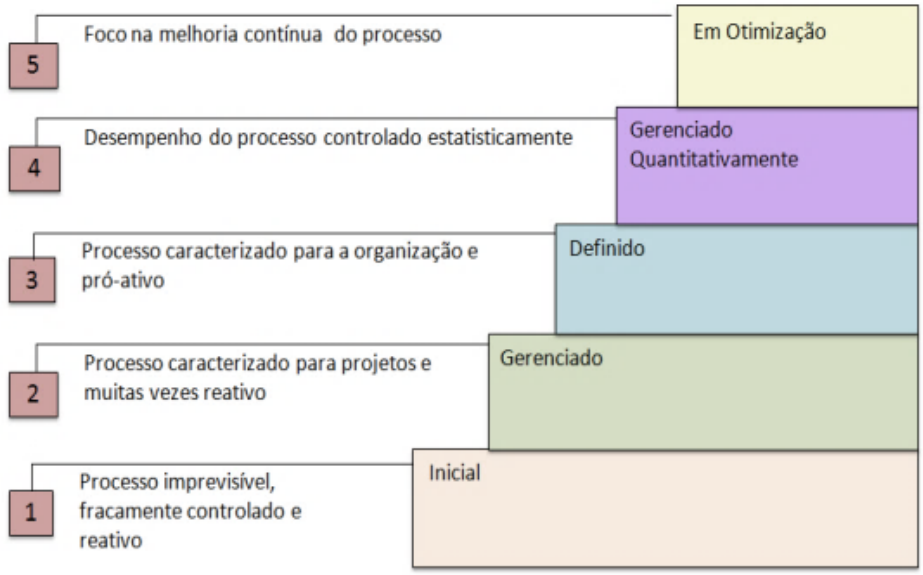


Figura 2.1 Níveis de Maturidade do CMMI-DEV (Adaptado de SEI, 2010).

Conforme a organização ganha maturidade, a melhoria dos processos torna-se padrão organizacional. A institucionalização do processo de melhoria provê o desenvolvimento de uma infraestrutura e uma cultura na organização que suporta métodos, práticas e procedimentos da empresa que permanecem mesmo após a saída de quem os definiu (Oliveira, 2006).

2.1.1.1 Processo de Gerência de Requisitos e o CMMI-DEV

O CMMI-DEV possui uma área de processo específica para a gerência de requisitos (*Requirements Management* - REQM), presente no nível 2 de maturidade. Esta área de processo tem o objetivo de "fornecer subsídios para gerenciar os requisitos dos produtos e componentes de produto do projeto e identificar inconsistências entre esses requisitos e os planos e produtos de trabalho do projeto" (adaptado de SEI, 2010). As recomendações do CMMI-DEV para esta área de processo são expressas por meio de uma meta específica (*Specific Goal* - SG) e práticas específicas (*Specific Practice* - SP) (adaptado de SEI, 2010):

- SG 1 Gerenciar Requisitos

- SP 1.1 Obter Entendimento dos Requisitos
- SP 1.2 Obter Comprometimento com os Requisitos
- SP 1.3 Gerenciar Mudanças nos Requisitos
- SP 1.4 Manter Rastreabilidade Bidirecional dos Requisitos
- SP 1.5 Identificar Inconsistências entre Produtos de Trabalho, Planos de Projeto e Requisitos

2.1.2 O Modelo MR-MPS-SW

O programa MPS.BR (Melhoria do Processo de Software Brasileiro), coordenado pela Associação para Promoção da Excelência do Software Brasileiro (SOFTEX), tem como objetivo o aumento da competitividade no mercado de software, guiando as organizações a atingirem qualidade a partir da melhoria de processos (SOFTEX, 2016a). De acordo com Bertollo (2006), este modelo baseia-se nos conceitos de maturidade e capacidade de processo para a melhoria e avaliação de produtos de software e serviços.

Neste contexto, o MPS.BR possui quatro componentes: Modelo de Referência MPS para Software (MR-MPS-SW), Modelo de Referência MPS para Serviços (MR-MPS-SV), Método de Avaliação (MA-MPS) e Modelo de Negócio para Melhoria de Processo de Software e Serviços (MN-MPS) (SOFTEX, 2016a). Neste trabalho, será abordado o Modelo de Referência MPS para Software (MR-MPS-SW), pois ele é destinado a organizações interessadas na implementação da melhoria em seus processos de software.

A Figura 2.2 ilustra os componentes do modelo MPS, os documentos relacionados e os padrões utilizados como base para a criação deste modelo.

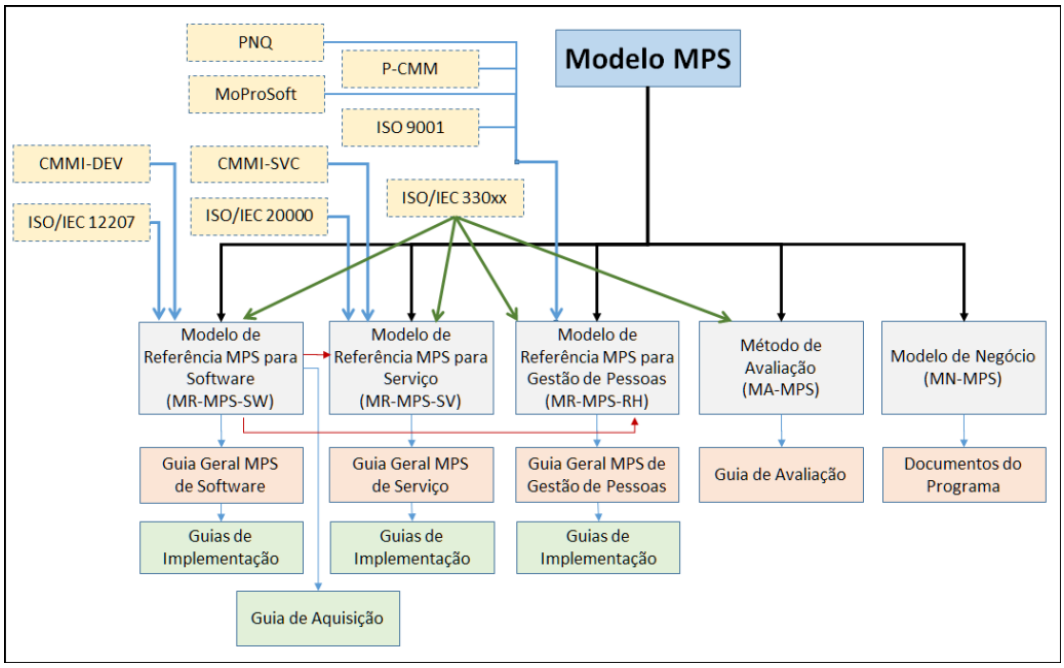


Figura 2.1 Componentes do Modelo MPS.BR (Adaptado de SOFTEX, 2016a).

O MR-MPS-SV possui como referência o MR-MPS-SW (SOFTEX, 2016b), a Norma Internacional ISO/IEC 20000, a Norma Internacional ISO/IEC 15504 e o modelo CMMI-SVC. Tem como objetivo descrever processos, níveis de maturidade e capacidade referentes a serviços, além das definições necessárias para se entendimento e aplicação.

O MA-MPS define os requisitos necessários para os avaliadores líderes, avaliadores adjuntos e Instituições Avaliadoras (IA). O processo e o método de avaliação MA-MPS baseiam-se na Norma Internacional ISO/IEC 33000 (SOFTEX, 2016b).

O MN-MPS detalha como devem ser implementados os modelos MR-MPS-SW e MR-MPS-SV pelas instituições implementadoras; define como a avaliação deve ser feita seguindo o modelo MA-MPS pelas instituições avaliadoras; como a organização de grupos de empresas devem ser feitas para a implementação do MR-MPS-SW e MR-MPS-SV e avaliação MA-MPS; descreve o processo para a certificação de consultores de aquisição; e programas anuais de treinamento do MPS.BR (SOFTEX, 2016a).

O MR-MPS-RH tem como referência: a NBR ISO 9001:2008; a ISO/IEC 33020; o P-CMM® (*People Capability Maturity Model*); o NMX-I-059/2-NYCE-2011 (*Tecnología de la información – Software – Modelos de procesos y evaluación para el*

desarrollo y mantenimiento de software – Parte 2: Requisitos de processos) (MoProSoft); e o PNQ – Prêmio Nacional da Qualidade (SOFTEX, 2016b).

O modelo MR-MPS-SW tem como uma de suas bases técnicas o modelo norte-americano CMMI-DEV e como ele utiliza do conceito de capacidade para medir a maturidade dos processos de software (SOFTEX, 2016a). O modelo é voltado para pequenas e médias empresas e tem como meta ser um modelo de melhoria nacional que esteja alinhado com os padrões internacionais de qualidade.

Os níveis de maturidade no uso do MR-MPS-SW são definidos pelo grau de capacidade atingido na execução dos processos. A cada nível é atribuído um perfil de processos e de capacidade de processos que indica onde a organização tem que colocar esforço para melhoria, de forma a atender os objetivos de negócio (Bertollo, 2006). O modelo define sete níveis de maturidade: A (Em Otimização), B (Gerenciado Quantitativamente), C (Definido), D (Largamente Definido), E (Parcialmente Definido), F (Gerenciado) e G (Parcialmente Gerenciado). O nível G é o nível inicial e progride até o nível A.

A capacidade de um processo representa um conjunto de atributos de processos descritos em termos de resultados, e indica o grau de refinamento e institucionalização com que o processo é executado na organização (SOFTEX, 2016a). No MR-MPS-SW à medida que a organização vai alcançando os níveis de maturidade, um maior nível de capacidade para desempenhar o processo deve ser atingido. A capacidade do processo possui nove atributos de processo (AP), que estão detalhados em termos dos resultados esperados para o alcance completo do atributo (SOFTEX, 2016a). Os atributos de processo do modelo MR-MPS-SW são:

- AP 1.1: O processo é executado;
- AP 2.1: A execução do processo é gerenciada;
- AP 2.2: Os produtos de trabalho do processo são gerenciados;
- AP 3.1: O processo é definido;
- AP 3.2: O processo está implementado;
- AP 4.1: O processo é objeto de análise quantitativa;

- AP 4.2: O processo é controlado quantitativamente;
- AP 5.1: O processo é objeto de melhorias incrementais e inovações;
- AP 5.2 O processo é objeto de implementação de melhorias inovadoras e incrementais.

Os atributos de processo AP 4.1, AP 4.2, AP 5.1 e AP 5.2 devem ser implementados apenas em processos críticos da organização em que se faz necessário uma análise de desempenho, os demais atributos devem ser implementados para todos os processos (SOFTEX, 2016a).

Segundo Bertollo (2006), a obtenção de um nível de maturidade ocorre quando são atendidos todos os resultados, propósitos dos processos e dos atributos de processos relacionados ao nível que está sendo implementado e aos anteriores. Os níveis são acumulativos, ou seja, caso a organização esteja implementando o nível E, isto significa que para ela atingir este nível de maturidade ela deve alcançar os atributos de processo descritos nos níveis E, F e G para todos os processos implementados (SOFTEX, 2016a). O Quadro 2.1 apresenta os processos de cada nível de maturidade.

Quadro2.1 Níveis de Maturidade e Processos do MR-MPS-SW (Adaptado de SOFTEX, 2016a).

Nível	Processos
A	Análise de Causas de Problemas e Resolução
B	Gerência de Projetos (evolução)
C	Análise de Decisão e Resolução Desenvolvimento para Reutilização Gerência de Riscos Gerência de Reutilização (evolução)
D	Desenvolvimento de Requisitos Projeto e Construção do Produto Integração do Produto Verificação Validação
E	Avaliação e Melhoria de Processo Organizacional Definição do Processo Organizacional Gerência de Recursos Humanos Gerência de Reutilização Gerência de Projetos (evolução)
F	Medição (para monitoração e controle) Gerência de Configuração Aquisição Garantia da Qualidade Gerência de Portfólio de Projetos
G	Gerência de Requisitos Gerência de Projetos

2.1.2.1 Processo de Gerência de Requisitos e o MR-MPS-SW

O processo de gerência de requisitos (GRE) encontrasse no nível G, ou parcialmente gerenciado, do MR-MPS-SW. O nível G é o primeiro nível de maturidade, o que exige cautela na implantação, tendo em vista que a sua adoção irá causar mudanças na cultura organizacional, para que a empresa adeque-se às boas práticas necessárias para a melhoria dos processos (SOFTEX, 2016b).

O principal objetivo do processo GRE é controlar a evolução dos requisitos do produto e dos componentes do produto, gerenciando todos os requisitos recebidos e gerados pelo projeto, o que incluem requisitos funcionais e não-funcionais, bem como os requisitos impostos ao projeto pela organização. A gerência dos requisitos fornece apoio para o planejamento e execução do projeto de maneira adequada. Quando os requisitos são levantados no início do projeto ou por solicitação de mudança, estes devem ser revisados para prevenir o mau entendimento da demanda, antes que os

requisitos sejam incluídos no escopo do projeto, para concretizar este passo o fornecedor de requisitos e a organização devem chegar a um acordo, e os demais envolvidos no projeto devem estar comprometidos com o que foi elicitado (SOFTEX, 2016b).

O processo de GRE também possui a responsabilidade de documentar as mudanças nos requisitos e suas justificativas, bem como prover e manter a rastreabilidade bidirecional entre os requisitos e produtos de trabalhos produzidos ao longo do projeto, a fim de facilitar a identificação das inconsistências entre os requisitos, os planos e produtos de trabalhos do projeto. Este processo é descrito com 5 resultados esperados, a saber (SOFTEX, 2016b):

- GRE1 - O entendimento dos requisitos é obtido junto aos fornecedores de requisitos;
- GRE2 - Os requisitos são avaliados com base em critérios objetivos e um comprometimento da equipe técnica com estes requisitos é obtido;
- GRE3 - A rastreabilidade bidirecional entre os requisitos e os produtos de trabalho é estabelecida e mantida;
- GRE4 - Revisões em planos e produtos de trabalho do projeto são realizadas visando a identificar e corrigir inconsistências em relação aos requisitos;
- GRE5 - Mudanças nos requisitos são gerenciadas ao longo do projeto.

2.2 Engenharia de Software Baseada em Evidências

Os trabalhos de Cochrane (2003) *apud* Mafra e Travassos (2006) já sustentavam a necessidade de adotar uma metodologia baseada em evidência na pesquisa científica em engenharia de software, devido a observação de experiências na área da Medicina, que durante um longo período os trabalhos gerados não utilizavam métodos para identificar, avaliar e sintetizar as informações coletadas em outros estudos disponíveis na literatura. Esta situação foi criticada por várias publicações na década de 80, onde evidenciavam que uma consequência de não organizar as pesquisas médicas a partir de revisões sistemáticas poderia ser a morte de indivíduos. Desde então, a comunidade de Medicina passou a adotar métodos formais para a condução de revisão sistemática, fato

comprovado pela grande quantidade de revisões formais publicadas a cada ano na área médica (NHSCRD, 2003).

Na área de Engenharia de software o uso de métodos formais de revisões sistemáticas da literatura começou a ser adotado por volta de 1980 e desde então diversos trabalhos vêm sendo publicados ao longo dos anos utilizando este procedimento. As abordagens baseadas em evidências possibilitaram descrever o uso de novas tecnologias, o que permitia identificar com certo nível de segurança o que funciona e o que não funciona (Mafra e Travassos, 2006).

Segundo Kitchenham e Charters (2007), a engenharia de software baseada em evidências utiliza meios que permitem integrar as evidências extraídas de pesquisa com experiência prática e valores humanos no processo de tomada de decisão para o desenvolvimento e a manutenção do software. A partir desta afirmação, é definido um caráter experimental para a engenharia de software que seria o uso da abordagem científica para o desenvolvimento, evolução e manutenção de software.

De acordo com Mafra e Travassos (2006), para se atingir um nível considerável de evidências a respeito da caracterização de uma determinada tecnologia em uso, a engenharia de software deve buscar na literatura dois tipos de estudos: estudos primários e estudos secundários. Os estudos primários são aqueles que descrevem as características de uma tecnologia em uso dentro de um contexto específico, como por exemplo experimentos, estudos de caso e *surveys*. Já os estudos secundários são aqueles que buscam identificar, avaliar e interpretar todos os resultados relevantes a um determinado tópico de pesquisa, fenômeno de interesse ou questão de pesquisa a partir da análise dos estudos primários, como por exemplo revisões e mapeamentos sistemáticos da literatura. Os resultados obtidos em estudos primários atuam como fonte de informação para estudos secundários. Esta relação é apresentada na Figura 2.3.

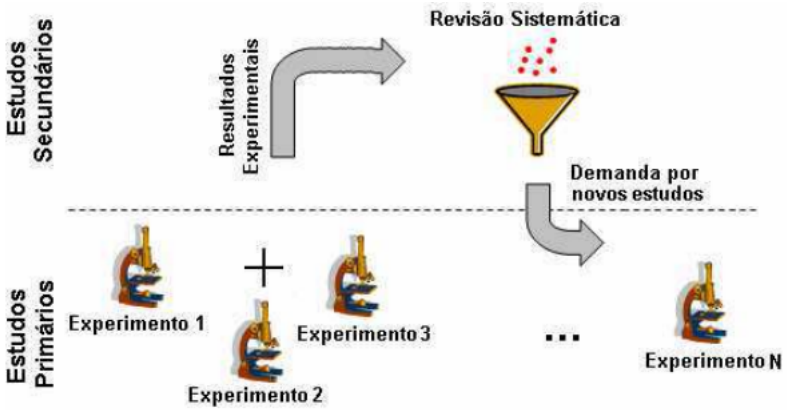


Figura 2.2 Relação entre Estudos Primários e Estudos Secundários (Barcelos e Travassos, 2006).

Um dos principais métodos da Engenharia de Software Baseada em Evidências é a Revisão Sistemática da Literatura (RSL), classificada como um estudo secundário, que tem como propósito avaliar e interpretar todo estudo disponível e relevante a respeito de uma questão de pesquisa, um tópico ou um fenômeno, o qual supostamente apresenta uma avaliação justa deste tópico de pesquisa à medida que utiliza uma metodologia de revisão rigorosa, confiável e aditável (Mafra e Travassos, 2006).

Um Mapeamento Sistemático da Literatura também tem como propósito avaliar toda a pesquisa disponível e relevante a respeito a uma questão de pesquisa, um tópico ou um fenômeno de interesse, no entanto de maneira mais abrangente e superficial do que uma RSL. As evidências de um domínio podem estar descritas em um alto nível de granularidade, permitindo a identificação de conjunto de provas e evidências de lacunas que podem ser usados para direcionar o foco de novas revisões sistemáticas ou identificar temas que para novos estudos primários podem ser conduzidos (Kitchenham e Charters, 2007).

Neste contexto, pode-se considerar que a condução de estudos secundários proporciona precisão e confiabilidade para a identificação de novos tópicos de pesquisas a serem investigados, além de outras vantagens como: a) utilização de uma metodologia bem definida; b) minimizar a parcialidade dos pesquisadores envolvidos; c) em estudos quantitativos, é possível combinar dados usando técnicas meta-analíticas; dentre outras (Kitchenham e Charters, 2007).

Na próxima subseção, o método de RSL é descrito, tendo em vista que foi o método aplicado neste estudo para o levantamento e análise de evidências empíricas que forneceram insumos para a criação da abordagem ágil de gerência de requisitos.

2.2.1 Método de Revisão Sistemática da Literatura

De acordo com Schütz, Sant'ana e Santos (2011), a RSL consiste em um método que tem base em critérios pré-determinados e evidências científicas consistentes, que quando seguidos colaboram para a seleção de estudos e/ou ferramentas para o desenvolvimento de trabalhos com informações originais. "Métodos sistemáticos são usados para evitar viés e possibilitar uma análise mais objetiva dos resultados, facilitando uma síntese conclusiva" (Sampaio e Mancini, 2007).

Para Kitchenham e Charters (2007), o método de RSL possibilita identificar, avaliar a qualidade e interpretar estudos sobre um tema em questão, assim como sua aplicabilidade. Com base nessa premissa, este tipo de método fornece mecanismos para uma revisão abrangente e não tendenciosa, com o objetivo de apresentar uma avaliação confiável, rigorosa e auditável a respeito de um tópico de investigação.

Uma boa RSL deve ter o intuito de realizar pesquisa exaustiva na literatura, em busca de evidências que corroborem com uma determinada hipótese, ou simplesmente obter um conhecimento aprofundado a respeito de um fenômeno de interesse. Para isso, a RSL utiliza estudos que foram publicados e validados relacionados a um tema de interesse, como: os estudos primários, estudos de natureza experimental e resultados obtidos com pesquisas e experimentação que utilizaram diferentes métodos, como *surveys*, estudo de caso e experimentos (Mafra e Travassos, 2006).

Uma boa RSL deve possuir a característica da abrangência, tendo em vista que deve coletar todos, ou pelo menos a maioria dos estudos referentes a um tópico relevante a pesquisa. Para tal, a RSL deve estar baseada em uma questão de pesquisa bem formulada que se possa responder, pois guiará quais estudos serão incluídos, que estratégia deve ser usada para a coleta dos estudos e quais dados devem ser extraídos de cada estudo (Counsell, 1997). A execução de uma RSL deve ser feita em cinco fases consecutivas: (1) formulação da questão, (2) localização dos estudos, (3) avaliação e

seleção dos estudos, (4) análise e síntese e (5) relatar e usar os resultados (De Medeiros *et al.*, 2014; Garza-Reyes, 2015).

Antes de iniciar a revisão, deve ser desenvolvido um protocolo que elimine o caráter tendencioso que possa haver de seus pesquisadores, possibilitando a replicação, por possuir uma estrutura de revisão definida anteriormente. Este protocolo deve estar fortemente baseado nas questões de pesquisa do estudo, para servir como principal diretriz na realização dos seguintes passos: (1) operacionalização dos principais conceitos; (2) identificação das palavras-chave que formularam a *string* de busca; (3) identificação de critérios de inclusão; (4) identificação de critérios de exclusão; (5) rastreio de referências a partir do título e *abstracts*; (6) execução da síntese; e, por fim, (7) coleta dos artigos mais relevantes para análise (Mafra e Travassos, 2006).

A condução de RSL permite que o pesquisador identifique conhecimentos específicos relacionados a sua pesquisa, seguindo um planejamento que evita duplicação de esforços e a repetição de erros passados (Mafra e Travassos, 2006). Desse modo, a RSL difere-se de uma revisão informal nos seguintes aspectos:

- **Resultados Inconsistentes:** segundo Biolchini *et al.* (2005), nas revisões informais a presença de inconsistência é um fator negativo, já que dificulta a análise e julgamento do pesquisador. Já nas revisões sistemáticas, esta inconsistência é um fator de incentivo para o pesquisador, pois estas inconsistências podem ser percebidas, o que pode auxiliar no apoio ou não de suas hipóteses;
- **Propósito:** segundo Mafra e Travassos (2006), uma revisão sistemática não é apenas uma revisão da literatura estruturada a partir de um planejamento. A revisão de literatura é parte de uma revisão sistemática, ou seja, o objetivo de um RSL é maior do que o de uma revisão de literatura informal, pois o objetivo de uma revisão sistemática é a análise dos dados coletados com o propósito da geração de evidências para o fenômeno em questão.

Na área de engenharia de software ainda é mais frequente o uso das revisões da literatura, executadas de maneira informal, sem planejamento a critérios estabelecidos, o que pode gerar uma revisão pouco abrangente, não auditável e pouco confiável, pois o

resultado pode sofrer de viés dos pesquisadores. Aspectos que podem ser combatidos com o uso de RSL, visto que a mesma estabelece um protocolo formal, rigoroso e passível de repetição. Vale ressaltar que o objetivo da RSL não é apenas o agrupamento de estudos referentes a um tópico pesquisado, é também uma abordagem metodológica para realizar pesquisas com fins experimentais (Mafra e Travassos, 2006).

Kitchenham e Charters (2007) definem um processo para a condução de uma RSL dividido em três fases, as quais serão melhor descritas nas próximas subseções.

2.2.1.1 Fase de Planejamento da Revisão

Na fase de planejamento é realizada a estruturação de um protocolo para a execução da RSL, devem ser definidos alguns pontos importantes que guiarão todo o procedimento de revisão, tais como a definição da questão de pesquisa, critérios objetivos para a avaliação dos estudos, métodos de seleção, avaliação e extração. Dyba *et al.* (2007) consideram a fase de planejamento a mais importante para a condução de revisão sistemática, pois é nesta etapa que são formuladas as questões de pesquisa que devem ser respondidas com base na RSL, o que torna todos os outros aspectos do processo de revisão dependentes delas.

Para Kitchenham e Charters (2007), antes de iniciar uma revisão sistemática, os pesquisadores devem assegurar que a revisão é necessária e que o protocolo deve ser capaz de responder as seguintes questões:

- Quais são os objetivos desta revisão?
- Que fontes foram pesquisadas para identificar os estudos primários? Houve alguma restrição?
- Quais foram os critérios de inclusão / exclusão e como foram aplicados?
- Que critérios foram utilizados para avaliar a qualidade dos estudos primários?
- Como foram aplicados os critérios de qualidade?
- Como os dados foram extraídos dos estudos primários?
- Como os dados foram sintetizados?
- Quais foram as diferenças entre os estudos pesquisados?
- Como os dados foram combinados?

Segundo Travassos e Biolchini (2007), a partir de alguns passos, confirmam e acrescentam novas informações sobre o que a fase de planejamento deve contemplar:

- Objetivos da pesquisa devem ser listados;
- Questões de pesquisa formuladas (*strings* de busca preparadas);
- Métodos que serão utilizados para executar a revisão e analisar os dados obtidos devem ser definidos;
- As fontes e a seleção de estudos devem ser planejadas;
- Um protocolo de revisão deve ser definido, documentado e disponibilizado.

Um ponto importante que deve ser realizado após o planejamento é a avaliação do protocolo gerado para garantir a viabilidade da pesquisa. É sugerido que essa avaliação seja feita por um ou mais especialistas em revisões sistemáticas.

2.2.1.2 Fase de Condução da Revisão

A fase de condução da revisão pode ser dividida nas seguintes fases: busca primária; seleção dos estudos primários; avaliação da qualidade dos estudos primários; e extração dos dados.

A fase de busca primária é definida pela utilização de *strings* de busca nas fontes de pesquisa definidas, tendo como resultado um conjunto de estudos primários potenciais para a pesquisa.

A fase de seleção dos estudos primários é onde ocorre a execução do processo de seleção definido no protocolo de revisão, com a aplicação dos critérios de inclusão e exclusão pré-definidos nos estudos retornados da busca primária, tendo como resultado o conjunto de estudos que serão utilizados na pesquisa.

A fase de avaliação da qualidade dos estudos primários ocorre após a seleção, onde estes são avaliados inicialmente com base em critérios de qualidade, previamente estabelecidos. Depois desta avaliação é atribuída uma nota final para cada estudo e com base em uma tabela com faixas de notas é atribuído um nível de qualidade.

A fase de extração dos dados pode ser realizada por meio de formulários, podendo ser apoiada por uma ferramenta computacional para a categorização destes dados obtidos dos estudos primários selecionados.

Para Travassos e Biolchini (2007), o processo para realizar a etapa de condução de uma revisão sistemática é definido pelos seguintes passos:

- Realização das Buscas nas fontes definidas: o processo deve ser transparente, repetível e documentado, assim como as mudanças que ocorrem no processo;
- Seleção dos Estudos Primários com os critérios de inclusão e exclusão definidos;
- Extração dos dados, desde informações gerais dos estudos às respostas para as questões de pesquisa. Formulários são um bom meio para registrar todos os dados necessários e o uso de uma ferramenta computacional pode apoiar a extração e registro dos dados e posterior análise;
- Avaliação da qualidade dos estudos é importante para balancear a importância de diferentes estudos, reduzir o viés (tendência a produzir "resultados tendenciosos" que se separam sistematicamente dos resultados verdadeiros), maximizar a validade interna e externa e guiar recomendações para pesquisas futuras;
- Síntese dos dados é realizada de acordo com as questões de pesquisa, por meio de quadros para realçar as similaridades e diferenças entre estudos. Se dados quantitativos estão disponíveis, pode-se considerar fazer uma meta-análise.

2.2.1.3 Fase de Apresentação da Revisão

Na fase de apresentação da revisão os resultados são sumarizados, com base na análise e na síntese dos dados realizadas durante a etapa de extração. A sumarização pode ser feita a partir da escrita de um relatório, onde devem ser apresentados os resultados com informações tabuladas e por meio de gráficos, pelos quais auxiliam ou respondem as questões de pesquisa.

Vale ressaltar que as etapas apresentadas não, necessariamente, precisam ser realizadas de forma sequencial, algumas delas podem ser realizadas de forma paralela.

2.3 Engenharia de Requisitos

De acordo com Pressman e Maxin (2016), dentre as áreas de conhecimento da

engenharia de software, a engenharia de requisitos possui grande importância no ciclo de vida de projetos de software, por ser responsável por fornecer os meios adequados para entender o que o cliente deseja, analisar suas necessidades, analisar o que é possível ser feito, negociar uma solução razoável, especificar uma solução sem ambiguidade, validar a especificação e gerenciar os requisitos conforme eles transformem-se em um software em operação.

Segundo o SWEBoK (*Software Engineering Body of Knowledge*) (IEEE, 2014), um requisito é compreendido como uma propriedade que o software deve ter para solucionar algum problema no mundo real, podendo ser descrito de duas formas: (i) uma condição ou capacidade necessária para que um usuário resolva um problema ou alcance um objetivo; (ii), uma condição ou funções que devem ser atendidas por um produto ou componente de produto para satisfazer um contrato, um padrão, uma especificação ou quaisquer documentos estabelecidos formalmente; ou (iii) uma representação documentada de uma condição, capacidade ou função. Desta forma, o processo para descobrir, analisar, gerenciar e validar esses requisitos, a fim de obter um produto de software que atenda aos objetivos dos usuários, é denominado de Engenharia de Requisitos (Sommerville, 2010).

A engenharia de requisitos ocorre quando é possível compreender, por meio de tarefas técnicas, os requisitos básicos para o desenvolvimento de um determinado produto, ou seja, foi possível analisar, identificar e organizar os elementos que o software deve possuir conforme as necessidades dos usuários e condições da organização. É por meio da engenharia de requisitos que se compreende o produto de software, considerando as atividades para criar e manter os requisitos de sistemas (Pressman e Maxin, 2016).

Entretanto, ao realizar uma análise em algumas das principais definições a respeito das atividades fundamentais da engenharia de requisitos presentes na literatura (Pressman e Maxin, 2016; IEEE, 2014; Wiegers *et al.*, 2013; Sommerville, 2010), identifica-se a existência de diferentes nomes para as atividades, algumas atividades definidas por pesquisadores diferentes possuem correlação em suas descrições de finalidades. O Quadro 2.2 apresenta uma análise da correlação entre as atividades de engenharia de requisitos segundo alguns dos principais autores na literatura

especializada.

Quadro 2.2 Correlação de atividades que compõem a Engenharia de Requisitos.

Pressman e Maxin (2016)	**SWEBoK (IEEE, 2014)**	**Wiegers *et al.*, (2013)**	**Sommerville (2010)**
Inception	---	---	*Feasibility Study*
Elicitation	*Elicitation*	*Elicitation*	*Analysis and Elicitation*
Negotiation	*Analysis*	*Analysis*	
Elaboration			
Specification	*Specification*	*Specification*	*Specification*
Validation	*Validation*	*Validation*	*Validation*
Management	*Management*	*Management*	*Management*

Dentro deste contexto, a engenharia de requisitos é formada por dois agrupamentos de atividades que estão relacionadas: i) o grupo de Desenvolvimento de Requisitos (DRE), que são as atividades de elicitação, análise, especificação e validação de requisitos; e ii) o grupo da Gerência de Requisitos (GRE), que inclui as tarefas de, acompanhar estado dos requisitos, controlar a versão, de manter a rastreabilidade bidirecional, gerenciar as mudanças (SEI, 2010; SOFTEX, 2016a; Wiegers *et al.*, 2013). O primeiro caracteriza-se por conter as atividades mais relacionadas com a construção do software em si, enquanto que o segundo caracteriza-se como um processo mais de suporte à construção de software. A Figura 2.4 exemplifica melhor este agrupamento. Inclusive, o CMMI-DEV categoriza o DRE como um processo de Engenharia, enquanto que o GRE é classificado como um processo de Gestão.

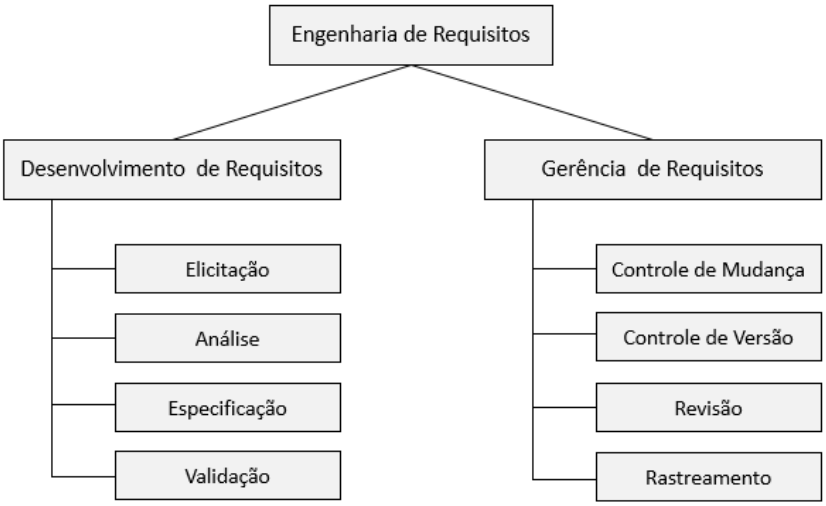


Figura 2.3 Atividades da Engenharia de Requisitos.

2.3.1 Processo de Gerência de Requisitos

De acordo com Pressman e Maxin (2016), o processo de gerência de requisitos ajuda a equipe de desenvolvimento na identificação, controle e rastreamento dos requisitos, além de apoiar no gerenciamento de mudanças dos requisitos durante o projeto de desenvolvimento do produto de software.

O trabalho de Wiegers *et al.* (2013) aponta que o processo de gerência de requisitos deve incluir as seguintes atividades: controle de mudanças, controle de versão, acompanhamento do estado dos requisitos e rastreamento de requisitos.

Para Sommerville (2010), os principais objetivos da gerência de requisitos são: gerenciar alterações nos requisitos acordados, gerenciar relacionamentos entre requisitos e gerenciar dependências entre requisitos e outros documentos produzidos durante o processo de software. O processo de GRE abrange atividades fundamentais em projetos de desenvolvimento de software. Devido os requisitos constituírem a base para a definição do sistema, é importante que haja tarefas para garantir a qualidade destes artefatos (Machado, 2011).

O principal objetivo do processo GRE é controlar a evolução dos requisitos do produto e dos componentes do produto, gerenciando todos os requisitos recebidos e gerados pelo projeto, o que incluem requisitos funcionais e não-funcionais, bem como

os requisitos impostos ao projeto pela organização. A gerência dos requisitos fornece apoio para o planejamento e a execução do projeto de maneira adequada. Quando os requisitos são levantados no início do projeto ou por solicitação de mudança, estes devem ser revisados para prevenir o mau entendimento da demanda, antes que os requisitos sejam incluídos no escopo do projeto. Para concretizar este passo, o fornecedor de requisitos e a organização devem chegar a um acordo, e os demais envolvidos no projeto devem estar comprometidos com o que foi elicitado (SOFTEX, 2016b).

O processo de GRE também possui a responsabilidade de documentar as mudanças nos requisitos e suas justificativas, bem como prover e manter a rastreabilidade bidirecional entre os requisitos e produtos de trabalhos produzidos ao longo do projeto, a fim de facilitar a identificação de inconsistências entre os requisitos, os planos e produtos de trabalhos do projeto (Machado, 2011).

2.4 Métodos Ágeis

O termo Ágil foi escolhido para representar um movimento que surgiu na década de 90 como resposta ao uso de métodos pesados para o desenvolvimento de software que eram predominantes na época, e que convencionalmente são chamados de métodos tradicionais (Sabbagh, 2014). Os métodos ágeis são alternativas para resolver alguns dos problemas encontrados nas abordagens tradicionais, e que buscam desburocratizar o processo para desenvolvimento de software, o que torna seus adeptos resilientes para enfrentar um mercado que cada vez mais exige velocidade na entrega de produto de qualidade (Qusef *et al.*, 2010; Sommerville, 2010).

Os métodos tradicionais são fortemente prescritivos, e podem ser caracterizados pelo foco em planejar de forma detalhada planos como: orçamento, escopo e cronograma na fase inicial do projeto, o que acaba tornando mudanças durante o projeto algo extremamente indesejável (Sabbagh, 2014). Um dos métodos tradicionais mais populares para o desenvolvimento de software é o modelo em cascata, ou *waterfall* (Royce, 1970). O método caracteriza-se por uma sequência de fases de desenvolvimento, onde cada fase somente inicia quando a anterior encerra-se, e os

artefatos produzidos como saída de uma fase serviam de entrada para a fase seguinte (Pressman, 2016). A Figura 2.5 representa o modelo cascata para desenvolvimento.

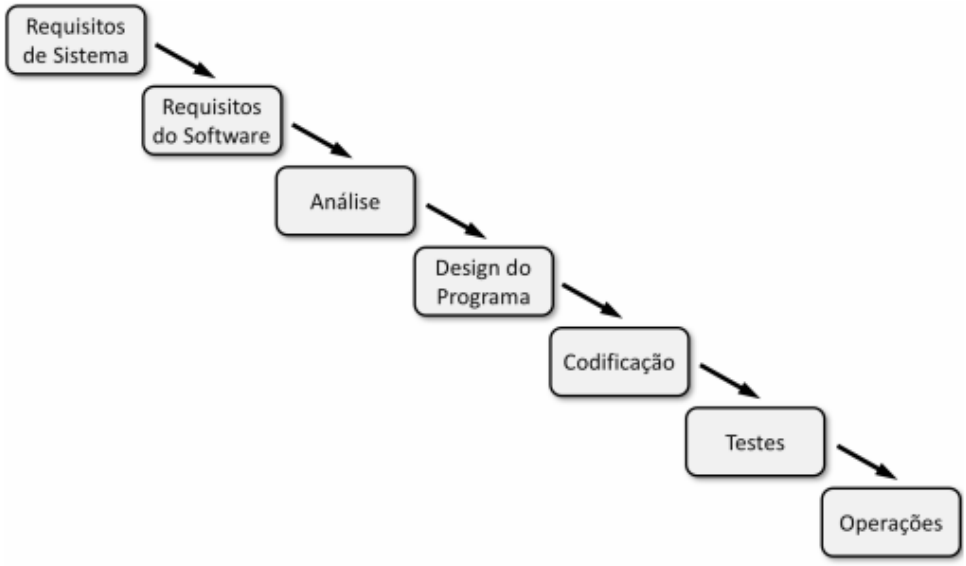


Figura 2.4 Representação do modelo cascata (Royce, 1970).

Entretanto, autores como Degrace e Stall (1990) já descreviam as razões por que os métodos tradicionais de desenvolvimento de software tendem a não funcionar, a partir das seguintes prerrogativas:

- Requisitos não são completamente compreendidos antes do início do projeto;
- Usuários só sabem exatamente o que querem após ver uma versão inicial do produto;
- Requisitos mudam frequentemente durante o processo de desenvolvimento;
- Novas ferramentas e tecnologias tornam as estratégias de desenvolvimento imprevisíveis.

Uma abordagem ágil deve seguir os princípios e valores do Manifesto para o Desenvolvimento Ágil de Software, criado em fevereiro de 2001. Esse manifesto foi assinado por dezessete especialistas em desenvolvimento de software, que diferente das práticas predominantes naquele período, estavam trazendo valor para seus clientes com o uso de abordagens leves e empíricas para a produção de software de qualidade (Agile Manifesto, 2001). Estes métodos tornaram-se área de interesse entre diversos pesquisadores e profissionais de TI (Tecnologia da Informação), devido aos diversos

benefícios que proporcionaram ao contexto atual de desenvolvimento de software, por exemplo desenvolvimento iterativo e mais colaborativo, comunicação *face-to-face*, *feedback* constante, entregas frequentes de software de valor, melhoria contínua. foco nas pessoas e no produto funcional, receptivo a mudanças no escopo, dentre outros (Heikkila *et al.,* 2015; Käpyaho e Kauppinen, 2015; Cao e Ramesh, 2008; Schwaber, 2007).

Os signatários do manifesto não tinham interesse de unificar a sua forma de trabalho ou chegar a um consenso. No entanto, apesar das diferentes práticas defendidas por cada um, os participantes encontraram um grupo de princípios e valores em comum, que definem o conceito de agilidade (Sabbagh, 2014).

O Manifesto Ágil reconhece que a utilização de processos, ferramentas, documentação, contratos e planos pode ser importante para o sucesso do projeto, porém são ainda mais importantes valores ágeis (Agile Manifesto, 2001). O Quadro 2.3 apresenta o texto de início deste manifesto ágil.

Quadro2.3 Manifesto para desenvolvimento ágil de software (Agile Manifesto, 2001).

Manifesto ágil
"Estamos descobrindo maneiras melhores de desenvolver software fazendo-o nós mesmos e ajudando outros a fazê-lo. Através deste trabalho, passamos a valorizar: ***Indivíduos e interação entre eles** mais que processos e ferramentas;* ***Software em funcionamento** mais que documentação abrangente;* ***Colaboração com o cliente** mais que negociação de contratos;* ***Responder a mudanças** mais que seguir um plano.* *Ou seja, mesmo havendo valor nos itens à direita, valorizamos mais os itens à esquerda."* *Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Roland Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland e Dave Thomas.* *© 2001, os autores acima.*

Os doze princípios Ágeis foram definidos no manifesto pelos seus autores e são apresentados em seguida:

1. Nossa maior prioridade é satisfazer o cliente por meio da entrega cedo e frequente de software com valor;

2. Aceitar mudanças de requisitos, mesmo no fim do desenvolvimento. Processos ágeis adequam-se a mudanças, para que o cliente possa tirar vantagens competitivas.
3. Entregar software funcionando com frequência, na escala de semanas até meses, com preferência aos períodos mais curtos.
4. Pessoas relacionadas a negócios e desenvolvedores devem trabalhar em conjunto e diariamente, durante todo o curso do projeto.
5. Construir projetos ao redor de indivíduos motivados, dando a eles o ambiente e o suporte necessários, e confiar que farão seu trabalho.
6. O método mais eficiente e eficaz de transmitir informações para e por dentro de um time de desenvolvimento é através de uma conversa cara a cara.
7. Software funcional é a medida primária de progresso.
8. Processos ágeis promovem um ambiente sustentável. Os patrocinadores, desenvolvedores e usuários, devem ser capazes de manter indefinidamente, passos constantes.
9. Contínua atenção à excelência técnica e bom design aumentam a agilidade.
10. Simplicidade, a arte de maximizar a quantidade de trabalho que não precisou ser feito.
11. Em intervalos regulares, o time reflete em como ficar mais efetivo, então se ajustam e otimizam seu comportamento de acordo.

Baseado nestes valores e princípios mencionados, o Desenvolvimento Ágil de Software (ASD - *Agile Software Development*) caracteriza-se por um conjunto de métodos e práticas ágeis utilizados para o desenvolvimento de software de qualidade de forma mais colaborativa, transparente e eficiente (Cohn, 2011; Agile Alliance, 2018). A Figura 2.6 elucida melhor a ideia de fundamentação destes métodos.

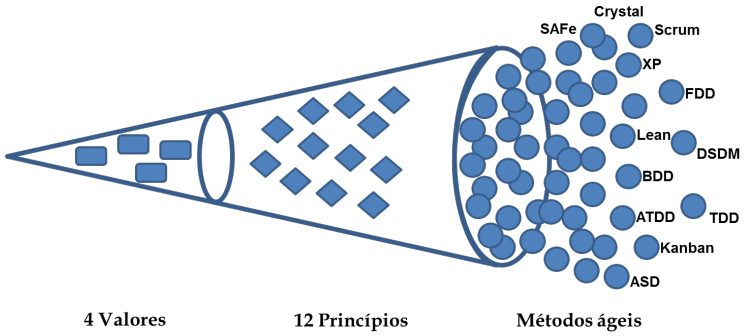


Figura 2.5 Fundamentação dos métodos ágeis.

A quantidade de novas abordagens para o Desenvolvimento Ágil de Software cresceu consideravelmente nos últimos anos, surgindo diversos, métodos, técnicas e práticas ágeis, as quais podem ser utilizadas e combinadas de diversas maneiras, de acordo com o contexto de desenvolvimento de software. Assim, permitiu aos adeptos de métodos ágeis a escolha de utilizar apenas algumas práticas definidas de um específico método ágil, por exemplo, é perfeitamente possível utilizar apenas a prática de *user stories* do XP (*eXtreme Programming*), sem a obrigatoriedade de utilizar todas as demais práticas descritas neste método (Vacari, 2015).

Por fim, alguns dos principais métodos ágeis utilizados na indústria de software são: Scrum, XP, FDD (*Feature Driven Development*), TDD, ATDD (*Acceptance Test Driven Development*), BDD (*Behavior Driven Development*), Kanban, Crystal, Lean, ASD (*Adaptative Software Development*), DSDM (*Dynamic systems development method*), dentre outros (Prikladnicki *et al.*, 2014).

2.5 Engenharia de Requisitos Ágil

Uma área que está ganhando destaque na indústria de software é a Engenharia de Requisitos Ágil (Agile-RE - *Agile Requirements Engineering*), que tem sido o foco de pesquisa em estudos como de Daneva *et al.* (2015), Heikkila *et al.* (2015), Inayat *et al.* (2014) e Cao e Ramesh (2008). Esta área busca aplicar os princípios e os valores dos métodos ágeis nas atividades de Engenharia de Requisitos, ou seja, esta abordagem permite elicitar, analisar, especificar, validar e gerenciar os requisitos de forma mais ágil, transparente e orientado aos *Customers* (Cao e Ramesh, 2008; Inayat *et al.*, 2014).

Na Agile-RE as atividades de Engenharia de Requisitos acontecem de forma simultânea com o desenvolvimento do software, diferente das abordagens tradicionais, onde requisitos geralmente são definidos no planejamento do projeto (Daneva *et al.*, 2015), por exemplo não é preciso elicitar todos os requisitos nas fases iniciais do projeto, apenas o suficiente para obter um entendimento geral do produto a ser produzido, e junto ao desenvolvimento, os requisitos vão sendo refinados (Heikkila *et al.*, 2015). Além disso, a Agile-RE diferencia-se da Engenharia de Requisitos Tradicional quanto à flexibilidade em aceitar mudanças nos requisitos, mesmo em estágios finais do desenvolvimento do produto; produzir documentações suficientes e não muito detalhadas; bem como, pelo fato de evoluir e detalhar os requisitos ao longo de curtas iterações de desenvolvimento (Qusef *et al.*, 2010; Paetsh *et al.*, 2003).

Outro ponto importante sobre a Agile-RE refere-se às suas práticas, as quais consistem em um conjunto de ações recomendadas para serem seguidas e aplicadas nos métodos ou técnicas utilizados pelas equipes de desenvolvimento em seus projetos de software. No Quadro 2.4 é possível verificar algumas destas práticas da Agile-RE, segundo os estudos conduzidos por Inayat *et al.,* (2014), Bjarnason *et al.* (2011) e Cao e Ramesh (2008).

Quadro 2.4 Práticas da Agile-RE.

ID	Práticas
P01	*Customer on-site.*
P02	*Face-to-face communication over written specifications;*
P03	*Gradual & Iterative Detailing of Requirements;*
P04	*Requirement prioritization goes extreme;*
P05	*Prototyping;*
P06	*Reviews and Acceptance tests;*
P07	*Agile Modeling*
P08	*Change management*
P09	*Test-driven development*
P10	*Pairing for requirements analysis*
P11	*Shared conceptualisations*

Embora existam estudos sobre a Agile-RE, a comunidade ágil ainda não possui um conhecimento consolidado e abrangente a respeito desta área, ou seja, há poucas evidências empíricas sobre este tema ou as experiências estão fragmentadas na literatura, geralmente focando em apenas um aspecto ou atividade da Engenharia de Requisitos, como por exemplo a priorização dos requisitos (Daneva *et al.*, 2015).

Contudo, realizar as atividades de Engenharia de Requisitos no contexto dos métodos ágeis ainda é considerado um grande desafio (Inayat *et al.,* 2014; Bjarnason *et al.*, 2011). Pode-se citar alguns exemplos de desafios da Agile-RE, os quais foram identificados a partir dos estudos de Medeiros *et al.* (2015), Inayat *et al.* (2014) e Jaqueira *et al.* (2012), todos conduzidos por meio de mapeamentos ou revisões sistemáticas da literatura com o objetivo de investigar sobre a Agile-RE, conforme apresentados no Quadro 2.5.

Quadro 2.5 Exemplos de Desafios da Agile-RE.

ID	Desafios
D01	Pouca disponibilidade do cliente.
D02	Requisitos não-funcionais são negligenciados.
D03	Controle de mudanças nos requisitos ineficiente.
D04	Rastreabilidade de requisitos ineficiente.
D05	*User stories* são inadequadas para descrever aspectos técnicos.
D06	Consenso na negociação ou priorização dos requisitos.
D07	Documentação geralmente mínima.
D08	Ineficiência na análise e inspeção dos requisitos
D09	Comunicação entre o cliente e a equipe.
D10	Custos e estimativas do projeto devido às mudanças constantes nos requisitos.

2.6 Método REACT para Desenvolvimento de Requisitos

O REACT (*Requirements Evolution in Agile ContexT*) é um método ágil proposto por Santos e Oliveira (2018), para dar suporte ao processo de desenvolvimento de requisitos de software, empiricamente fundamentado em outras abordagens ágeis encontradas na literatura especializada, nas práticas da Agile-RE, e nas boas práticas constantes nos modelos CMMI-DEV e MR-MPS-SW, especificamente em seus

respectivos processos de *Requirements Development* e Desenvolvimento de Requisitos. Esta abordagem ágil tem como objetivo evoluir as expectativas e as necessidades de negócio dos clientes ao longo do desenvolvimento de um produto de software. O REACT possui um ciclo de vida composto por 5 cerimônias, 4 papéis e 11 artefatos, os quais podem ser visualizados na Figura 2.7.

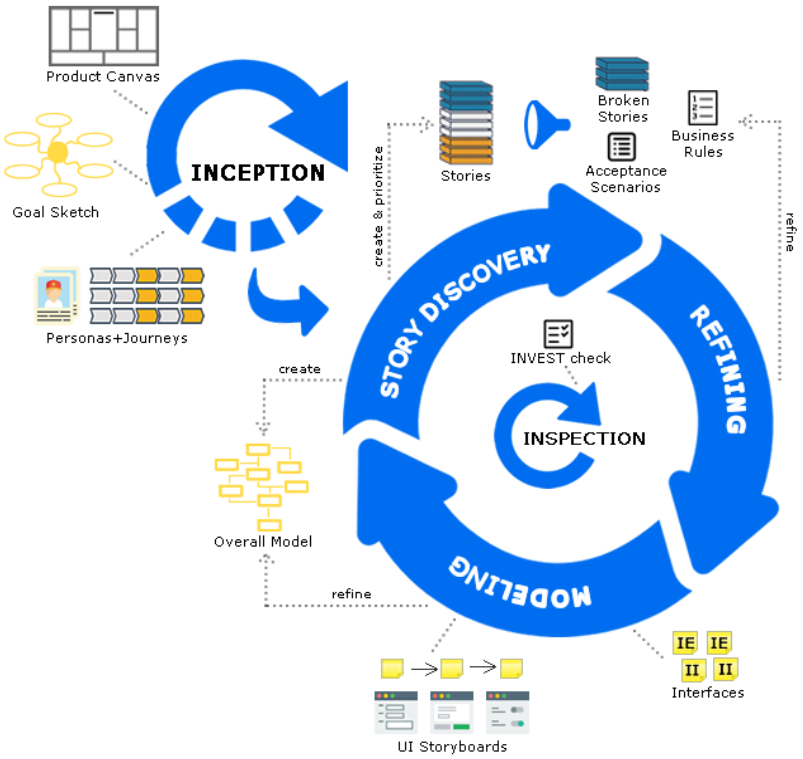


Figura 2.6 Ciclo de Vida do REACT (Santos e Oliveira, 2018).

De modo geral, a primeira cerimônia do REACT consiste em estabelecer uma visão do produto pretendido, definindo restrições, metas e requisitos do cliente em alto nível. Após isso, entra-se em um ciclo contínuo e iterativo de 4 cerimônias a fim de evoluir os requisitos do produto ao longo de todo o projeto de software. É recomendado que estas 4 cerimônias aconteçam de forma iterativa, ou seja, dentro de uma curta iteração de desenvolvimento, típico em ambientes ágeis de desenvolvimento de software (Santos e Oliveira, 2018).

É importante lembrar que o objetivo do REACT é de elicitar, analisar, especificar e validar os requisitos de software, não tendo como objetivo gerenciar requisitos, entregar software ou prover um modelo de ciclo de vida de todo o projeto de

desenvolvimento de software. Deste modo, é recomendado que o REACT seja utilizado com outros métodos ágeis que deem suporte a outros processos do desenvolvimento de software (Santos e Oliveira, 2018).

Dentro deste contexto, o presente trabalho pretende ser uma evolução do método REACT, estendendo-se para o processo de GRE e as práticas de Agile-RE, e fundamentado em evidências empíricas presentes na literatura. O método construído neste trabalho foi denominado de REACT-M (REACT- *Managemen*t), e apesar de ser aderente ao REACT para DRE, a abordagem também pode ser utilizada por outros métodos para desenvolvimento ágil de requisitos, possibilitando que seus adeptos possam usá-lo de acordo com suas necessidades.

2.7 Trabalhos Relacionados

O trabalho de Medeiros (2015) conduz uma revisão sistemática da literatura com o intuito de identificar como a engenharia de requisitos está sendo tratada em projetos ágeis de desenvolvimento de software. O trabalho identificou diversas abordagens ágeis para apoio às atividades de engenharia de requisitos, como por exemplo as histórias de usuário e os cenários de aceitação. No entanto, o estudo apenas discorre sobre técnicas de elicitação e especificação dos requisitos, deixando de lado outras atividades importantes, como as de gerência de requisitos.

O estudo de Inayat et al. (2014) desenvolveu uma revisão sistemática da literatura a fim de identificar práticas ágeis voltadas à engenharia de requisitos e avaliar quais os desafios estão relacionados a estas práticas, como por exemplo a negligência de requisitos não-funcionais, a documentação insuficiente e a pouca disponibilidade do cliente. A partir da RSL desenvolvida, os pesquisadores propuseram um método que contemple algumas das principais atividades de DRE, como elicitação, especificação e validação.

O estudo conduzido por Sehrish e Shah (2017) realizou uma revisão sistemática da literatura com o objetivo de apresentar as limitações encontradas pela engenharia de requisitos em ambientes ágeis e interpretar quais são os problemas e desafios que uma pessoa ágil enfrenta na implementação de práticas ágeis. Os resultados contribuem para

um melhor entendimento sobre como resolver alguns problemas particulares de requisitos no processo de desenvolvimento de software com métodos ágeis

Embora os três estudos tenham como um dos objetivos a identificação de práticas ágeis para a engenharia de requisitos, este trabalho difere-se por também abranger métodos e técnicas ágeis para apoio à gerência de requisitos em sua revisão sistemática de literatura. Além disso, os estudos relacionados não buscam definir um processo que cubra todas as principais atividades de engenharia de requisitos, deixando de lado diversas atividades importantes quando se trabalha com requisitos em projetos de software, principalmente quando se trata de atividades direcionadas para a gerência de requisitos. Portanto, o presente estudo é uma extensão do trabalho de Santos e Oliveira (2018), e tem como objetivo a identificação e o endereçamento de abordagens ágeis para as atividades de gerência de requisitos constantes nos modelos de qualidade CMMI-DEV e MR-MPS-SW, e para construção de um método que guie seus adeptos a utilizar um processo de GRE em ambiente ágil ou em transição para o ágil.

É importante ressaltar que os três trabalhos relacionados seguiram os *guidelines* definidos por Kitchenham e Charters (2007), assim como o presente estudo para a condução de revisões sistemáticos, permitindo uma revisão da literatura não informal, de forma mais rigorosa e confiável.

3 UMA REVISÃO SISTEMÁTICA DA LITERATURA SOBRE ENGENHARIA DE REQUISITOS E MÉTODOS ÁGEIS

Este capítulo apresenta uma Revisão Sistemática da Literatura (RSL), que teve como objetivo identificar quais são as abordagens existentes nas metodologias ágeis para apoio as atividades de Engenharia de Requisitos dentro do contexto de projetos de software. A metodologia usada para condução e extração de resultados da RSL são apresentadas neste capítulo, e estão divididas nos seguintes tópicos:

- Metodologia da RSL;
- Estratégia da RSL;
- Sumarização dos Resultados da RSL;

Vale ressaltar que a RSL conduzida neste trabalho preocupou-se em coletar evidências empíricas dentro do período não coberto pelo trabalho de Santos e Oliveira (2018), justificado pois a RSL conduzida pelos autores possui o mesmo objetivo e utiliza o mesmo protocolo para condução e extração de resultados. Deste modo, a fundamentação do REACT-M leva em consideração os resultados coletados nas RSL de ambos os estudos, o que cobre um período de 17 anos, iniciado em 2001, ano de criação do manifesto ágil.

3.1 Metodologia da Revisão Sistemática

Este estudo conduziu uma RSL seguindo as orientações propostas por Kitchenham e Charters (2007), por ser amplamente adotada como referência para conduções de RSL na Engenharia de Software (Dyba e DingsoyR, 2008). A RSL é uma forma de estudo secundário com o objetivo de identificar e avaliar trabalhos disponíveis para uma determinada questão de pesquisa e, em um certo período de tempo

(Kitchenham e Charters, 2007). Deste modo, este estudo restringe-se ao intervalo entre 01/2016 até 06/2018, devido ser o período ainda não coberto pelo trabalho de Santos e Oliveira (Santos e Oliveira, 2018), a respeito da área de Engenharia de Requisitos Ágil. A Figura 3.1 apresenta o processo de RSL proposto por Kitchenham e Charters (2007), o qual foi utilizado por este estudo. Vale lembrar que este processo foi descrito em mais detalhes na Seção 2.2.1 desta dissertação.

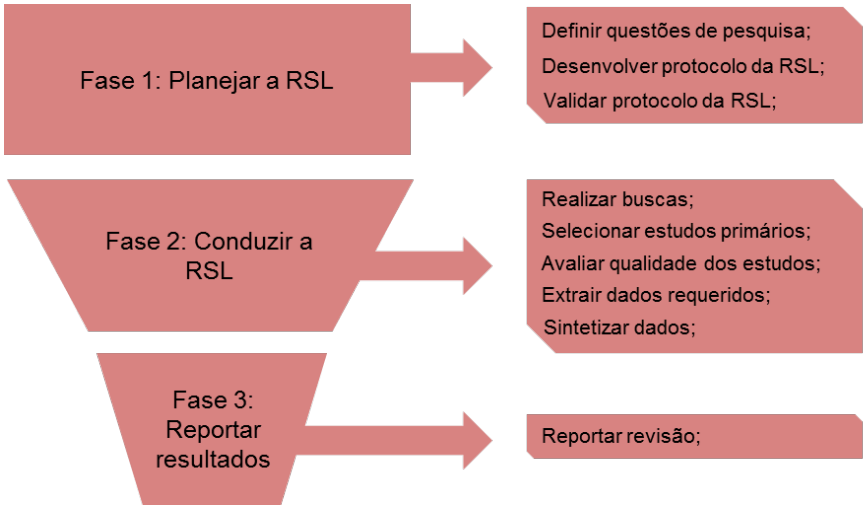


Figura 3.1 Processo para condução de RSL (Kitchenham e Charters, 2007).

Para um melhor entendimento sobre a metodologia aplicada para a condução da RSL desta dissertação, foi elaborado o Quadro 3.1 que sintetiza as recomendações de Cruzes e Dyba (2011), que sugerem o uso da técnica de síntese e análise temática; e de Marconi e Lakatos (2003), que definem variáveis independentes e dependentes de pesquisa.

Quadro 3.1 Quadro metodológico da RSL.

Método de pesquisa	Revisão Sistemática da Literatura
Natureza dos dados	Qualitativa
Técnica para análise dos dados	Síntese e Análise Temática.
Variáveis independentes	Engenharia de Requisitos e Métodos Ágeis
Variáveis dependentes	Práticas, Técnicas, Métodos, Metodologias

Ressalta-se que a revisão sistemática deve ser conduzida a partir de um protocolo pré-estabelecido, o que busca garantir que a RSL possua efetivo valor científico e possibilidade de repetição do seu processo, pois caso isso não seja prezado, a revisão torna-se informal, podendo sofrer de análise parcial dos pesquisadores que a estão conduzindo, o que diminui o grau de confiabilidade das mesmas. O protocolo utilizado na RSL deste estudo pode ser encontrado no Apêndice A.

Outro ponto importante é o fato da equipe revisora ter sido composta por 2 pesquisadores, sendo um mestrando e um doutor orientador deste estudo. Todas as etapas foram realizadas em colaboração entre os pesquisadores. É importante citar que todas as etapas desta RSL foram acompanhadas e validadas pelo pesquisador e orientador, o qual possui notável experiência em revisões sistemáticas. Esta etapa da dissertação durou nove meses, executada no período de abril/2018 até dezembro/2018.

Nas próximas seções, a estratégia planejada e definida no protocolo da RSL (Apêndice A) para este estudo é descrita com mais detalhes, bem como, os resultados desta RSL.

3.2 Estratégias da Revisão Sistemática

O protocolo da RSL (Apêndice A) é o documento no qual encontra-se o planejamento para a condução de uma revisão sistemática. Neste protocolo, os objetivos, o escopo, as restrições, os critérios de seleção e avaliação, dentre outras estratégias para que uma revisão sistemática seja conduzida com sucesso, são definidas.

3.2.1 Objetivo da Revisão

Esta revisão sistemática teve como objetivo de identificar e avaliar quais abordagens ágeis, existentes na literatura, apoiam as atividades de Engenharia de Requisitos de software, buscando obter um melhor entendimento de como os requisitos evoluem no contexto dos métodos ágeis. Desta forma, têm-se a seguinte estrutura para o objetivo, conforme proposto em (Santos, 2010):

- **Analisar:** relatos de experiência e publicações científicas a partir de um estudo baseado em revisão sistemática;

- **Com o propósito de:** Identificar abordagens existentes nos métodos ágeis para apoio ao processo de Engenharia de Requisitos;
- **Com relação à:** definição e uso de métodos, técnicas e práticas, no contexto dos métodos ágeis, empregados para a implantação e execução das atividades de Engenharia de Requisitos em projetos de desenvolvimento de software;
- **Do ponto de vista:** de pesquisadores e organizações desenvolvedoras/mantenedoras de software;
- **No contexto:** acadêmico e industrial.

3.2.2 Questão de Pesquisa Principal

O principal objetivo da RSL é identificar trabalhos que apresentam, de maneira primária ou secundária, abordagens ágeis que possam ser agregadas a uma metodologia para a Gerência de Requisitos Ágil, estando em consonância com a metodologia para Desenvolvimento de Requisitos Ágil proposta por Santos e Oliveira (2018). Para elucidar o objetivo, foi estabelecida uma questão de pesquisa principal (QP). A questão levantada foi organizada conforme a estrutura *Population*, *Intervention*, *Context, Outcomes*, *Comparison* (PICOC), recomendada por Kitchenham e Charters (2007). Entretanto, apenas os itens População, Intervenção e Resultados foram considerados relevantes para a pesquisa. Nesse sentido, definiu-se a seguinte estrutura para a questão de pesquisa principal:

- **População (P):** Organizações de Software e Projetos de Software;
- **Intervenção (I):** Abordagens existentes nos métodos ágeis para apoiar as atividades de Gerência de Requisitos;
- **Resultados (O):** Métodos, técnicas e práticas ágeis para a gerência de requisitos de software.

Com base na estrutura PIO foi elabora a questão de pesquisa principal:

QP - Quais são as abordagens existentes nas metodologias ágeis que apoiam as atividades de Gerência de Requisitos no contexto de projetos de desenvolvimento de software?

Por “abordagens”, as quais pretende-se investigar, entende-se como: métodos, técnicas e práticas.

3.2.3 Questões Secundárias

Um conjunto de questões de pesquisa secundárias (QS) foram estabelecidas com o objetivo de esclarecer detalhes importantes que a revisão sistemática procura identificar e estão relacionadas diretamente com a questão de pesquisa principal (Palomino *et al*, 2016). Segundo Palomino *et al* (2016), Questões Secundárias têm o objetivo de esclarecer detalhes importantes que a revisão sistemática procura identificar e estão relacionadas diretamente com a questão de pesquisa principal; e Questões Bibliométricos descrevem informações relacionadas aos estudos primários selecionados, tais como: tipo de publicação, evolução dos estudos ao longo dos anos, dentre outros.

As QS estabelecidas desenvolvidas para este trabalho são apresentadas a seguir:

QS1 - Quais atividades de Gerência de Requisitos são apoiadas pelas abordagens encontradas?

QS2 - Caso a abordagem seja uma prática ou técnica, a mesma pertence a algum método ágil existente? Qual?

QS3 - Qual o contexto de aplicação da abordagem encontrada?

QS4 - Quais os ativos (papéis e artefatos) envolvidos nessas abordagens?

3.2.4 Escopo e Recursos da Pesquisa

Um escopo foi definido com o objetivo de garantir a viabilidade deste trabalho. O escopo é descrito abaixo:

I. Para a seleção das fontes de busca, foram definidos os seguintes critérios:
   - Disponibilidade para consultas web;
   - Disponibilidade para busca de artigos através do domínio da UFPA;
   - Disponibilidade de artigos na íntegra;
   - Presença de mecanismos que faça uso de palavras-chave;

- Relevância das fontes de busca no meio científico (fontes que são bastante utilizadas em outros estudos secundários);

II. Como restrições para a pesquisa têm-se:
    - A pesquisa não pode incorrer em ônus financeiro aos pesquisadores. Portanto, apenas foram selecionadas as fontes que possibilitam consultas de forma gratuita (também foram consideradas fontes que oferecem consultas de forma gratuita a partir do acesso pelo domínio da UFPA);
    - Foram apenas considerados os estudos obtidos por meio das fontes selecionadas e em conformidade com os critérios de inclusão e exclusão;
    - A pesquisa esteve restrita aos resultados publicados entre de janeiro de 2016 até julho de 2018, contemplando, desta forma, um período não coberto pelo trabalho de Santos e Oliveira (2018) a respeito do tema;
    - Foram considerados apenas estudos que estejam diretamente relacionados ao contexto de projetos de software, indústria de software, engenharia ou qualidade de software, e afins;
    - Foram considerados apenas artigos em inglês ou português devido à proficiência dos pesquisadores nestes idiomas;

III. Para a execução da revisão sistemática, os seguintes recursos foram utilizados:
    - Um pesquisador da UFPA (um aluno de mestrado em Ciência da Computação);
        - O aluno de mestrado, autor desta dissertação, foi o responsável pela condução da revisão sistemática, atuando nas etapas de seleção, avaliação da qualidade, análise e extração de resultados;
    - Validações sobre documentos e procedimentos da condução desta revisão sistemática foram realizadas através de reuniões com o coordenador do Projeto SPIDER e orientador deste estudo.

3.2.5 Busca Primária

Dentro desta etapa o pesquisador utilizou métodos previamente definidos para selecionar os estudos primários nas fontes de buscas utilizadas pela RSL. Esta busca pode ser feita de duas formas:

- **Buscas Manuais:** são aquelas definidas para fontes que não possuem um mecanismo de busca, ou seja, não possuem a função de consultar estudos através do uso de *string* de busca. Geralmente estas fontes são as de anais de eventos ou periódicos que não têm seus trabalhos indexados em bases de dados;
- **Buscas Automáticas:** são aquelas definidas para fontes que possuem um mecanismo de busca, ou seja, as que possuem trabalhos indexados em repositórios de estudos *online* que podem ser consultados a partir de *string* de busca.

Alguns passos foram definidos a fim de realizar as buscas automáticas, a saber:

- Seleção de Fontes de Busca;
- Identificação de Palavras-chave e sinônimos;
- Geração de *String* de Busca.

É importante ressaltar que este estudo conduziu apenas buscas automáticas, devido visar uma busca abrangente e com o objetivo de eliminar o viés de seleção. Nas próximas seções estes passos são detalhados.

3.2.6 Seleção de Fontes de Busca

Os trabalhos foram coletados em seis fontes de busca selecionadas devido à relevância ao meio científico e também pelo fato de serem bastante utilizadas em outras revisões sistemáticas. Estas fontes de busca são apresentadas no Quadro 3.2. Todas as fontes são eletrônicas e se utilizam da *string* de busca visualizada no Quadro 3.3 da Seção 3.2.8.

Quadro 3.2 Fontes de busca selecionadas.

Fonte de busca	Data da busca
IEEE Xplore	Julho, 2018.
ACM Digital Library	Julho, 2018.
Ei Compendex	Julho, 2018.
Scopus	Julho, 2018.
ISI Web of Knowledge	Julho, 2018.
Science Direct	Julho, 2018.

3.2.7 Identificação de Palavras-Chaves e Sinônimos

A partir das questões de pesquisa, as palavras-chave foram identificadas em acordo com a estrutura População, Intervenção e Resultados (PIO) para a posterior formulação da *string* de busca. Segue a listagem de palavras-chave definidas:

- **População**
  - **Inglês:** *Software Development, Software Project, Software Organization, Software Enterprise, Software Company, Software Industry, Software Institute, Software Research Group, Software Technology Center;*
  - **Português:** Desenvolvimento de Software, Projeto de Software, Organização de Software, Empresa de Software, Indústria de Software, Instituição de Software, Grupo de Pesquisa de Software, Centros de Tecnologia de Software;
- **Intervenção**
  - **Inglês:** *Requirements, Feature, Needs, User Story, Requirements Engineering, Task, Sprint, Backlog, Stakeholders, Product Owner, Participants, Team, Management, Understanding, Commitment, Approval, Traceability, Bidirectional Traceability, Inconsistencies, alignment, Project Work, Change Management, Manage Change, Change Log, Impact Analysis, Product Requirements, Product Component Requirements, Agility, Agile, Scrum, Lean, Crystal, Kanban, XP, Extreme Programming, FDD, Feature-Driven Development, TDD, Test-Driven Development, BDD, Behaviour-Driven Development, ATDD, Acceptance Test-Driven Development, ASD, Adaptive Software Development;*
  - **Português:** Requisitos, Funcionalidade, Necessidade, História do Usuário, Engenharia de Requisitos, Tarefa, Fornecedor, Cliente, Stakeholders, Participantes, Equipe, Time, Backlog, Gerência de Requisitos, Fornecedor, Entendimento, Comprometimento, Rastreabilidade, Rastreabilidade Bidirecional, Identificar Inconsistências, Gerência de Mudanças, Histórico de Mudança, Análise de Impacto,

Requisitos do Produto, Requisitos do Componente do Produto, Agilidade, Ágil, Scrum, Lean, Crystal, Kanban, XP, Programação Extrema, FDD, Desenvolvimento Orientado a Funcionalidade, TDD, Desenvolvimento Orientado a Teste, BDD, Desenvolvimento Orientado a Comportamento, ATDD, Desenvolvimento Orientado a Teste de Aceitação, ASD, Desenvolvimento Adaptativo de Software;

- **Resultados**
  - **Inglês:** *Methodology, Method, Technique, Practice;*
  - **Português:** Metodologia, Método, Técnica, Prática;

3.2.8 Geração de String de Busca

A string de busca utilizada neste estudo é apresentada no Quadro 3.3. Ressalta-se que a *string* apresentada está na sua forma genérica, durante a execução da consulta nas fontes de busca, algumas adaptações foram necessárias, devido cada fonte de busca conter regras específicas para a configuração da *string*.

O agrupamento das palavras-chave é feito a partir dos operadores <OR> e <AND>. O operador <OR> é utilizado para relacionar as palavras-chave consideradas sinônimos. O operador <AND> é utilizado para agrupar o conjunto de palavras-chave, de acordo com a estrutura PICOC (ou PIO no caso desta pesquisa) conforme sugerido por Santos (2010):

P <and> I <and> C <and> O

A *string* de busca foi validada a partir de testes nas fontes de busca a fim de refiná-la até os pesquisadores deste trabalho a considerarem eficiente para a realização das buscas dos estudos. Conforme recomendação de Kitchenham e Charters (2007), um grupo de controle com estudos primários os quais estão fortemente aderentes aos objetivos desta pesquisa foi utilizado como parâmetro neste momento para validar a *string*.

Quadro 3.2 String de Busca Genérica.

***String* de Busca**
("Software" AND ("Project" OR "Development" OR "Organization" OR "Enterprise" OR "Company" OR "Industry" OR "Institute" OR "Research Group" OR "Technology Center")) AND (("Requirements" OR "Feature" OR "Needs" OR "User Story" OR "Backlog" OR "Task" OR "Sprint") AND ("Stakeholders" OR "Product Owner" OR "Participants" OR "Team" OR "Engineering" OR "Manage" OR "Understanding" OR "Commitment" OR "approval" OR "Traceability" OR "Bidirectional Traceability" OR "Inconsistencies" OR "alignment" OR "Project Work" OR "Change" OR "Log" OR "Impact Analysis")) AND (("Agility" OR "Agile" OR "Scrum" OR "Lean" OR "Crystal" OR "Kanban" OR "XP" OR "Extreme Programming" OR "FDD" OR "Feature Driven Development" OR "TDD" OR "Test Driven Development" OR "BDD" OR "Behaviour Driven Development" OR "ATDD" OR "Acceptance Test Driven Development" OR "ASD" OR "Adaptive Software Development") AND ("Methodology" OR "Method" OR "Technique" OR "Practice"))*

3.2.9 Estratégia de Seleção dos Estudos Primários

Após a coleta dos estudos nas fontes de pesquisa, os trabalhos foram selecionados conforme os procedimentos:

I) Definição de critérios de inclusão e exclusão (Quadro 3.4), a fim de nortear os pesquisadores na avaliação dos trabalhos, além de auxiliar no rigor da pesquisa e reduzir avaliações tendenciosas (Kitchenham e Charters, 2007);

II) Selecionar estudos primários a partir da leitura de título, resumo e palavras-chaves, com o intuito de eliminar estudos que claramente não estão alinhados ao objetivo desta revisão sistemática;

III) Realizar a leitura integral dos estudos primários selecionados. É importante ressaltar que todos os procedimentos utilizados no estudo foram validados por um especialista na área de engenharia de software, o qual possui o título de doutor, e atua como professor e pesquisador da área em questão na Universidade Federal do Pará (UFPA).

Além disso, a ferramenta de gerência de referências JabRef foi utilizada para otimizar a eliminação de trabalhos duplicados.

Quadro 3.2 Critérios de inclusão e Exclusão.

ID	Critérios de Inclusão
CI1	Estudos que apresentem primária ou secundariamente abordagens, no contexto dos métodos ágeis, de apoio às atividades de Engenharia de Requisitos.
CI2	Estudos que apresentem relatos de experiência da indústria, ou pesquisas de caráter experimental ou teórico, contanto que apresentem exemplos de aplicação, descrição de experimentos ou casos reais de uso de abordagens, no contexto dos métodos ágeis, para apoio às atividades de Engenharia de Requisitos.
ID	**Critérios de Exclusão**
CE1	Estudos que não estejam disponíveis livremente para consulta ou download (em versão completa) através das fontes de busca ou através da ferramenta de busca Google (http://www.google.com.br)
CE2	Estudos que claramente não atendam as questões de pesquisa
CE3	Estudos repetidos terão apenas sua primeira ocorrência considerada.
CE4	Estudos do tipo: resumos, keynote, speeches, cursos, tutoriais, workshops e afins.
CE5	Estudos que não mencionem as palavras-chave da pesquisa no título, resumo ou nas palavras-chave do artigo, salvo trabalhos que abordem melhoria do processo de software, nos quais seja observada possibilidade do Engenharia de Requisitos e métodos ágeis serem tratados ao longo do trabalho.
CE6	Excluir se o estudo não estiver inserido no contexto de Projetos de Software, Indústria de Software ou Engenharia de Software.
CE7	Excluir se o estudo não estiver apresentado em uma das linguagens aceitas (Inglês e Português)

Foram considerados apenas trabalhos escritos em inglês e português. O primeiro justificado pela grande abrangência de periódicos e conferências internacionais que a definem como língua padrão. O segundo por ser importante considerar pesquisas de âmbito nacional, devido a relevância do Programa MPS.BR para o estudo.

3.2.10 Estratégia de Avaliação da Qualidade dos Estudos Primários.

Os estudos primários selecionados também foram avaliados de acordo com critérios de qualidade, com o intuito de mensurar a relevância e valor científico de seu conteúdo. Assim, durante a análise dos estudos primários e coleta de resultados, foram aplicados os critérios de qualidade definidos na Quadro 3.5, possibilitando um processo adicional de avaliação dos artigos, de modo a identificar trabalhos que ainda deviam ser desconsiderados da pesquisa.

Quadro 3.2 Critérios de qualidade.

Categoria	**ID**	**Critérios de Qualidade**
Introdução	CQ1	Os objetivos ou questões do estudo são claramente definidos (incluindo justificativas para a realização do estudo)?
	CQ2	O tipo de estudo está definido claramente?
Desenvolvimento	CQ3	Existe uma clara declaração dos objetivos da pesquisa?
	CQ4	O trabalho é bem/adequadamente referenciado (apresenta trabalhos relacionados ou semelhantes e baseia-se em modelos e teorias da literatura)?
Conclusão	CQ5	O estudo relata de forma clara e não ambígua os resultados?
	CQ6	Os objetivos ou questões do estudo são alcançados?
Questões de Pesquisa	CQ7	O estudo lista primária ou secundariamente as metodologias ou métodos ou técnicas ou práticas ágeis para apoiar as atividades de Gerência de Requisitos?
	CQ8	O estudo endereça explicitamente as abordagens ágeis para alguma atividade de Gerência de Requisitos?
Estudos Experimentais	CQ9	Existe um método ou um conjunto de métodos descrito para a realização do estudo?
Estudos Teóricos	CQ10	Existe um processo não tendencioso na escolha dos estudos?
Relatos de Experiência Industrial	CQ11	Existe uma descrição sobre a(s) organização(ões) onde foi conduzido o estudo?

Para avaliar o nível de adequação aos critérios de qualidade, foi adotada a estratégia de avaliação proposta por Costa (2010), onde se utiliza a escala de *Likert*-5, permitindo respostas gradativas de 0 (discordo totalmente) a 4 (concordo totalmente), como apresentado no Quadro 3.6. É importante frisar que a escala *Likert*-5 caracteriza-se por abranger cinco características: a validade da investigação, as ameaças à validade, a pertinência, a aplicabilidade e a consistência das evidências (Silva *et al*., 2010).

Quadro 3.2 Escala de Likert-5.

Escala de *Likert*-5	
Concordo totalmente (4)	Deve ser atribuído no caso em que o trabalho apresente no texto os critérios que atendam totalmente a questão.
Concordo parcialmente (3)	Deve ser atribuído no caso em que o trabalho atenda parcialmente aos critérios da questão.
Neutro (2)	Deve ser atribuído no caso em que o trabalho não deixe claro se atende ou não a questão;
Discordo parcialmente (1)	Deve ser atribuído no caso em que os critérios contidos na questão não são atendidos pelo trabalho avaliado;
Discordo totalmente (0)	Deve ser atribuído no caso em que não existe nada no trabalho que atenda aos critérios da questão.

Para facilitar a adoção da escala *likert*, cada nota da escala foi adaptada para atender cada critério de qualidade definido na Quadro 3.6. As notas da escala *likert*-5 adaptada podem ser vistas na Quadro 3.7:

Quadro 3.2 Escala para cada critério de qualidade.

Critério	**Escala**
CQ1	4 - Define e justifica o estudo claramente. 3 - Define claramente o estudo, porém a justificativa não é clara. 2 - Define claramente o estudo, mas não justifica. 1 - A definição dos objetivos do estudo não é clara. 0 - Não define o estudo.
CQ2	4 – Define o tipo de estudo, referenciando na literatura a metodologia. 3 – Define o tipo de estudo, porém sem referenciar a metodologia.

Critério	Escala
	2 – Não define o tipo de estudo. É possível inferir facilmente. 1 – Não define o tipo de estudo. É possível inferir com dificuldade. 0 – Não é possível inferir o tipo de estudo.
CQ3	4 – Define claramente uma seção com o contexto da pesquisa. 3 – O contexto da pesquisa está incluído em uma seção não exclusiva. 2 – O contexto da pesquisa está disperso ao longo do texto. 1 – O contexto da pesquisa está disperso e é insubstancial. 0 – O contexto da pesquisa não é abordado.
CQ4	4 – O texto apresenta uma seção de trabalhos relacionados. 3 – O texto apresenta trabalhos relacionados em uma seção não exclusiva. 2 – O texto apresenta trabalhos relacionados dispersos ao longo do texto. 1 – O texto não apresenta trabalhos relacionados, mas se apoia na literatura. 0 – O texto não apresenta trabalhos relacionados nem se apoia na literatura.
CQ5	4 – Resultados são claramente apresentados na seção de conclusão. 3 – Resultados são claramente referenciados na seção de conclusão. 2 – Resultados apresentados na conclusão não são claros. 1 – Resultados referenciados na conclusão não são claros. 0 – Não são apresentados resultados.
CQ6	4 – Os resultados estão totalmente aderentes ao objetivo do estudo. 3 – Os resultados estão aderentes ao objetivo do estudo, no entanto o autor faz ressalvas. 2 – Os resultados são parcialmente aderentes ao objetivo do estudo. 1 – Os resultados não estão aderentes ao objetivo do estudo. 0 – Não é alcançado nenhum resultado.
CQ7	4 – Algum dos elementos é claramente descrito. 3 – Algum dos elementos é apresentado, mas não aprofundado. 2 – Algum dos elementos é avaliado, porém não descrito. 1 – Algum dos elementos é citado indiretamente. 0 – Nenhum dos elementos é apresentado direta ou indiretamente.
CQ8	4 – Endereça a abordagem ágil para alguma atividade de forma detalhada (o nome, o que representa e como se usa a abordagem). 3 – Endereça a abordagem ágil para alguma atividade de forma superficial

Critério	Escala
	(o nome e o que representa a abordagem). 2 – Endereça a abordagem ágil para alguma atividade de forma resumida (apenas o nome da abordagem). 1 – Não Endereça a abordagem ágil para alguma atividade. 0 – Não é apresentada nenhuma abordagem ágil.
CQ9	4 – O método de experimento é definido e referenciado claramente. 3 – O método de experimento é definido claramente. 2 – O método de experimento é citado. 1 – O método de experimento não é citado, porém é possível inferir. 0 – Não é possível inferir o método de experimento.
CQ10	4 – O texto descreve critérios para a escolha dos estudos. 3 – O texto não descreve critérios para a escolha dos estudos, porém apresenta estudos que discordam do estudo apresentado. 2 – O texto descreve apenas estudos aderentes ao estudo apresentado. 1 – O texto descreve estudos insuficientes. 0 – O texto não descreve estudos base.
CQ11	4 – A área de atuação, tamanho e origem da organização são informados. 3 – Apenas duas das características do item 4 são informadas. 2 – Apenas uma das características do item 4 é informada. 1 – Nenhuma das características do item 4 é informada. 0 – O estudo não foi conduzido em uma ou mais organizações.

Como sugerido por Beecham (2007) *apud* Costa (2010), a partir da soma das pontuações, o trabalho foi enquadrado em um dos cinco níveis de qualidade classificados, como pode ser visto no Quadro 3.4.

Quadro 3.2 Níveis de qualidade (Beecham, 2007 apud Costa, 2010).

Faixa de Notas	Avaliação
Excelente	>86%
Muito Boa	66%-85%
Boa	46%-65%
Média	26%-45%

Faixa de Notas	Avaliação
Baixa	< 26%

3.2.11 Estratégia de Extração e Síntese dos dados

Para esta pesquisa, todos os estudos primários selecionados após a etapa de seleção dos estudos foram integralmente lidos. A extração foi realizada de forma estruturada, conforme o processo sugerido por Cruzes e Dyba (2011). Utilizou-se uma planilha eletrônica para a coleta dos dados da publicação (autor, referência, ano, editor); dados do contexto (tipo de estudo, métodos de pesquisa, análise dos dados, tamanho da amostra); e das evidências (trechos de texto relacionados com as questões de pesquisa). Desta forma, registrando detalhes significativos dos estudos com o objetivo de responder as questões de pesquisa.

Para este estudo, optou-se por realizar a técnica de síntese e análise temática dos dados, conforme as recomendações feitas por Cruzes e Dyba (2011), ver Figura 3.2. A partir da leitura integral dos estudos selecionados e do uso da planilha de extração dos dados, códigos foram atribuídos a segmentos de texto relevantes às questões de pesquisa. Estes códigos foram agrupados em temas, em seguida estes temas foram agrupados em categorias (temas de ordem superior) para a criação de um modelo temático que explicasse o fenômeno ou as questões de pesquisa.

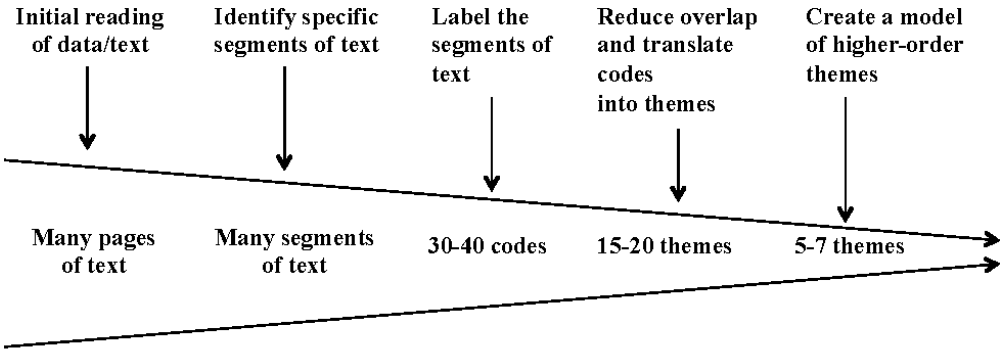


Figura 3.2 Processo de extração e síntese dos dados (Cruzes e Dyba, 2011).

3.3 Sumarização dos Resultados da Revisão Sistemática

Nesta etapa, os resultados obtidos da revisão sistemática executada neste trabalho foram somados aos resultados da revisão sistemática conduzida por Santos e Oliveira (2018). Ressalta-se que ambas RSL seguiram o mesmo processo de coleta e

avaliação de estudos explicados nas subseções anteriores. Com isso, um total de 177 estudos primários foram selecionados, dos quais foram possíveis extrair informações para responder as questões de pesquisa de interesse deste estudo. Estes estudos primários selecionados estão listados, por meio de identificadores (ID), para uma melhor referência ao longo do texto, conforme ilustrado no Apêndice B desta dissertação.

Uma visão detalhada dos estudos coletados por fonte de busca pode ser observada na Tabela 3.1. É possível perceber que as consultas retornaram uma quantidade considerável de estudos duplicados. Isto deve-se ao fato de que a maioria destas fontes indexam, também, estudos de outras fontes. Também é importante ressaltar que a quantidade de estudos retornados por cada fonte de busca pode ser justificada por diversos fatores, tais como: a quantidade total de estudos na fonte; e a relevância da fonte; e ordem de execução das buscas.

Tabela 3.31 Número de estudos por fonte de busca.

Fonte	**Retornados**	**Excluídos**		**Incluídos**	
		Duplicados	**CE**	**Qtde.**	**%**
IEEE	590	10	481	99	56%
ACM	933	33	880	23	13%
Scopus	1628	737	875	21	12%
Science Direct	1395	155	1236	4	2%
Ei Compendex	1320	377	916	27	15%
ISI Web of Knowledge	644	410	235	3	2%
Total:	**6510**	**1722**	**4611**	**177**	**100%**

Quanto à avaliação da qualidade dos 177 estudos primários selecionados para compor esta pesquisa, percebeu-se que a maioria dos estudos foi avaliada com a qualidade Muito boa (52%), seguido de Excelente (25%), depois Boa (18%) e por último Média (5%). Nenhum estudo foi considerado de Baixa qualidade, desta forma garantindo que esta revisão sistemática identificou estudos primários de qualidade significativa para a análise e para a extração dos dados. Na Figura 3.3 é possível ver a distribuição da qualidade dos 177 estudos primários incluídos nesta pesquisa.

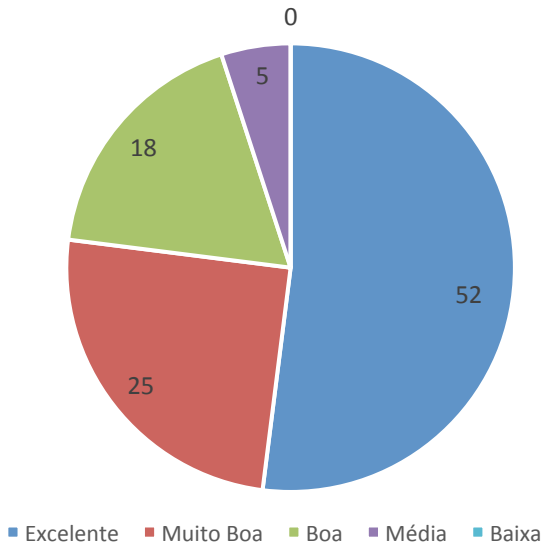


Figura 3.3 Qualidade dos estudos primários.

Outra análise feita foi referente aos países que mais publicaram sobre Engenharia de Requisitos no contexto de métodos ágeis. Segundo a análise dos dados dos estudos selecionados desta RSL, observa-se que as nações com mais publicações referentes ao tema são em sua maioria da América do Norte ou Europa. No entanto, países de outros continentes também abordam a relação de requisitos com métodos ágeis em seus estudos. É importante ressaltar que o Brasil também apresenta estudos referentes ao tema, evidenciando uma representatividade significativa da comunidade científica e da indústria de software brasileira no contexto da Agile-RE. Estes dados podem ser conferidos na Figura 3.4.

A seguir, os demais resultados desta RSL serão apresentados por meio de dois segmentos: o primeiro apresenta os resultados da questão de pesquisa principal (Q1) desta RSL, os quais justificaram e atenderam satisfatoriamente ao objetivo da condução desta revisão sistemática e de um dos objetivos específicos desta dissertação, contribuindo fortemente para a elaboração da abordagem ágil para o gerência de requisitos de software; enquanto que o segundo contém os resultados obtidos para cada questão de pesquisa secundária.

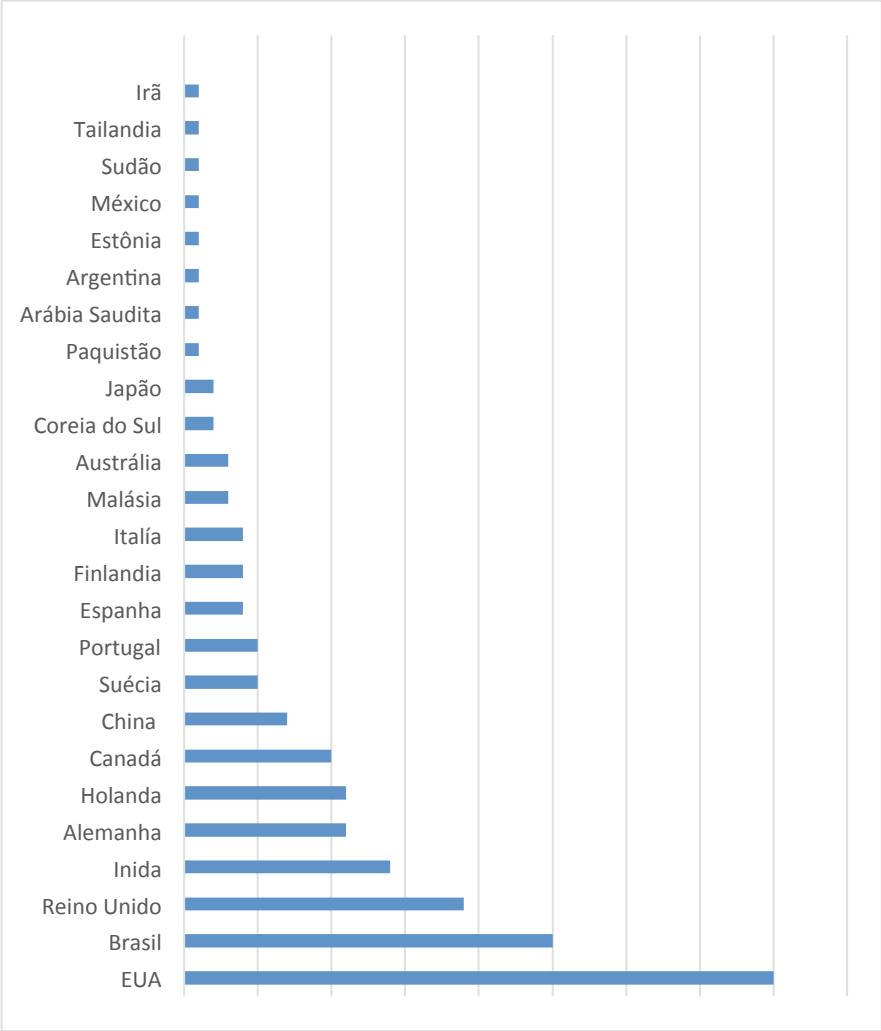


Figura 3.3 Países que mais publicaram sobre Agile-RE.

Com o uso da técnica de síntese e análise temática proposta por Cruzes e Dybå (2011), foram identificados dentro dos 177 estudos primários (Apêndice B), 35 abordagens ágeis para suporte as atividades de GRE em um ambiente ágil, conforme ilustrado no Quadro 3.9. Com este resultado foi possível analisar que a maioria dos estudos sobre ER-Ágil, disponíveis na comunidade científica, possuem foco maior nas atividades relacionadas ao processo DRE, ou seja, buscam propor abordagens ágeis para elicitar, analisar, especificar e validar de requisitos. Poucos possuem foco em propor explicitamente o uso de abordagens ágeis para as atividades de acompanhar estado dos requisitos, controlar a versão, manter a rastreabilidade bidirecional e gerenciar as mudanças. Muitos estudos propõem o uso de abordagens voltadas para a gerência de projetos como Scrum, *SaFe* e ASD, que possuem práticas que podem auxiliar em

algumas atividades de GRE, porém não são voltadas especificamente para estas atividades.

Ressalta-se que algumas das abordagens encontradas não são específicas para requisitos, mas podem ser adaptadas a um processo ágil de gerência de requisitos. Para ajudar na tipificação das abordagens, seguiu-se as recomendações feitas por Falbo (1998), as abordagens foram identificadas com o uso de códigos e categorizadas em três tipos de abordagens: técnicas, métodos e práticas. Esta estratégia permitiu mapear as abordagens e os respectivos estudos, onde estas abordagens foram encontradas.

Quadro 3.9 - Mapeamento de abordagens por estudos primários.

QP - Quais são as abordagens existentes nas metodologias ágeis que apoiam as atividades de Gerência de Requisitos no contexto de projetos de desenvolvimento de software?			
Cód.	**Nome da Abordagem Ágil**	**Estudos Primários (ID)**	**Categ.**
A01	*CBRank (Case-Based Ranking)*	[EP173]	Técnica
A02	*Discover to Deliver*	[EP159]	
A03	*FuzzyHCV*	[EP173]	
A04	*HYPEX (Hypothesis Experiment Data-Driven Development)*	[EP50], [EP159]	
A05	*Human Stories*	[EP168]	
A06	*Goal Sketching*	[EP50], [EP159]	
A07	*Invest (Independent, Negotiable, Valuable, Estimable and Small)*	[EP52], [EP108], [EP168]	
A08	*Lean canvas*	[EP171]	
A09	*MMFs (Minimal Marketable Features)*	[EP173]	
A10	*QIP(Quality Improvement Paradigm)*	[EP167]	
A11	*Personas*	[EP94], [EP108], [EP139], [EP152], [EP176], [EP161]	
A12	*Planning Poker*	[EP24], [EP62], [EP167]	
A13	*User Story Mapping*	[EP05], [EP159]	
A14	*Value Proposition Design*	[EP118], [EP159]	
A15	*AAOM (Agile Agentoriented Modelling)*	[EP162], [EP172]	Método
A16	*ASD (Adaptive Software Developmen)*	[EP4], [EP165], [EP175]	
A17	*DSDM (Dynamic Systems Development Method)*	[EP157], [EP165], [EP175]	
A18	*eXtremeProgramming*	[EP1],[EP2], [EP6], [EP7], [EP9], [EP12], [EP16], [EP17], [EP20], [EP27], [EP33], [EP34], [EP45], [EP46], [EP52], [EP57], [EP67], [EP68],	

QP - Quais são as abordagens existentes nas metodologias ágeis que apoiam as atividades de Gerência de Requisitos no contexto de projetos de desenvolvimento de software?			
Cód.	**Nome da Abordagem Ágil**	**Estudos Primários (ID)**	**Categ.**
		[EP69], [EP71], [EP76], [EP81], [EP82], [EP84], [EP85], [EP89], [EP92], [EP108], [EP109], [EP111], [EP112], [EP124], [EP125], [EP127], [EP131], [EP134], [EP135], [EP137],[EP146], [EP150], [EP151], [EP153], [EP157], [EP160], [EP165]	
A19	*FDD (Feature Driven Development)*	[EP9], [EP165]	
A20	*Hibrid Waterfall*	[EP67], [EP132], [EP164]	
A21	*Kanban*	[EP107], [EP149], [EP160], [EP165]	
A22	*Lean*	[EP160]	
A23	*QUS (Quality User Story)*	[EP156], [EP168], [EP169], [E1P70]	
A24	*SAFe (Scaled Agile Framework)*	[EP103], [EP160], [EP161], [EP164]	
A25	*Scrum*	[EP1], [EP7], [EP9], [EP16], [EP17], [EP18], [EP40], [EP42], [EP47], [EP53], [EP65], [EP66], [EP67], [EP70], [EP73], [EP75], [EP84], [EP86], [EP92], [EP103], [EP109], [EP113], [EP127], [EP129], [EP136], [EP146], [EP149], [EP154], [EP157], [EP160], [EP161], [EP163], [EP165], [EP167], [EP175], [EP176]	
A26	*Water-Scrum-Fall*	[EP149], [EP161]	
A27	*Automatic Generation of Traceability Links*	[EP166]	Práticas
A28	*Overall Model*	[EP9], [EP136], [EP146], [EP165]	

QP - Quais são as abordagens existentes nas metodologias ágeis que apoiam as atividades de Gerência de Requisitos no contexto de projetos de desenvolvimento de software?			
Cód.	**Nome da Abordagem Ágil**	**Estudos Primários (ID)**	**Categ.**
A29	*Inception Deck*	[EP177]	
A30	*Lightweight Traceability*	[EP163]	
A31	*Feature*	[EP9], [EP12], [EP33], [EP34], [EP47], [EP48], [EP67], [EP132], [EP165]	
A32	*Product Backlog*	[EP1], [EP7], [EP9], [EP16], [EP17], [EP40], [EP66], [EP67], [EP75], [EP84], [EP86], [EP109], [EP154], [EP157], [EP160], [EP167]	
A33	*Review meetings*	[EP18], [EP65], [EP70], [EP73], [EP92], [EP113], [EP146], [EP165], [EP175]	
A34	*TCR (Test Cases as Requirements)*	[EP158]	
A35	*User Stories*	[EP1],[EP2], [EP6], [EP7], [EP9], [EP12], [EP16], [EP17], [EP20], [EP27], [EP33], [EP34], [EP45], [EP46], [EP52], [EP57], [EP67], [EP68], [EP69], [EP71], [EP76], [EP81], [EP82], [EP84], [EP85], [EP89], [EP92], [EP108], [EP109], [EP111], [EP112], [EP124], [EP125], [EP127], [EP131], [EP134], [EP135], [EP137],[EP146], [EP150], [EP151], [EP153], [EP162], [EP168], [EP169], [EP170], [EP176]	

O mapeamento das abordagens permitiu responder a questão principal deste estudo (QP), atendendo diretamente ao objetivo da revisão sistemática. Identificou-se 14 técnicas (42%), 12 métodos (35%) e 9 práticas (22%). Podemos observar que

algumas das abordagens encontradas pertencem ou são adaptações de métodos como Scrum e FDD, como por exemplo: A32 (*Product Backlog*) e A28 (*Overall Model*) respectivamente. Por outro lado, abordagens como por exemplo: A34 (*User Stories*) e A02 (*Review meetings*) são técnicas ou práticas baseadas nos princípios e valores ágeis, que podem ser utilizados para apoio de uma ou mais atividade de gerência de requisitos.

Observou-se também a preocupação de alguns estudos em propor abordagens que possam ser aplicadas a grandes organizações, a fim de facilitar o gerenciamento de requisitos em ambientes com vários membros integrados e/ou lidar com projetos considerados críticos e que precisam de documentação mais detalhada, a exemplo as abordagens A17 (*DSDM*) e A24 (*SAFe*). Algumas abordagens, como A20 (*Hibrid Waterfall*) e A26 (*Water-Scrum-Fall*), recomendam aplicar abordagens híbridas que mesclam práticas ágeis e de métodos tradicionais como RUP (*Rational Unified Process*) para lidar com projetos grandes, ou que a organização possua características que dificultem o uso de um método ágil "puro".

Outra análise importante está relacionada com a atenção de algumas abordagens para a rastreabilidade unidirecional e bidirecional, como por exemplo: A13 (*User Story Mapping*), A27 (*Automatic Generation of Traceability Links*). Entende-se que a rastreabilidade entre os artefatos gerados em um projeto é crucial para outras atividades da gerência de requisitos, como gestão de mudanças proposta pela A02 (*Discover to Deliver*) e análise de impacto das abordagens A01 (*CBRank*) e A09 (*MMFs*).

Este estudo utiliza como base os processos de gerência de requisitos (GRE) definidos nos modelos de qualidade CMMI-DEV e MR-MPS-SW. Deste modo, foram definidas 5 atividades fundamentais para a gerência de requisitos, que são:

I) Entendimento dos requisitos junto aos fornecedores de requisitos;
II) Validação dos requisitos junto a equipe técnica;
III) Rastreabilidade bidirecional entre os requisitos e os produtos derivados;
IV) Revisões dos requisitos e produtos derivados; e
V) Gerência de Mudanças.

Conforme ilustrado na Quadro 3.10, identificou-se que a atividade (I) foi a que obteve maior número de abordagens de apoio, com 15 abordagens, seguida das

atividades (IV) e (V), ambas com 9 abordagens. Depois a atividade (III) com 6 abordagens e por último a atividade (II) com 4 abordagens. Observa-se que muitas abordagens apoiam cerca de uma ou mais atividades de maneira simultânea, ressaltando as abordagens que são métodos de gestão ágil como: A16, A19, A20, A25 e A26. Com isso, é possível perceber que há um *gap* científico sobre não haver um método ágil o qual atenda todas as atividades de GRE.

Quadro 3.10 - Mapeamento de abordagens por atividade de GRE.

QS1 - Quais atividades de Gerência de Requisitos são apoiadas pelas abordagens encontradas?		
Cod.	**Atividade de GRE**	**Cód. da Abordagem**
I	Entendimento dos requisitos junto aos fornecedores de requisitos	A04, A05, A06, A08, A09, A11, A14, A16, A17, A19, A21, A25, A26, A28, A34, A35
II	Validação dos requisitos junto a equipe técnica	A12, A29, A31, A32
III	Rastreabilidade bidirecional entre os requisitos e os produtos derivados	A13, A15, A21, A27, A30, A33
IV	Revisões dos requisitos e produtos derivados	A01, A03, A07, A10, A23, A24, A26, A29, A32
V	Gerência de mudanças	A02, A16, A17, A20, A24, A25, A26, A31, A33

Algumas abordagens encontradas são práticas e técnicas que pertencem ou são recomendadas por alguns métodos ágeis mais utilizados na indústria de software. Observa-se no Quadro 3.11 que o Scrum foi o método ágil com mais abordagens identificadas para a gerência de requisitos. Outra análise importante é o fato destes métodos não serem específicos à área de requisitos, muitos são destinados para gestão ou modelos de ciclo de vida, no entanto todos possuem alguma abordagem de apoio às atividades de GRE, evidenciando a presença e a importância da área de requisitos nos principais métodos ágeis. Observa-se que o FDD é o método ágil que possui maior relacionamento com requisitos dentre os demais, servindo de referência para auxiliar profissionais e organizações na escolha de um método ágil que seja mais difundido no mercado de software e mais aderente à área de requisitos.

Outra análise possível de se obter é que em sua maioria os métodos identificados não são específicos para a área de requisitos, muitos possuem foco em gestão, teste ou

modelos de ciclo de vida, no entanto todos possuem alguma técnica ou prática para apoio as atividades de ER-Ágil, o que evidencia a importância da área de requisitos para o funcionamento adequado de um processo de desenvolvimento de software em ambiente ágil.

Quadro 3.11 - Mapeamento de abordagens por método ágil.

QS2 - Caso a abordagem seja uma prática ou técnica, a mesma pertence a algum método ágil existente? Qual?	
Método ágil	**Cód. da Abordagem**
ASD	A32
DSDM	A32
FDD	A28, A31
Scrum	A12, A32, A34
XP	A35

As abordagens coletadas nos estudos primários desta revisão sistemática estão agrupadas em dois contextos, nos quais seus respectivos estudos foram aplicados: industrial ou acadêmico, conforme o Quadro 3.12. Obteve-se: uma frequência de 14 estudos (61%) com abordagens aplicadas no contexto acadêmico; enquanto que 9 estudos (39%) possuíram abordagens aplicadas no contexto industrial. Observa-se que não existe diferença significativa entre os contextos. Este resultado permite inferir que a academia e a indústria de software estão propondo e validando abordagens ágeis de apoio ao processo de gerência de requisitos.

Quadro 3.12 - Mapeamento dos estudos primários com o contexto de aplicação.

QS3 - Qual o contexto de aplicação da abordagem encontrada?		
Estudos primários (ID)	**Contexto**	**(%)**
[EP4], [EP5], [EP6], [EP7], [EP8], [EP9], [EP10], [EP12], [EP13], [EP14], [EP15], [EP17], [EP19], [EP20], [EP21], [EP22], [EP23], [EP26], [EP27], [EP30], [EP32], [EP34], [EP35], [EP36], [EP37], [EP39], [EP41], [EP42], [EP46], [EP47], [EP48], [EP49], [EP51], [EP52], [EP53], [EP54], [EP55] ,[EP57], [EP58], [EP59], [EP61], [EP63], [EP65], [EP68], [EP69], [EP71], [EP72], [EP78], [EP83], [EP84], [EP85], [EP87], [EP88], [EP91], [EP92], [EP96], [EP97], [EP98], [EP100], [EP104], [EP105], [EP108], [EP109], [EP113], [EP114], [EP115], [EP117], [EP118], [EP123], [EP124], [EP126], [EP128], [EP129], [EP130], [EP133], [EP136], [EP140], [EP141], [EP142], [EP147], [EP148], [EP151],	Acadêmico	61%

QS3 - Qual o contexto de aplicação da abordagem encontrada?		
Estudos primários (ID)	**Contexto**	**(%)**
[EP152], [EP154], [EP158], [EP159], [EP162], [EP163], [EP165], [EP166], [EP167], [EP168], [EP169], [EP170], [EP171], [EP173], [EP175], [EP176]		
[EP1], [EP2], [EP3], [EP11], [EP16], [EP18], [EP24], [EP25], [EP28], [EP29], [EP31], [EP33], [EP38], [EP40], [EP43], [EP44], [EP45], [EP50], [EP56], [EP60], [EP62], [EP64], [EP66], [EP67], [EP70], [EP73], [EP74], [EP75], [EP76], [EP77], [EP79], [EP80], [EP81], [EP82], [EP86], [EP89], [EP90], [EP93], [EP94], [EP95], [EP99], [EP101], [EP102], [EP103], [EP106], [EP107], [EP110], [EP111], [EP112], [EP116], [EP119], [EP120], [EP121], [EP122], [EP125], [EP127], [EP131], [EP132], [EP134], [EP135], [EP137], [EP138], [EP139], [EP143], [EP144], [EP145], [EP146], [EP149], [EP150], [EP153], [EP155], [EP156], [EP157], [EP160], [EP161], [EP164], [EP172], [EP174], [EP177]	Industrial	39%

Algumas abordagens ágeis do tipo técnica ou método possuem artefatos ou papéis específicos que são utilizados durante a execução das mesmas. Estes ativos foram mapeados de acordo com as respectivas abordagens, conforme ilustrado no Quadro 3.13. Este resultado auxilia a entender melhor as abordagens quanto a sua concepção e aplicação, orientando melhor os profissionais que almejam fazer uso das mesmas em suas organizações.

Quadro 3.13 - Mapeamento de abordagens por ativo.

QS4 - Quais os ativos (papéis e artefatos) envolvidos nessas abordagens?		
Cód. da Abordagem	**Artefatos**	**Papéis**
A01	*Check list*	
A11	*Persona Mapping*	
A12	*User Story Points*	
A13	*Stories Card* *User Story Mapping*	
A15	*Goal Model*	
A16		*Facilitator* *Customer* *Team*
A23	*Check list*	
A25	*Bakclog* *Features* *Burndown*	*Product Owner* *Srum Master* *Team*
A26	*Bakclog*	*Product Owner*

QS4 - Quais os ativos (papéis e artefatos) envolvidos nessas abordagens?		
Cód. da Abordagem	**Artefatos**	**Papéis**
	Features	*Srum Master* *Team*
A27	*Mapping matrix*	
A28	*Class Diagram*, *Prototype*	
A33	*Test Cases*	

4 REACT-M: UM MÉTODO ÁGIL PARA A GERÊNCIA DE REQUISITOS DE SOFTWARE

Dentro deste capítulo será apresentado o principal produto deste estudo, o método REACT-M, que é uma proposta de método ágil para apoio às atividades de gerência de requisitos, empiricamente fundamentada em outras abordagens ágeis encontradas na literatura especializada; nas práticas defendidas pela Agile-RE; e nas boas práticas constantes nos processos de Gerência de Requisitos dos modelos CMMI-DEV e MR-MPS-SW. Esta abordagem possui como objetivo dar sustentação às atividades de gerência de requisitos, possibilitando um controle gerenciado da evolução dos requisitos de um produto de software. A ideia é que todos os interessados na construção do produto interajam de forma mais colaborativa a fim de lidar de forma mais efetiva com os desafios que podem surgir em um projeto de desenvolvimento de software, em relação às atividades de engenharia de software.

É importante lembrar que o REACT-M pode ser considerado uma extensão do método REACT (Santos e Oliveira, 2018), que tem o objetivo de cobrir as atividades de desenvolvimento ágil de requisitos. No entanto, o REACT-M pode ser adotado de forma independente, podendo ser adaptado para uso com outras estratégias ágeis para elicitar, analisar, especificar e validar os requisitos de software, bem como pode ser combinado com abordagens preocupadas com a gerência de projetos. Desta forma, é recomendando que o REACT-M seja utilizado com algum outro método ágil para desenvolvimento de software e gestão de projetos como Scrum, XP, SAFe ou ASD.

Inicialmente, todas as etapas da metodologia para a construção do REACT-M serão descritas. Após isso, o ciclo e vida proposto para o método, os papéis envolvidos, as cerimônias e os artefatos que compõem o REACT-M serão apresentados na forma de um guia de utilização, o que facilitará o entendimento por explicar o uso como um

passo-a-passo para a condução do método. Também será utilizado um exemplo fictício de artefatos para demonstrar de forma mais prática a aplicabilidade do REACT-M. Os tópicos deste Capítulo são:

- Metodologia para Criação do REACT-M;
  - Fase de Planejamento;
  - Fase de Seleção;
  - Fase de Composição;
- Guia de Utilização do REACT-M;
  - Atividade de Identificar Fornecedores de Requisitos;
  - Atividade de Elaborar *Backlog* de Tarefas;
  - Atividade de Avaliar e Dimensionar o *Backlog* Com a Equipe Técnica
  - Atividade de Rastreabilidade de Requisito
  - Atividade de Revisar e Registrar Inconsistências
  - Atividade de Tratar Mudanças no *Backlog*

4.1 Metodologia para Criação do REACT-M

A concepção do método ágil REACT-M seguiu um fluxo de etapas sistematicamente organizadas: Planejamento, Seleção e Composição (ver Figura 4.1), a fim de garantir uma concepção confiável da abordagem ágil proposta.

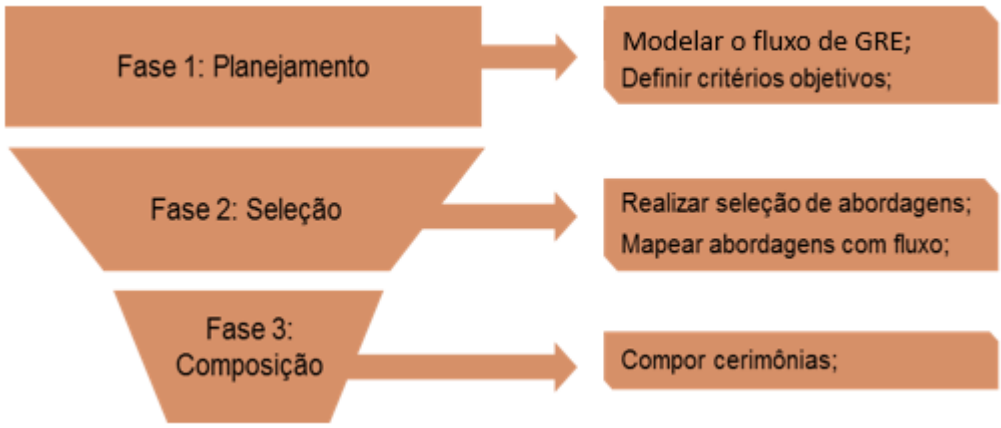


Figura 4.1 Fases para elaboração do método ágil REACT-M.

4.1.1 Fase de Planejamento

A primeira etapa desta fase consistiu na modelagem de um fluxo de atividades para o processo de GRE adotado pelo estudo e que norteou a concepção do ciclo de vida do REACT-M. Ressalta-se que foi utilizado como base o fluxo proposto pelo REACT (Santos e Oliveira, 2018), que contempla as atividades de apoio ao processo de DRE, as atividades do REACT-M foram agregadas a este fluxo para dar cobertura ao processo de GRE. Deste modo, este fluxo contempla as macro-atividades de DRE: Elicitação, Análise, Especificação e Validação de requisitos (Pressman e Maxin, 2016; SOFTEX, 2016b; Wirgers *et al.*, 2013); bem como as macro-atividades de GRE: Manter Requisitos, Manter a Rastreabilidade Bidirecional, Revisar Artefatos, Gerenciar as Mudanças (SEI, 2010; SOFTEX, 2016a; Wiegers *et al.*, 2013). O objetivo deste fluxo é dar visão das atividades de GRE que devem ser realizadas, e também quais atividades de DRE dão apoio gerencial. Para esta etapa, os modelos de qualidade de software MR-MPS-SW (SOFTEX, 2016a) e CMMI-DEV (SEI, 2010) foram utilizados para embasar as tarefas presentes neste fluxo, uma vez que estes modelos servem como excelentes referências de boas práticas esperadas em um processo de software. Vale lembrar que os processos de DRE e GRE são processos constantes nestes modelos de maturidade. A Figura 4.2 ilustra o fluxo modelado.

A segunda etapa foi de definição de critérios objetivos, os quais embasam a seleção de um conjunto menor de abordagens dentre as 35 abordagens ágeis identificadas na RSL conduzida (consultar Quadro 3.6 na Seção 3.3 deste estudo). A definição destes critérios foi pensada pelos pesquisadores com base em uma análise dos 12 princípios do Manifesto Ágil (Agile Manifesto, 2001) (consultar Seção 2.4 deste estudo), identificando conceitos fundamentais os quais permeiam por todos os princípios. Estes critérios podem ser conferidos na Tabela 4.1

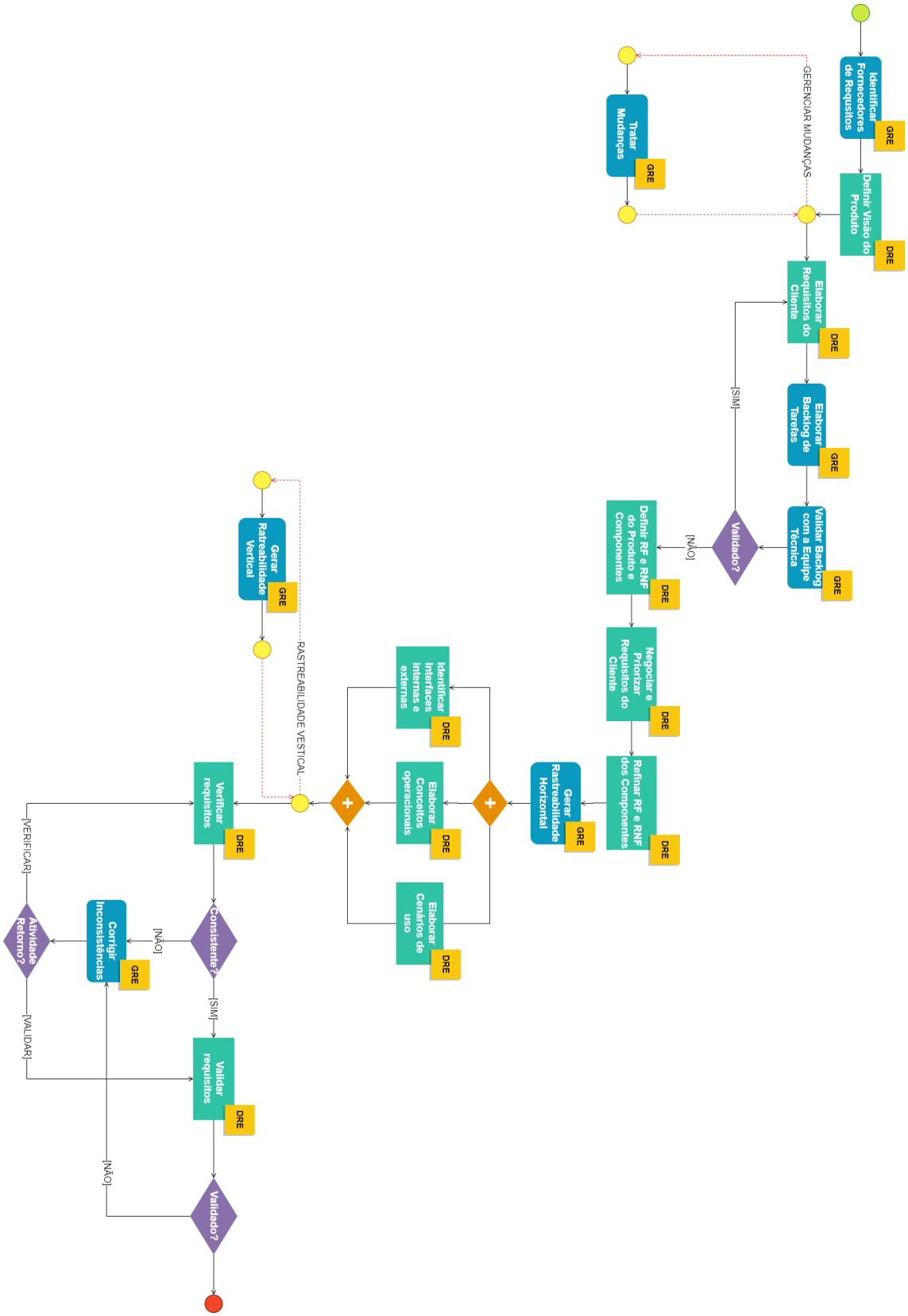


Figura 4.2 Fluxo de tarefas contemplando os processos DRE e GRE.

Quadro 4.1 Critérios Objetivos para a seleção de abordagens ágeis da RSL

ID	Critérios Objetivos	Descrição
C1	Simplicidade	A abordagem selecionada deve proporcionar ou utilizar recursos simples para a sua execução, onde não haja artefatos ou cerimônias desnecessárias e nem gere um "gargalo" que impeça ou dificulte o bom funcionamento do método ágil.
C2	Orientado a metas	A abordagem selecionada deve fazer uso do paradigma de *Goal-driven development.*
C3	Centrado no *Customer* e usuários finais	A abordagem selecionada deve envolver o *Customer* e os usuários finais na maioria de suas atividades, proporcionando um gerenciamento de requisitos de um produto fortemente alinhado aos objetivos e necessidades de negócio de seus usuários.

4.1.2 Fase de Seleção

A primeira etapa desta fase teve como objetivo realizar a seleção de um conjunto menor de abordagens dentre as 35 abordagens ágeis identificadas nesta RSL (consultar Quadro 3.6 na Seção 3.3 deste estudo). Para isto, um *checklist* de avaliação contendo os critérios objetivos de seleção definidos anteriormente no Quadro 4.1 foi aplicado em todas as 35 abordagens ágeis. Utilizou-se uma adaptação da escala *Likert*-5 para a pontuação destas abordagens (consultar Quadro 4.2).

Quadro 4.2 Escala Likert-3 para pontuação das abordagens ágeis

Escala *Likert*-3	Descrição
Totalmente Aderente (2)	A abordagem possui fortes indícios em relação ao critério objetivo.
Aderente (1)	A abordagem possui poucos indícios em relação ao critério objetivo.
Não Aderente (0)	A abordagem não menciona e nem possui indícios em relação ao critério objetivo.

Desta forma, um total de 23 abordagens ágeis dentre técnicas, métodos e práticas foram selecionadas para compor a criação do REACT-M, conforme apresentado no Quadro 4.3.

Quadro 4.3 Abordagens ágeis selecionadas

Cód.	Nome da Abordagem Ágil	Categ.
A01	*CBRank (Case-Based Ranking)*	Técnica
A02	*Discover to Deliver*	
A05	*Human Stories*	
A07	*Invest (Independent, Negotiable, Valuable, Estimable and Small)*	
A08	*Lean canvas*	
A11	*Personas*	
A12	*Planning Poker*	
A13	*User Story Mapping*	
A16	*ASD (Adaptive Software Developmen)*	Método
A17	*DSDM (Dynamic Systems Development Method)*	
A18	*eXtremeProgramming*	
A19	*FDD (Feature Driven Development)*	
A20	*Hibrid Waterfall*	
A21	*Kanban*	
A22	*Lean*	
A23	*QUS (Quality User Story)*	
A24	*SAFe (Scaled Agile Framework)*	
A25	*Scrum*	
A26	*Water-Scrum-Fall*	
A28	*Overall Model*	Prática
A30	*Lightweight Traceability*	
A32	*Product Backlog*	
A35	*User Stories*	

Já na segunda etapa, um mapeamento entre estas 23 abordagens ágeis selecionadas com as tarefas presentes no fluxo de GRE modelado na fase anterior foi realizado. O objetivo deste mapeamento foi de definir quais abordagens ágeis selecionadas endereçam cada tarefa deste fluxo, assim, permitindo uma análise mais precisa no momento de composição do REACT-M, como pode ser visto na Figura 4.3, onde uma caixa explicativa contendo as abordagens ágeis selecionadas encontra-se associada a cada tarefa que compõe o fluxo.

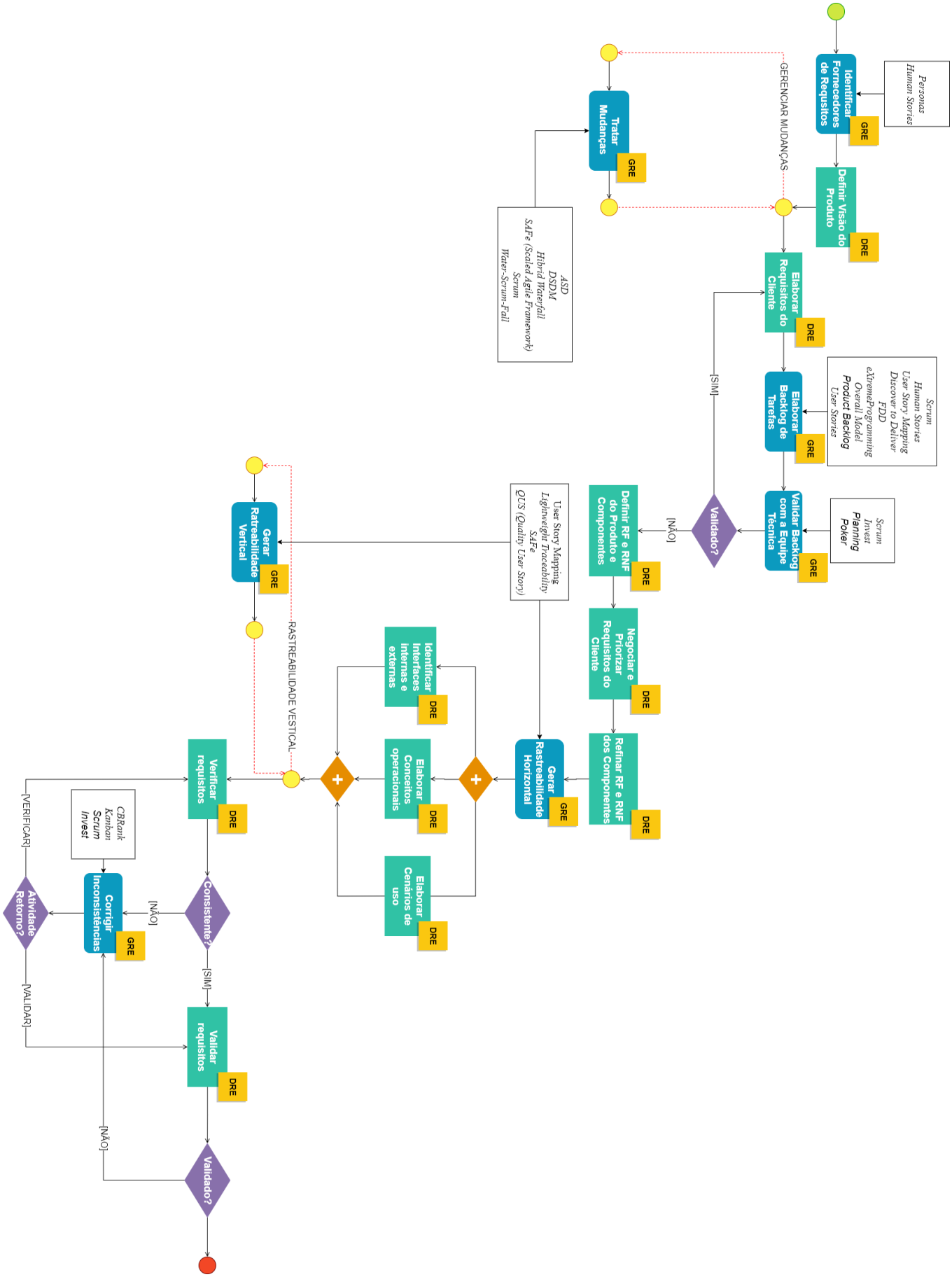


Figura 4.3 Mapeamento de Atividades do Processo de GRE com abordagens ágeis selecionadas.

4.1.3 Fase de Composição

A última etapa para criar o REACT-M consistiu em definir um conjunto de cerimônias que compreendessem e agrupassem as tarefas presentes no fluxo modelado da Figura 4.2, bem como agrupassem um conjunto específico de técnicas, papéis envolvidos e artefatos extraídos a partir da análise das 23 abordagens ágeis selecionadas na fase anterior. Um ponto importante de salientar é que o REACT-M compartilha as mesmas cerimônias e papéis do REACT para desenvolvimento de requisitos ágil, justificado por este estudo ter como objetivo que o método proposto seja altamente aderente ao método REACT. As cerimônias adotadas estão embasadas seguindo as boas práticas da Agile-RE descritas na Seção 2.5 deste estudo. Estas práticas ajudaram na criação de cada cerimônia, pois elas guiaram os pesquisadores para um encadeamento eficiente, lógico e consistente das abordagens ágeis selecionadas, posto que algumas adaptações nas abordagens ágeis selecionadas foram necessárias de se fazer, a fim de não deixar nenhum *gap* existente em todo o processo de GRE, mas também estar fortemente aderente à filosofia da Agile-RE. Portanto, as cerimônias definidas para o REACT foram:

- ***Inception***: objetivo de obter uma visão do produto pretendido, das metas de negócio do *Customer* e de seus usuários-chave. Esta cerimônia deve ser executada no início do projeto, mas também é recomendada em requisições de mudanças;
- ***Story Discovery***: objetivo de construir o *Backlog* de requisitos do software, assim como obter uma visão geral do domínio do negócio;
- ***Refining***: objetivo de refinar os requisitos com mais detalhes que permitam um melhor entendimento dos requisitos por todos, além de manter a rastreabilidade entre os artefatos;
- ***Modeling***: também tem como objetivo refinar os requisitos com mais detalhes, contudo sob uma perspectiva dos requisitos de mais baixo nível em termos de modelagem e implementação, mantendo a rastreabilidade entre os artefatos e controlando o fluxo de atividades;

- ***Inspection***: objetivo de verificar e validar os vários níveis de requisitos desenvolvidos ao longo do REACT-M. Nesta cerimônia também é obtido o comprometimento da equipe com os requisitos que serão implementados.

Já os papéis envolvidos no REACT-M são:

- ***Customer***: é o cliente para o qual o produto de software está sendo desenvolvido. Seu objetivo é tomar decisões sobre os requisitos ao longo do projeto;
- ***Domain Expert***: é o representante do *Customer*, o qual possui uma grande *expertise* em determinadas áreas de domínio dentro do negócio do *Customer*. Seu objetivo é ajudar o *Customer* e o *Team* a desenvolver os requisitos de um produto que realmente atenda às necessidades do negócio;
- ***Team***: é a Equipe de desenvolvimento responsável por evoluir os requisitos do produto ao longo do projeto;
- ***Facilitator***: é uma espécie de *coach* com grande *expertise* sobre o REACT-M, responsável por guiar todos os envolvidos com interesse na evolução dos requisitos a aplicar o REACT-M da melhor forma possível em cada contexto de projeto de software.

4.2 Guia de utilização do REACT-M

O REACT-M será apresentado como um guia prático para a sua utilização, onde as atividades executadas no método serão descritas como um passo-a-passo e para demonstrar de forma mais prática a aplicabilidade do método serão utilizados exemplos fictícios. O REACT-M está fortemente alinhado ao método REACT para desenvolvimento de requisitos (DRE). Assim, durante o guia serão feitas referências às atividades de DRE contidas no método REACT. Para mais informações, o guia de utilização do REACT pode ser consultado no trabalho de Santos e Oliveira (2018). O REACT-M é composto por 5 cerimônias, 6 atividades e 4 papéis. A Figura 4.4 mostra o ciclo de vida do REACT-M adaptado do método REACT.

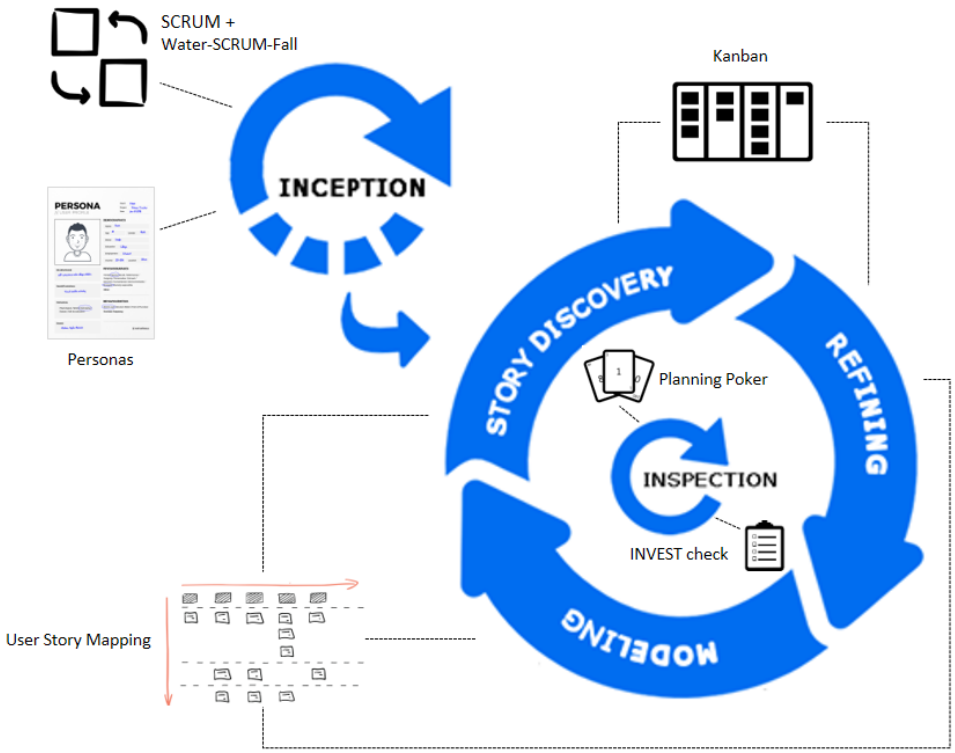


Figura 4.4 Ciclo de Vida REACT-M

A primeira cerimônia do REACT-M, denominada de *Inception*, consiste em definir uma visão de alto nível do produto a ser desenvolvido. A descrição do produto deve ser feita de acordo com as metas de negócio do *Customer* e também das expectativas e necessidades dos usuários finais do produto. Deste modo, uma das atividades fundamentais nesta cerimônia é a identificação dos fornecedores de requisitos. O REACT-M propõe mapear os fornecedores de requsitos a partir da técnica de Personas, que permite uma visão mais empática dos usuários, o que possibilita melhores descrições das *Users Stories*. Esta cerimônia deve ocorrer uma vez no início do projeto, no entanto, dependendo das mudanças ocorridas ao longo do projeto, os participantes podem decidir realizá-la outras vezes. Uma boa dica é realizar novamente antes de cada entrega do produto, ou a cada solicitação de mudança dos *Customers*.

As cerimônias de *Story Discovery*, *Refining* e *Modeling* são focadas na elaboração dos diversos níveis de requisitos que são gerados durante o ciclo de desenvolvimento e que possibilitam um entendimento mais amplo sobre as

funcionalidades que estão sendo implementadas. Nestas cerimônias, o REACT-M age como um método de suporte que propõe mecanismos para o controle e gerenciamento das tarefas que estão sendo desenvolvidas dentro do ciclo a partir da utilização do quadro Kanban adaptado. Esta técnica também disponibiliza meios para manter a rastreabilidade entre as tarefas de trabalho geradas nestas cerimônias. O REACT-M sugere que a rastreabilidade seja feita de forma leve e não burocrática, utilizando uma adaptação do método de *User Story Mapping*.

A cerimônia de *Ispection* deve ser executada em paralelo com as cerimônias de *Discovery*, *Refining* e *Modeling*. As atividades desta cerimônia têm o objetivo de garantir a qualidade das *Users Stories* e artefatos derivados, para isso o REACT-M propõe o uso dos critérios INVEST que auxiliam os membros do *Team* a definir as informações necessárias para que uma *User Story* possa entrar no ciclo de desenvolvimento. Além de avaliar as *Users Stories* e demais artefatos gerados, esta cerimônia também objetiva mensurar o esforço necessário para o desenvolvimento das *Users Stories* a partir da técnica de *Planning Poker* nativa do método *Scrum*.

A seguir as atividades propostas pelo método REACT-M e que são executadas dentro de cada uma das cerimônias serão apresentadas com maior riqueza de detalhes para que o leitor do guia compreenda como as atividades do método devem ser executadas.

4.2.1 Atividade de Identificar Fornecedores de Requisitos

Para efetivamente conseguir identificar os requisitos do produto é importante identificar quem são os fornecedores de requisitos e seus objetivos. O REACT-M propõe identificar os fornecedores de requisitos a partir da técnica de *Personas*. Uma Persona representa um indivíduo que irá utilizar ou será impactado pelo produto. Deve-se descrever não somente o papel da *Persona*, mas também seus objetivos específicos, criando uma representação realista de um usuário, o que auxilia a levantar os requisitos do ponto de vista de quem de fato terá alguma interação com o produto final. Importante enfatizar que o REACT-M não está interessado no uso da técnica de *Personas* para o mapeamento dos usuários-chave, e sim da identificação dos fornecedores de requisitos como sugere o uso desta técnica.

Quem participa: *Domain Experts*, *Customer*, *Facilitator* e *Team* (todos ou representantes).

Descrição Prática:

Todos os participantes devem participar juntos para definir uma visão geral do produto utilizando um *Canvas*, conforme sugerido no método REACT para DRE. Dentre os componentes preenchidos neste *Canvas* existe um especifico para a definição do nome das *Personas* do produto. O *Domain Expert* deve dividir os participantes em grupos pequenos e entregar um *template* de *Persona* para ser preenchido. Em seguida, cada grupo deve escolher uma *Persona* identificada no *Canvas*. O *Domain Expert* deve solicitar a cada grupo que preencha as informações da *Persona* de acordo com o *template*. Após o término do preenchimento dos *templates*, cada grupo deve apresentar a sua *Persona* para todos os participantes. Os demais participantes devem dar *feedback* ao grupo sobre a sua *Persona* a fim de melhorá-la. Um exemplo de *template* de *persona* preenchido pode ser visto a Figura 4.5



Figura 4.5 Template de Persona.

4.2.2 Atividade de Elaborar Backlog de Tarefas

Os requisitos levantados formam o *Backlog* do Produto, o *Domain Expert* como representante dos *Customers* é o responsável por manter esse *Backlog* sempre atualizado e gerenciado, prezando por um princípio dos métodos ágeis que diz que o produto deve ser entregue incrementalmente para o *Customer* e cada parte entregue deve gerar valor de negócio para os usuários. Para isso é necessário que o *Backlog* seja priorizado a fim de auxiliar o *Customer* a definir as ordens de entrega das partes do produto. Técnicas para a priorização de requisitos ágeis podem ser encontradas no guia de utilização do método REACT para DRE. Para auxiliar o gerenciamento da ordem de entregas sugere-se a utilização de um *Kanban* adaptado para o *Domain Expert*. O *Domain Expert Kanban* dá uma visão do andamento do projeto em nível de *Users Stories*, permitindo que sejam facilmente identificadas as funcionalidades já entregues ao *Customer*, as que estão no ciclo de desenvolvimento e as que ainda serão desenvolvidas nos próximos ciclos.

Quem participa: *Domain Experts*, *Facilitator*, *Team*.

Os participantes do projeto devem executar as técnicas de *User Story*, O*verall Model* e *Business Goals & Constraint Goals* descritas no método REACT de DRE para o levantamento das *Users Stories*, que constituem os requisitos do cliente em alto nível, bem como realizar a priorização das mesmas. As *Users Stories* levantadas constituem o *Backlog* do produto, onde o *Domain Expert* deve elaborar um *Domain Expert Kanban* para facilitar o gerenciamento do *Backlog*. Este *Kanban* pode ser colocado em uma parede ou pode-se utilizar uma versão digital. As *Users Stories* geradas pelos participantes devem ser incluídas na seção *Product Backlog do Kanban*. De acordo com a priorização feita pelo *Customer*, o *Domain Expert* deve mover as *Users Stories* de maior prioridade para a seção *Sprint Backlog*, o que indica que essas *Users Stories* são as que devem ser implementadas no ciclo de desenvolvimento atual. Em seguida, o *Team* deve avaliar se as *Users Stories* selecionadas para o ciclo de desenvolvimento possuem informações suficientes para serem implementadas. Esta avaliação deve ser feita conforme as práticas propostas na atividade de Avaliar *Backlog* com a Equipe Técnica. A cada entrega realizada, o *Domain Expert* deve mover as *Users Stories* entregues para a seção *Delivered* do *Kanban*. A cada ciclo de desenvolvimento, o

processo executado nesta atividade deve ser repetido até que o produto seja entregue por completo. Um exemplo de *Domain Expert Kanban* pode ser visto na figura 4.6.

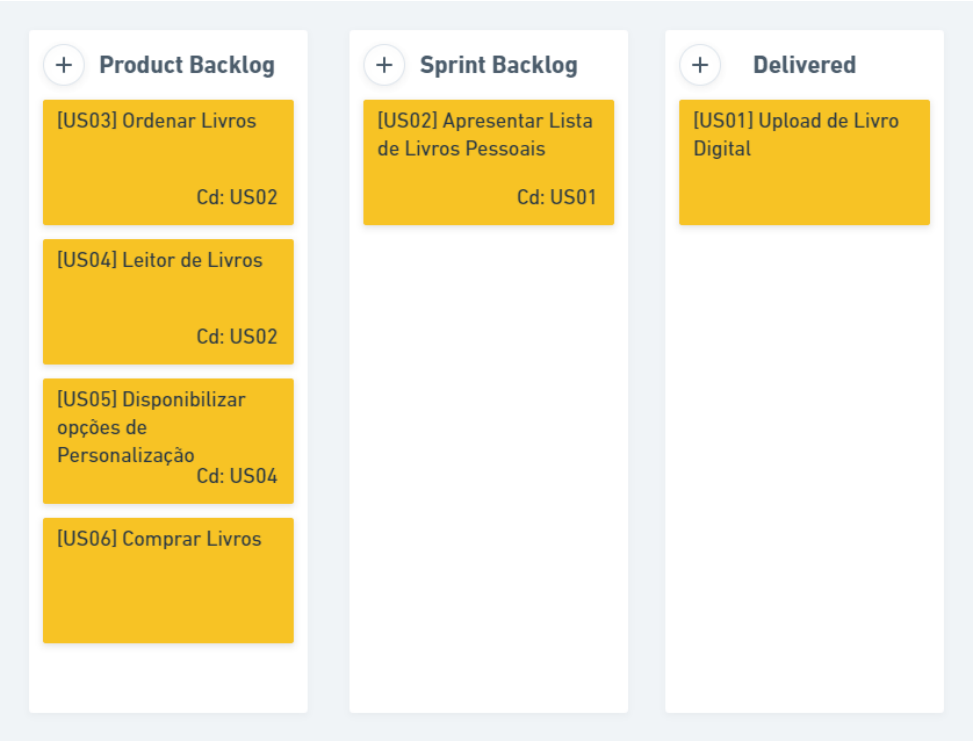


Figura 4.6 Template de Domain Expert Kanban.

Para o desenvolvimento de uma *User Story* pode ser que o *Team* precise executar diversas tarefas, como por exemplo criar um protótipo, criar uma tela do sistema, criar uma tabela no banco de dados, obter dados a partir de uma interface externa, etc. Por esse motivo, é importante que o *Team* também controle suas atividades para que não ocorra problemas na entrega das funcionalidades definidas no *Backlog*. Para auxiliar o gerenciamento das tarefas executadas pelo *Team*, sugere-se também o uso de um *Kanban* adaptado para o processo de desenvolvimento utilizado pelo *Team*.

Os integrantes do *Team* elaboram um *Kanban* conforme o *template* apresentado na Figura 4.7, onde o quadro pode ser colocado em uma parede ou pode-se utilizar uma versão digital. Em seguida, as *Users Stories* selecionadas para entrar no ciclo de desenvolvimento devem ser colocadas na seção *Sprint Backlog* do *Team Kanban*. Posteriormente, o *Team* deve discutir entre si quais tarefas devem ser executadas para entregar as *Users Stories* no fim do ciclo. Estas tarefas devem ser anotadas e colocadas na seção *To Do* do *Team Kanban*. Cada tarefa deve estar devidamente relacionada com

a *User Story* responsável pela sua geração. Para realizar este relacionamento, o *Team* deve seguir as práticas sugeridas na atividade de Rastreabilidade descrita neste guia. Quando um integrante do *Team* for executar uma tarefa, este deve movê-la para a seção *In Progress* do *Team Kanban*. Após a finalização da tarefa, o integrante deve movê-la para a seção *Testing* do *Team Kanban*. Caso o integrante do *Team* responsável por realizar o teste encontre algum defeito na tarefa, este deve gerar uma tarefa descrevendo o defeito identificado e a colocar na seção *To Do* para que possa ser corrigida. A tarefa de defeito também precisa ser relacionada com a tarefa que a derivou. A tarefa deve ser movida para a seção *Done* quando os testes validarem a qualidade do que foi desenvolvido. Funcionalidades que geraram defeitos só devem ser movidas para *Done* quando seus defeitos também forem testados e movidos para *Done*.

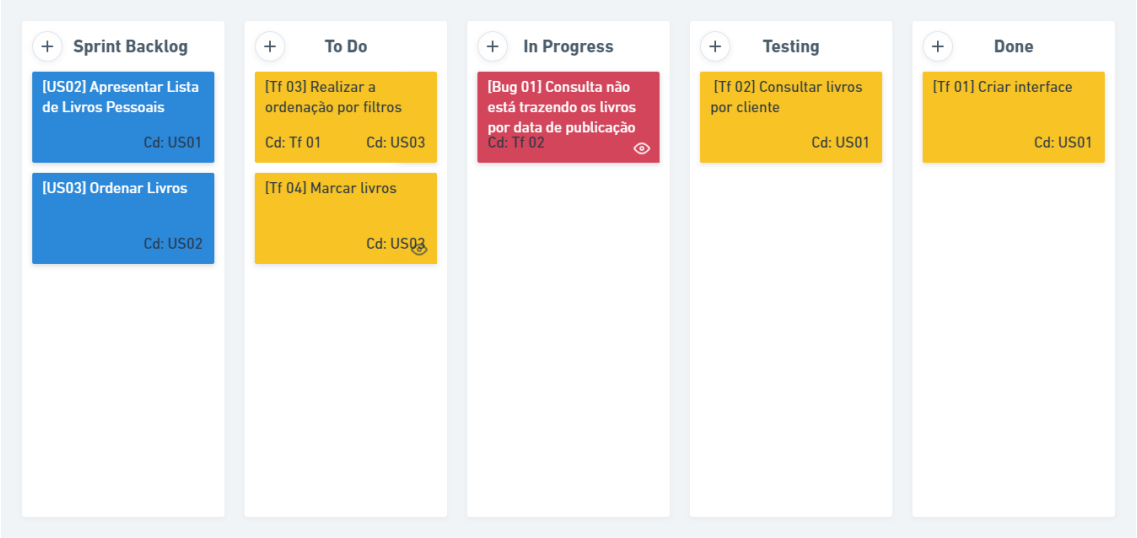


Figura 4.7 Template de Team Kanban.

4.2.3 Atividade de Avaliar e Dimensionar 0 Backlog com a Equipe Técnica

O *Backlog* do Produto deve ser validado pelo *Team* para que este indique se as *Users Stories* possuem informações de negócio suficientes para entrar no ciclo curto de desenvolvimento. Para esta tarefa, é interessante que o *Team* defina critérios de qualidade que os itens do *Backlog* devem possuir. O *Team* deve validar as *Users Stories* que o *Domain Expert* selecionou para entrar no ciclo de desenvolvimento com o intuito de verificar se existem informações de negócio suficientes para que a funcionalidade seja desenvolvida. O REACT-M propõe o uso do conceito *Definition of Ready* do *Scrum*. Este conceito age de forma bem parecida com a técnica *Definition of Done*, mas

enquanto "*Done*" garante qualidade na saída do ciclo, "*Ready*" garante qualidade na entrada do ciclo. Assim, o *Team* define o que se espera que as *Users Stories* possuam para entrar no ciclo de desenvolvimento. Para facilitar a definição do que o *Team* espera das *Users Stories*, sugere-se a utilização dos critérios INVEST que definem características que os requisitos devem ter para serem validados:

> ***INDEPENDENT*** – Deve ser independente de outros requisitos para ser implementado ou se possui a menor dependência possível;
>
> ***NEGOTIABLE*** – Deve estar priorizado pelo *Customer*;
>
> ***VALUABLE*** – Deve atender um ou mais objetivos definidos nas *Personas*;
>
> ***ESTIMABLE*** – Deve ser possível estimar o esforço para a implementação;
>
> ***SMALL*** – Deve ser pequeno suficiente para ser implementado em uma iteração;
>
> ***TESTABLE*** – Deve ser possível gerar cenários de teste de aceitação.

Quem participa: *Domain Experts*, *Facilitator*, *Team*.

Descrição Prática:

O *Domain Expert* deve explicar o valor de negócio de cada *User Story* selecionada para o ciclo de desenvolvimento. Em seguida, o *Team* deve validar os requisitos de acordo com os critérios INVEST. Para facilitar a validação, o REACT-M propõe a utilização do *Template* apresentado na Figura 4.8. Caso a *User Story* não seja validada, o *Team* e o *Domain Expert* devem conversar sobre o que precisa ser melhorado para que posteriormente a *User Story* possa ser novamente validada. Caso a *User Story* não seja validada, alguns cenários podem ocorrer como: a necessidade de quebrar a *User Story* em menores, alterar a descrição da *User Story*, criar novos cenários de aceitação ou até mesmo cancelar a *User Story*, caso seja verificado que ela não atenda nenhuma necessidade de negócio.

DEFINITION OF READY	
☐	Caso possível, a user story pode ser implementada independente de outras users stories?
☐	A user story foi priorizada pelo cliente?
☐	A user story reflete a expectativa do cliente?
☐	A user story agrega valor para o negócio do cliente?
☐	É possível estimar o esforço de implementação da user story?
☐	A user story é pequena o suficiente que não pode ser partida em mais de uma?
☐	Através dos critérios de aceite é possível gerar os cenários de teste?

Figura 4.8 Template de Validação utilizando Critérios INVEST.

O *Team* também tem a reponsabilidade de estimar o esforço necessário para a implementação dos itens do *Backlog*, levando em consideração questões técnicas e a experiência que os membros possuem. Além disso, é importante frisar que para estimar é necessário levar em consideração todas as tarefas do desenvolvimento. O *Planning Poker* é uma técnica do *Scrum* que permite ao *Team* gerar estimativas rapidamente. Nessa técnica não é realizada estimativa por horas de trabalho, as estimativas são feitas utilizando *User Story Point* que representa um valor abstrato de tamanho. A ideia do *Planning Poker* é pontuar os itens de *Backlog*, onde cada membro da equipe possui um baralho de 13 cartas numeradas com a sequência de *Fibonacci*. Neste baralho, cada valor contido na carta corresponde ao *story point* que o membro julga necessário para implementar a estória que está sendo estimada. Além das cartas com os valores de *Fibonacci*, há outras cartas.

Primeiro escolhe-se uma *User Story* que a estimativa aproxime-se de 3 *story points* para servir como referência. A partir desta *User Story* serão feitas estimativas para outras estórias seguindo o *Planning Poker*. O *Domain Expert* lê para o *Team* a *User Story* e contextualiza a respeito do seu valor de negócio. Em seguida, cada membro do *Team* define individualmente quantos *User Story Points* são necessários para implementar e selecionar a carta correspondente. Quando todos os membros tiverem escolhido uma carta, todos viram ao mesmo tempo. Caso os valores sejam muito divergentes, cada membro apresenta uma justificativa geralmente do valor mais alto ao mais baixo. Então, é feita outra rodada até que as estimativas de esforço

cheguem a uma convergência, isto é, todas as estimativas sejam aproximadamente as mesmas para a *User Story*. Assim, o resultado será o valor que, ao final, tiver maior ocorrência ou a média entre estes valores finais.

4.2.4 Atividade de Rastreabilidade de Requisito

Rastreabilidade significa, de uma forma mais simplificada, relacionar diferentes artefatos gerados pelo seu processo a partir de um vínculo que pode ser "puxado" trazendo consigo todos os artefatos que estão relacionados entre si. É um mecanismo para navegar entre os artefatos que mantém algum tipo de relacionamento. Em um projeto Ágil a rastreabilidade é tratada como mais um elemento que visa aproximar aquilo que o usuário pede daquilo que ele recebe. A rastreabilidade tem que ser trabalhado de uma maneira que dispense o uso de planilhas atividades burocráticas que não contribuem para o principal objetivo que é entregar valor ao final do ciclo.

Quem participa: *Domain Experts*, *Facilitator*, *Team*.

Descrição Prática:

O *Domain Expert* deve atribuir para cada *User Story* um ID único, como o exemplo da Figura 4.9. Caso exista dependência entre *Users Stories*, a *User Story* que possui a dependência deve incluir o ID dos outros na caixa link, onde a *User Story* está descrita, no caso do REACT-M é utilizado como exemplo um cartão. Caso uma *User Story* seja muito grande e precise ser quebrada em menores, as *Users Stories* derivadas devem possuir um ID que permita rastreá-lo até a *User Story* que a gerou. Qualquer informação ou tarefa que surgiu de uma *User Story*, o *Team* deve incluir em sua descrição uma caixa para a inclusão do ID das *Users Stories* que a originou, como no exemplo da Figura 4.9. Qualquer artefato gerado a partir de uma *User Story* também deve possuir um ID que permita identificar o relacionamento parental, deste modo, permitindo um vínculo entre os diversos níveis de artefatos gerados.

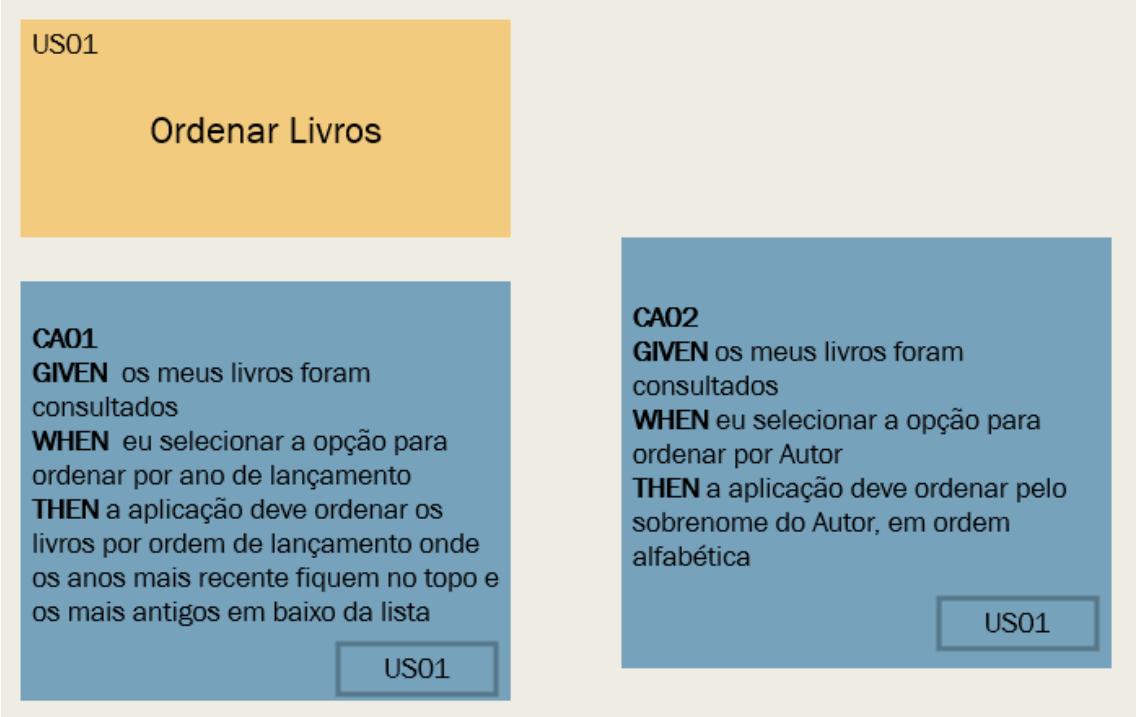


Figura 4.9 Exemplos de Rastreabilidade Entre Users Stories e Artefatos derivados.

4.2.5 Atividade de Revisar e Registrar Inconsistências

Os vários níveis de *User Stories* devem ser verificados e validados constantemente. A ideia é avaliar se as *User Stories* produzidas estão corretas, são testáveis, pequenas suficientes e se estão alinhadas às reais necessidades do *Customer* em relação ao produto. O REACT-M propõe que os artefatos gerados ao longo do projeto sejam sempre validados. Para facilitar a validação dos requisitos, sugere-se a utilização dos critérios INVEST semelhante ao que ocorre na atividade de Avaliar *Backlog* com a Equipe Técnica.

Quem participa: *Domain Experts*, *Facilitator*, *Team*.

Descrição Prática:

O *Team* deve validar os artefatos gerados a partir das tarefas que foram executadas dentro do ciclo de desenvolvimento. Para isso o REACT-M propõe utilizar novamente os critérios INVEST para avaliar a qualidade do que está sendo desenvolvido. Nem todos os critérios INVEST podem fazer sentido para o artefato que está sendo avaliado. Assim, o integrante do *Team* que está realizando a validação deve avaliar apenas os critérios possíveis de acordo com a tarefa executada. Caso seja

identificada alguma inconsistência, o integrante do *Team* deve gerar uma tarefa de defeito e colocar no *Team Kanban* para que seja corrigido, conforme descrito na atividade de Elaborar *Backlog* de Tarefas. Quando a inconsistência for corrigida, novo teste de validação deve ser feito para confirmar a correção.

4.2.6 Tratar Mudanças no Backlog

Esta atividade deve ser executada quando mudanças no produto sejam identificadas pelo *Domain Expert*. As mudanças devem ser pensadas como um sinal que o *Customer* está aprendendo com o andamento do projeto e que irá trazer melhores funcionalidades para o produto, tornando-os mais competitivos no mercado e alinhados aos interesses do usuário. Mesmo em projetos ágeis é importante que as mudanças possuam um nível de gerenciamento que não burocratize o desenvolvimento. O REACT-M sugere usar uma gerência leve para mudanças, utilizando práticas extraídas dos métodos *Scrum* e *Water-Scrum-Fall*, que possuem eventos que permitem, ao longo de um ciclo, que o *Customer* possa inclusive entender aquilo que está sendo construído e solicitar mudanças.

Quem participa: *Domain Experts*, *Customer*, *Facilitator* e *Team* (todos ou representantes).

Descrição Prática:

O *Customer* solicita uma mudança para o produto. Então, o *Domain Expert* deve atualizar o *Backlog* inserindo as novas mudanças. Em seguida, o *Domain Expert* e o *Customer* devem priorizar novamente o *Backlog*, utilizando a técnica de priorização *Business Goals* e *Constraint Goal* propostas no método REACT para DRE. O *Backlog* precisa ser novamente validado pelo *Team*, utilizando as práticas da atividade de Avaliar *Backlog* com a Equipe Técnica. Após essa sequência de tarefas, as mudanças estarão aptas para entrar nos próximos ciclos de desenvolvimento. Adicionalmente, todo ciclo proposto no REACT-M deve ser executado novamente para contemplar a mudança identificada.

Para melhor controlar as solicitações de mudança do projeto, o *Domain Expert* pode utilizar um formulário leve, onde o REACT-M propõe a utilização de um *template* conforme a Figura 4.10. A cada nova mudança solicitada pelo *Customer*, o *Domain*

Expert deve inserir no formulário um registro da mudança. Para isso: ele deve atribuir na coluna RQM (ReQuisição de Mudança) uma ordem sequencial da mudança solicitada; o campo Data da Solicitação deve ser preenchido com a data em que a mudança foi informada pelo *Customer*; a coluna Nome deve ser preenchida com o nome do *Customer* que solicitou a mudança, podendo ser uma pessoa ou entidade; a próxima coluna deve ser preenchida com uma breve descrição da mudança solicitada; por fim, a coluna esforço deve ser preenchida com o total de *Story Points* que o *Team* definiu para desenvolver a mudança.

Mudanças						
N°	Data da Solicitação	Responsável	Descrição	User Story		Esforço
				Impacto	Novas	
RQM01	28/05/2019	Gestora Luiza	Incluir a funcionalidade de compartilhar trecho do livro que está lendo nas redes sociais (Instagram, Facebook e Twitter)	US02; US03	US04; US05	12

Figura 4.10 Formulário para controle de mudanças.

5 AVALIAÇÃO DA ABORDAGEM PROPOSTA

Este capítulo apresenta a avaliação da proposta do REACT-M. Esta avaliação foi conduzida por meio de um Estudo de Caso realizado em um projeto de desenvolvimento de software pertencente ao Projeto SPIDER (*Software Process Improvement: DEvelopment and Research*), dentro da Universidade Federal do Pará – UFPA. O projeto SPIDER foi escolhido para a execução do experimento por ter interesse no estudo proposto; ter um ambiente favorável e acessível à execução do estudo de caso; e por possuir projetos de software que possibilitariam e seriam beneficiados com a execução do REACT-M.

Vale ressaltar que o Projeto SPIDER foi institucionalizado em 2009 no ICEN da UFPA e possui colaboradores como corpo integrante: da Graduação da Faculdade de Computação, a partir dos cursos de Bacharelado em Ciência da Computação e Bacharelado em Sistemas de Informação; do Doutorado e do Mestrado do PPGCC (Programa de Pós-Graduação em Ciência da Computação); do Doutorado do PPGEE (Programa de Pós-Graduação em Engenharia Elétrica); e do Mestrado e Doutorado do Centro de Informática da Universidade Federal de Pernambuco.

Os tópicos deste Capítulo são:

- Metodologia do Estudo de Caso;
- Fase de Planejamento;
- Fase de Execução;
    - Primeira Iteração de Desenvolvimento;
    - Segunda Iteração de Desenvolvimento;
- Fase de Avaliação;
- Sumarização dos Resultados do Estudo de Caso.

5.1 Metodologia do Estudo de Caso

Nesta seção o estudo de caso para a avaliação da aplicabilidade do REACT-M será descrito. O mesmo teve como objetivo avaliar a eficiência e aplicabilidade do método proposto em um ambiente real de desenvolvimento de software, a fim de identificar as forças, as fraquezas, as oportunidades de melhoria e as ameaças relacionadas ao uso do REACT-M. Para isto, um estudo de caso foi escolhido como método de pesquisa por se tratar de uma abordagem qualitativa observacional, onde o pesquisador interage de forma semiformal com os participantes da pesquisa por meio de entrevistas e conversas programadas a fim de levantar as avaliações ou opiniões destes sujeitos sobre um determinado fenômeno, solução, método, processo ou prática inserido no ambiente destes sujeitos (Wainer, 2007).

Este estudo de caso foi organizado em 3 fases distintas, conforme a imagem apresentada na Figura 5.1. Cada fase e suas respectivas etapas serão detalhadas nas próximas seções desta dissertação.

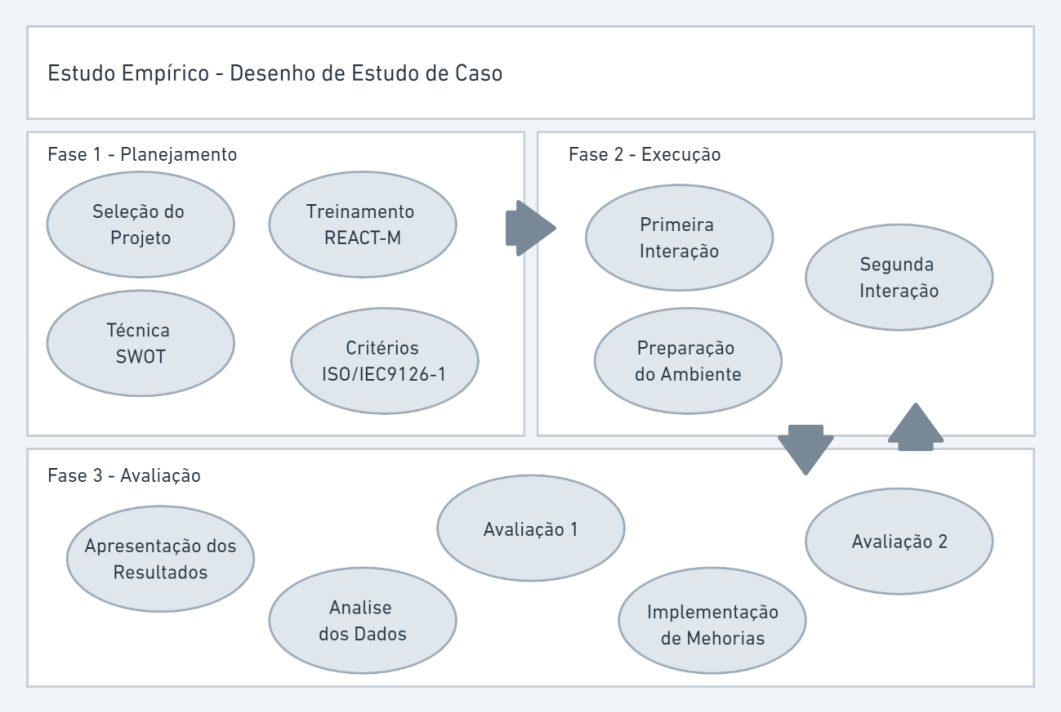


Figura 5.1 Desenho do Estudo de Caso.

Em conjunto com a Figura 5.1 é apresentada a estratégia metodológica aplicada para a condução do estudo de caso. Este quadro foi criado a partir das recomendações

de Wainer (2007) e de Silva e Menezes (2005), os quais definem as características de um estudo de caso, conforme o Quadro 5.1.

Quadro 5.1 Quadro metodológico do Estudo de Caso

Tipo de Pesquisa	Exploratória
Procedimento de pesquisa	Estudo de Caso
Natureza dos dados	Qualitativa
Tipo de registro dos dados	• Anotações de campo • Transcrição dos *Brainstormings*
Instrumento para coleta e análise dos dados	• Análise documental • *Brainstorming* • Aplicação da técnica de Análise SWOT, a fim de identificar forças, fraquezas, oportunidades de melhoria e ameaças quanto à aplicabilidade do REACT-M.

5.2 Fase de Planejamento

Inicialmente, foi feita a definição do tipo de projeto e da equipe que participaria do experimento. Para isto, foi estabelecido os critérios para a definição: I) a equipe deve utilizar métodos ágeis para o desenvolvimento de seus produtos de software; II) o projeto selecionado deve estar na etapa de ideação, ou seja, ainda não está bem definido e suas funcionalidades precisam ser descobertas. Este ambiente é propício para a execução do REACT-M, pois os requisitos do produto ainda não estão definidos, o que permite que a gerência de requisitos proposta pelo REACT-M seja feita desde a etapa de levantamento dos requisitos e continue pelas demais iterações do projeto.

A equipe selecionada para o estudo de caso foi composta por 5 Desenvolvedores, 1 Cliente, 1 Especialista do Domínio e um Facilitador do REACT-M. O projeto selecionado tinha como objetivo ensinar aos desenvolvedores programação *Front-end*, assim torná-los mais capacitados para a execução de outros projetos conduzidos dentro do laboratório SPIDER.

Duas iterações de desenvolvimento com duração de 14 dias úteis cada foram planejadas para a aplicação do REACT-M. Ao final de cada iteração, uma avaliação foi aplicada junto à Equipe. Para esta avaliação, a técnica de análise SWOT (*Strengths,*

Weaknesses, Opportunities e Threats) foi escolhida e adaptada para o contexto desta pesquisa, sendo aplicada por meio de um Formulário de Avaliação SWOT (consultar Apêndice C). Deste modo, objetivando avaliar os itens que compõem o REACT-M.

Além disso, critérios de avaliação foram definidos com o intuito de ajudar a aquipe ao longo da fase de avaliação do REACT-M. Optou-se por utilizar alguns critérios adaptados da norma ISO/IEC 9126-1 (ABNT, 2003), devido esta norma ser uma referência quanto à avaliação de qualidade de produto de software, a saber: Eficiência, Simplicidade, Utilidade, Aplicabilidade, Confiabilidade e Colaboração. Deste modo, ao final da avaliação os participantes foram convidados a preencher um formulário no qual deram suas considerações a respeito do método REACT-M.

Por fim, um pequeno treinamento junto à equipe foi conduzido, com a finalidade de ambientar os participantes do estudo de caso sobre o método REACT-M no projeto de desenvolvimento de software de responsabilidade desta equipe. O guia prático apresentado neste estudo foi entregue a cada participante para um melhor entendimento sobre o REACT-M.

5.3 Fase de Execução

Durante a execução REACT-M, o Facilitador e o Cliente participaram de todas as cerimônias do método junto com o restante da Equipe, o que proporcionou um ambiente fortemente colaborativo e participativo, como é tipicamente sugerido no uso de métodos ágeis.

O primeiro passo nesta fase foi preparar o ambiente com os materiais e ferramentas que a Equipe iria utilizar para a aplicação do REACT-M. A Equipe optou por utilizar recursos como *post-it*, parede e folhas de papel A4 para criar e manter os artefatos durante o momento de ideação do produto. No entanto, a Equipe também definiu a utilização da ferramenta *Whimsical* para o gerenciamento do *Backlog* e dos artefatos correlatos, porque a ferramenta simula a utilização de *post-its* e paredes, porém sem as limitações físicas.

5.3.1 Primeira Iteração de Desenvolvimento

Inicialmente, foi realizada uma cerimônia para o planejamento do produto que

seria desenvolvido, onde nesta reunião todos os participantes do estudo de caso estavam presentes. O Cliente também possuía muitas dúvidas sobre o produto que realmente iria satisfazer os seus usuários finais. Para auxiliar nessa etapa de descoberta, o Facilitador primeiramente aplicou a técnica de *Personas* conforme Figura 5.2, onde todos os participantes interagiram a fim mapear os objetivos dos fornecedores de requisitos do produto, que neste caso era um grupo formado pelos usuários-chave e o próprio cliente. Foi preciso o Facilitador aplicar abordagens ágeis para o desenvolvimento dos requisitos do produto, para isso foi usada uma adaptação de *Canvas,* que permitiu a equipe extrair das *Personas* quais *Features* o produto precisaria possuir, possibilitando uma visão de alto nível do produto como um todo. Para cada *Feature* identificada, foi utilizada a técnica de *Journey* que permitiu a equipe ter uma visão a partir de fluxo da interação da *Persona* com a *Feature* implementada no produto. Esta visão permitiu a extração dos PBI (*Product Backlog Itens*) que deveriam ser implementados no produto para entregar a *Feature*. Os PBI foram escritos no formato de *User Stories*, que facilita o entendimento do Cliente sobre o que seria desenvolvido. O conjunto desses PBI formou o *Backlog* do produto que foi priorizado pelo cliente com a técnica de *User Stoty Mapping*.

As abordagens de *Journeys* e *User Stories* são sugeridas pelo método REACT, pois o método REACT-M é especificamente voltado para a gerência de requisitos.

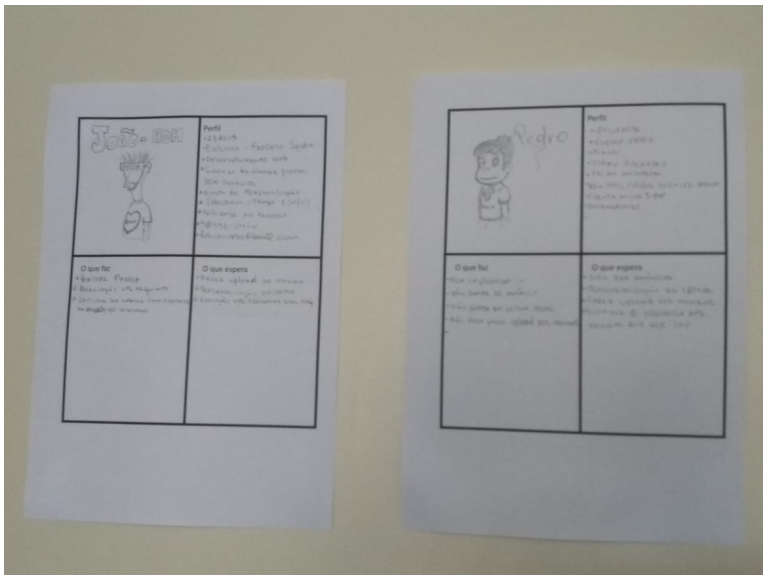


Figura 5.2 Personas Mapeadas do Produto.

Terminada esta etapa, os PBI foram migrados para o *Domain Expert Kanban* criado vitualmente na ferramenta *Whimsical*, conforme Figura 5.3. Todos os PBI foram colocados na lista *Product Backlog* do *Domain Expert Kanban*, com exceção dos PBI de maior prioridade e que foram definidos pelo cliente para a primeira entrega. Estes foram movidos para a lista *Sprint Backlog*, dando visão para todos do que deve ser desenvolvido na primeira iteração. Para cada PBI foi atribuído um código identificador único e cada PBI, caso o PBI possua dependência com outro, era colocado no cartão o código destes PBI gerando a rastreabilidade horizontal entre eles.

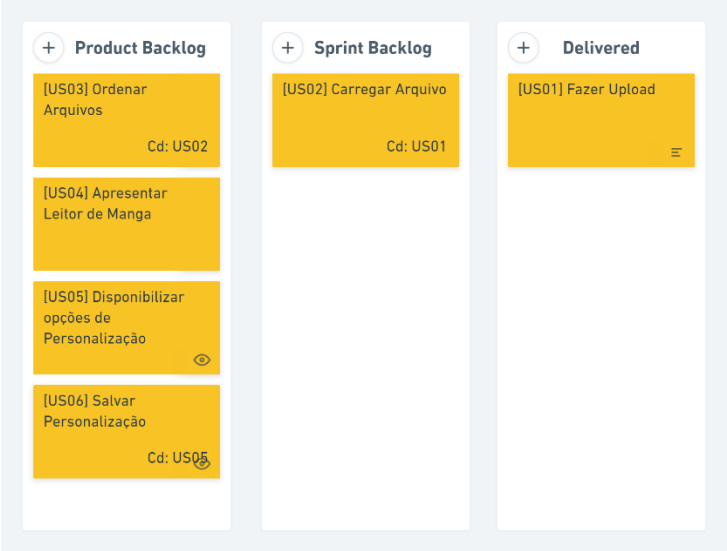


Figura 5.3 Domain Expert Kanban.

Para facilitar o fluxo de trabalho da Equipe dentro da iteração, como proposto no REACT-M, foi criado o *Team Kanban*, conforme a Figura 5.4, que tem foco no gerenciamento dos PBI que estão sendo desenvolvidos dentro da iteração corrente. Com o entendimento do que precisa ser desenvolvido na primeira entrega, os participantes puderam voltar suas atenções para estes PBI, e com a ajuda do Cliente e do Especialista do Domínio foram desenvolvidos os Critérios de Aceite destes PBI utilizando os conceitos de *Definition of Ready*, que forneceu detalhes do negócio importantes para a implementação dos PBI. Os critérios de aceite foram também criados na ferramenta *Whimsical*, onde para cada critério de aceite foi atribuído um código de identificação único, e também era colocado o código dos PBI que o critério de aceite pertencia, gerando uma rastreabilidade vertical entre os artefatos. A Equipe ficou encarregada de validar as informações dos PBI utilizando os critérios INVEST, conforme a Figura 5.5,

a fim de verificar se estavam adequados para serem desenvolvidos dentro da iteração.

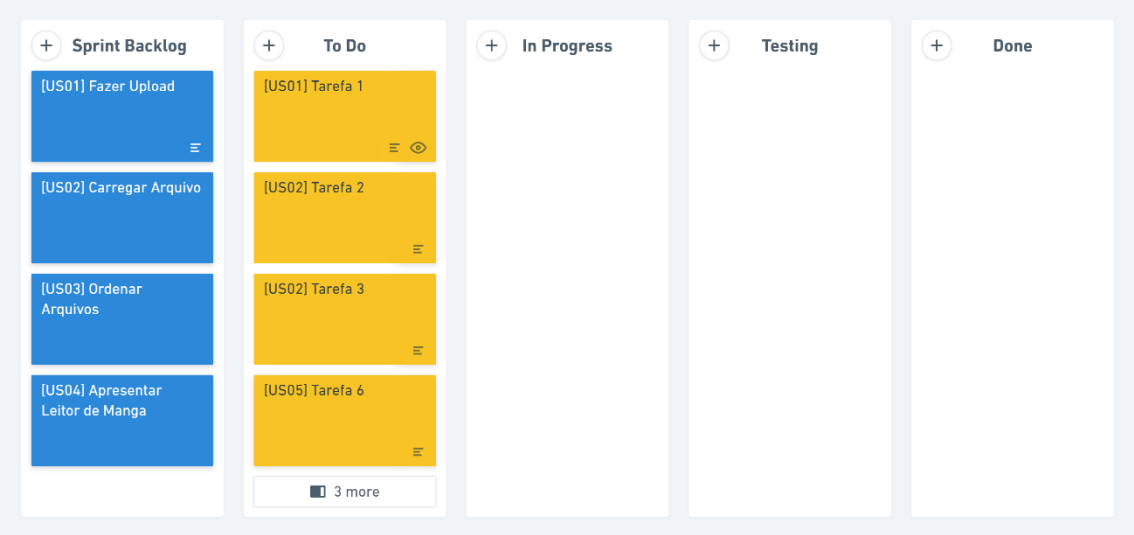


Figura 5.4 Team Kanban.



Figura 5.5 Critérios de Aceite.

Após a validação do PBI, a Equipe possuía informações suficientes para estimar o esforço que levaria para implementar os PBI. Para realizar a estimativa foi utilizada a técnica de *Planning Poker*, onde todos os membros participaram. Como resultado, chegou-se a um valor estimado de esforço para cada PBI e também a definição das tarefas técnicas que deviam ser implementadas. As Tarefas definidas foram criadas na lista *To Do* do *Team Kanban*, e para cada uma delas foi atribuído um código identificador único, também foi incluído o código identificador do PBI que a derivou,

gerando rastreabilidade entre os artefatos de trabalho. Concluindo, foi estipulado, de acordo com a soma de esforço dos PBI, o prazo de 14 dias para o término da iteração.

Durante a iteração, a Equipe utilizou o *Team Kanban* para se auto gerenciar, sempre pedindo ajuda do Facilitador caso precisasse. Destaco que para um PBI ser dado como *Done* no *Team Kanban*, eram realizados testes no incremento gerado. O incremento precisava ser validado de acordo com os critérios INVEST. Caso fosse identificada alguma inconsistência, era aberto um defeito e inserido na lista *To Do* para ser corrigido.

Ao fim da iteração de desenvolvimento o cenário era de muitas incertezas e descobrimentos. Contudo, ao fim desta iteração todos sabiam quais requisitos eram os mais prioritários para serem desenvolvidos, assim como já possuíam especificações suficientes e enxutas para iniciarem o desenvolvimento do produto, concomitantemente, com a execução do REACT-M.

5.3.2 Segunda Iteração de Desenvolvimento

Antes de iniciar a segunda iteração de desenvolvimento, uma avaliação do REACT-M foi realizada junto à Equipe. Após isto, um conjunto de melhorias no REACT-M foi concebido para, então, executar a segunda iteração de desenvolvimento já contemplando estas melhorias. Estas melhorias podem ser visualizadas no Quadro 5.2.

A segunda iteração iniciou com uma solicitação de mudança da parte do cliente. Para realizar a mudança no escopo foram utilizadas as medidas propostas no REACT-M, que consistem em entender a mudança solicitada e basicamente executar cada cerimônia do método novamente para incluir a mudança no *Backlog*. Nesta segunda iteração, em síntese, a Equipe já detinha um entendimento e detalhamento mais significativo sobre os requisitos do produto, bem como um bom domínio sobre a execução do REACT-M, permitindo assim a execução das cerimônias do REACT-M de maneira mais eficiente e ágil. Para controlar as mudanças do projeto a Equipe utilizou o formulário proposto pelo REACT-M, conforme a Figura 5.6.

Mudanças						
Nº	Data da Solicitação	Responsável	Descrição	PBIs		Esforço
				Impacto	Novas	
RQM01	26/10/2019	João	Incluir opção de pesquisa no Header do sistema.	US06	US07; US08	8

Figura 5.6 Formulário para controle de mudanças.

5.4 Fase de Avaliação

Ao final de cada iteração de desenvolvimento, uma avaliação foi realizada junto à Equipe a fim de obter suas considerações a respeito da aplicabilidade do REACT-M, permitindo a identificação das forças, das fraquezas, das oportunidades de melhoria e das ameaças relacionados aos itens que compõem o REACT-M, lembrando: ciclo de vida, artefatos, papéis e cerimônias.

Foram utilizados três instrumentos para a coleta de dados:

i) As avaliações foram realizadas no formato de um *Brainstorming*, pelo qual o pesquisador avaliador deste estudo de caso conduziu uma sessão em grupo com os participantes deste estudo de caso com o objetivo de levantar suas considerações e observações sobre o REACT-M;

ii) A técnica de Análise SWOT foi aplicada por meio do Formulário de Avaliação SWOT; e,

iii) A Equipe fez uma análise documental, ou seja, analisou todos os artefatos gerados ao longo do estudo de caso para fundamentar suas considerações e observações.

Desta forma, as transcrições dos *Brainstormings* e as anotações de campo foram registradas, analisadas e compiladas para gerar os resultados empíricos deste estudo de caso. Estes resultados podem ser conferidos nos quadros a seguir.

Quadro 5.2 Forças do REACT-M.

Critérios	S (*Strengths*)
Eficiência	**Artefato**: Equipe julgou que os artefatos ajudaram muito no desenvolvimento. **Artefato**: Equipe julgou que o artefato *Persona* foi um ótimo ponto inicial para direcionar as ideias do cliente a atingir o objetivo que era gerar o *Backlog* do Produto. **Artefato**: Equipe julgou que o artefato *Team Kanban* ajudou a gerenciar as tarefas que precisavam ser feitas na iteração. **Papeis:** Equipe julgou relevante a divisão dos papéis e atribuição das reponsabilidades, de modo que qualquer problema se sabia a quem consultar.
Simplicidade	**Papel**: Equipe julgou simples as responsabilidades de cada papel. **Artefato**: Equipe julgou simples o uso dos artefatos *Domain Expert Kanban* e *Team Kanban* **Artefato**: Equipe julgou simples o uso dos critérios INVEST para validar os itens do *Backlog*.
Utilidade	**Artefato**: A rastreabilidade foi bastante útil para ligar os artefatos, o que facilitou os desenvolvedores entenderem o que precisavam desenvolver. **Ciclo de vida**: Equipe achou interessante a ideia de dividir o que vai ser desenvolvido por ciclos, onde o que é prioridade para o cliente é implementado primeiro, mas levando também em consideração o esforço estimado pela equipe.
Aplicabilidade	
Confiabilidade	**Ciclo de vida**: Todas as etapas e artefatos tem ligação entre si, não há lacunas.
Colaboração em Equipe	**REACT-M**: A participação de toda a equipe foi muito significativa em todas as cerimônias, sempre discutindo e compartilhando conhecimento sobre o produto.

Quadro 5.3 Fraquezas do REACT-M.

Critérios	W (*Weaknesses*)
Eficiência	**Cerimônias:** Realizar a cerimônia de *Inception* toda em apenas um encontro
Simplicidade	**Artefato**: Equipe teve dificuldade de utilizar a ferramenta proposta para o gerenciamento do *Team Kanban*, onde se sugeriu a utilização de outra ferramenta como o Trello.
Utilidade	**Artefato**: Equipe julgou o uso dos critérios INVEST não tão relevante, tendo em vista que a Equipe participou de todo processo de levantamento e geração do *Backlog*, onde isto já indicava que estavam em conformidade com o *Backlog*.
Aplicabilidade	**Artefato:** Controlar as mudanças em um formulário apartado não foi considerado interessante pela equipe.

Critérios	W (*Weaknesses*)
Confiabilidade	**Artefato:** Equipe considerou realizar a rastreabilidade manual um ponto fraco, pois existem ferramentas que permitem gerar quase automática.
Colaboração em Equipe	

Quadro 5.4 Oportunidades de Melhoria do REACT-M.

Critérios	O (*Opportunities*)
Eficiência	**Artefato:** Tornar a validação do *Backlog* tarefa implícita no caso de toda equipe participar do levantamento do *Backlog*.
Simplicidade	**Artefato:** Equipe considerou que os critérios de aceite poderiam ficar juntos com o item de *Backlog*, como uma descrição do item, no caso da utilização de ferramenta.
Utilidade	
Aplicabilidade	
Confiabilidade	
Colaboração em Equipe	

Quadro 5.5 Ameaças ao REACT-M.

Critérios	T (*Threats*)
Eficiência	
Simplicidade	
Utilidade	
Aplicabilidade	**Artefato**: Aplicar os critérios INVEST para o teste do incremento ficou um pouco difícil para a equipe entender.
Confiabilidade	
Colaboração em Equipe	**Papeis:** Em alguns momentos a equipe não soube dividir as tarefas igualmente entre os membros, sobrecarregando alguns.

Por fim, a partir destes resultados empíricos, um conjunto de melhorias foi identificado para ser aplicado em uma nova versão do REACT-M com a finalidade de garantir o melhor uso deste método ágil para o desenvolvimento de requisitos de software. Estes ajustes foram realizados em alguns itens específicos que compõem o REACT-M, como artefatos ou cerimônias. Desta forma, proporcionando um método mais flexível, eficiente, ágil e realista ao contexto atual de desenvolvimento de software. Estes ajustes podem ser conferidos no Quadro 5.6.

Quadro 5.6 Melhorias no REACT-M.

Item do REACT-M	Melhoria
Critérios INVEST	1. Tornar opcional, caso todos os membros da equipe participem do levantamento do *Backlog*, a validação do mesmo, tornando-se implícita. 2. Possibilitar que a equipe escolha quais critérios do INVEST irá utilizar para testar o incremento.
User Story Mapping	1. No caso de uso de uma ferramenta que gere rastreabilidade, não exigir que seja feita manualmente. 2. O critérios de aceite gerados para um item do *Backlog* podem ficar descritos juntos, sem a necessidade de criar outro artefato para isso.
Kanban	1. Utilizar uma ferramenta que facilite o acesso da equipe 2. Incluir prática de atribuição de tarefas para os membros da equipe
Formulário de Mudança	1. Tornar opcional o uso deste formulário, onde a equipe deve definir quando e para qual projeto deve utilizar.
Inception	1. Realizar a cerimônia a cada nova iteração.

6 CONCLUSÕES

Neste capítulo serão apresentados os principais resultados obtidos com a execução do estudo. Na sumarização dos resultados serão destacados pontos como: I) as principais contribuições à área da Engenharia de Requisitos e dos Métodos Ágeis; II) as limitações deparadas na execução do projeto; e III) algumas oportunidades de melhorias identificadas e propostas de trabalhos futuros a serem executados a partir do estudo realizado.

6.1 Sumarização dos Resultados deste Estudo

O estudo realizado concentrou-se em investigar na literatura especializada abordagens existentes nos métodos ágeis para apoio às atividades de Gerência de Requisitos no contexto de projetos de desenvolvimento de software. Lembrando que o processo de Gerência de Requisitos é um processo que dá suporte ao processo de Desenvolvimento de Requisitos e possui grande importância para controlar com qualidade a evolução dos requisitos de um produto de software. Por tal importância, o processo está contido nos modelos de qualidade CMMI-DEV e MR-MPS-SW.

As abordagens ágeis que deram suporte ao estudo foram coletadas por meio de uma Revisão Sistemática da Literatura, a qual identificou diversas abordagens ágeis de apoio ao processo de Engenharia de Requisitos. Em posse dessas abordagens ágeis, foi realizado um procedimento de avaliação da qualidade que coletou um grupo menor de abordagens voltadas ao processo de Gerência de Requisitos. A partir da análise destas evidências empíricas, bem como das práticas da Engenharia de Requisitos Ágil e das boas práticas do processo de Gerência de Requisitos constantes nos modelos de qualidade CMMI-DEV e MR-MPS-SW, uma proposta de método ágil para a gerência de requisitos de software foi elaborada, o REACT-M, apresentado no Capítulo 4 desta

dissertação. É importante ressaltar que o REACT-M pode ser considerado uma extensão do método REACT desenvolvido para o processo de desenvolvimento de requisitos.

O REACT-M difere-se de outras abordagens ágeis voltadas à Engenharia de Requisitos por propor um método específico para a Gerência de Requisitos que indica abordagens ágeis que podem ser usadas para executar as atividades de: I) identificar os fornecedores de requisitos; II) elaborar o *Backlog* do produto; III) avaliar o *Backlog* com a equipe técnica; IV) gerar rastreabilidade bidirecional entre os artefatos; V) revisar o *Backlog* com a finalidade de identificar possíveis inconsistências; e VI) tratar mudanças no *Backlog*. Além disso, o método preocupa-se em descrever as atividades dentro de cerimônias que possuem *time-box* definidos pelos participantes. É importante frisar que o método foi concedido de modo que possa ser utilizado tanto em projetos pequenos como em projetos complexos de grande porte, visto que requisitos são o pilar para a construção do produto, o que torna importante a gerência destes para a boa condução do projeto. O REACT-M propõe-se a ser aplicado em diferentes contextos de projetos de desenvolvimento de software a fim de tornar a gerência de requisitos algo mais prático e menos burocrático, o que permite a equipe focar na entrega de valor ao fim do ciclo de desenvolvimento.

Após a construção do REACT-M, um estudo de caso foi realizado com o objetivo de avaliar a aplicabilidade desta proposta de método ágil em um projeto real de desenvolvimento de software. Como resultado percebeu-se que o REACT-M auxiliou a equipe a gerenciar os requisitos de maneira mais simples, organizada, eficiente, colaborativa e ágil. O estudo de caso foi executado apenas nos dois primeiros ciclos de desenvolvimento do projeto, no entanto a equipe continuou utilizando o REACT-M até o fim do projeto, devido julgarem o REACT-M como um método que ajudou a organizar uma parte importante do fluxo de trabalho da equipe.

Por fim, esta pesquisa pretendeu contribuir com as áreas de engenharia de software, métodos ágeis e de melhoria do processo de software fornecendo um método ágil para o gerenciamento de requisitos de um produto de software, o qual pode ser aplicado na indústria de software por profissionais ou empresas que buscam evoluir os requisitos de seus produtos de software a partir de métodos ágeis, bem como servir de

referência no meio científico para a realização de outros estudos com interesse nestes tópicos de pesquisa.

6.2 Contribuições

A seguir algumas contribuições obtidas durante o desenvolvimento deste estudo são apresentadas:

- **Revisão Sistemática da Literatura** – a revisão sistemática da literatura avaliou 6510 artigos, selecionando 177 destes por meio da aplicação de um protocolo de revisão sistemática. O protocolo criado, incluindo toda sua estratégia definida, é flexível o suficiente para ser utilizado em outros contextos, além de Engenharia de Requisitos e Métodos ágeis. Esta revisão sistemática gerou um arcabouço de evidências empíricas sobre métodos ágeis e requisitos, o qual pode servir de referência para fundamentar outras pesquisas no contexto da Engenharia de Requisitos Ágil;
- **Abordagem Ágil** – A criação do método REACT-M foi uma das principais contribuições do estudo, sendo embasada a partir de evidências empíricas obtidas a partir da execução da Revisão Sistemática da Literatura, das práticas da Engenharia de Requisitos Ágil e das boas práticas constantes nos modelos de qualidade CMMI-DEV e MR-MPS-SW. Espera-se que esta abordagem auxilie na execução das atividade de gerência de requisitos em projetos de desenvolvimento de software dentro de organizações que almejam implementar a melhoria do processo de software por meio de métodos ágeis;
- **Produção de Trabalho Científico** – houve a publicação de dois artigos no contexto desta dissertação de mestrado: na conferência "*16th International Conference on Information Systems and Technology Management*" (CONTECSI 2019), onde foi publicado os resultados da Revisão Sistemática da Literatura conduzida neste estudo; e na conferência "*Computer on the Beach*" que ainda irá acontecer em abril de 2020, onde foi será publicado o método ágil REACT-M.

6.3 Limitações

Uma das limitações deparadas ao longo do projeto foi a execução da Revisão Sistemática da Literatura (RSL) por apenas dois pesquisadores, o que pode acarretar no problema de viés de seleção dos estudos. No entanto, como medida para minimizar esta questão, um protocolo formal descrevendo de maneira organizada o passo-a-passo de procedimentos que deveriam ser executados para a coleta de estudos foi definido. Além disso, foi realizado um teste-piloto que possibilitou que os pesquisadores entendessem as regras da RSL, bem como melhorassem pontos do protocolo para garantir a confiabilidade do procedimento. Ressalta-se que todas as etapas do processo de RSL foram validadas por um pesquisador com grande experiência na condução de revisões sistemáticas.

Outra dificuldade foi alcançar uma definição consensual entre os pesquisadores sobre a categorização das abordagens ágeis em técnicas, métodos e práticas durante a etapa de extração e síntese dos dados da RSL. Como solução para esta questão, adotou-se as recomendações feitas por Falbo (1998), que consiste em uma ontologia para ser utilizada em ambientes de desenvolvimento de software. Ainda sobre a RSL, outra limitação está relacionada à quantidade de fontes de pesquisas consultadas no estudo, que devido ao volume grande de estudos que possuem, apenas buscas automatizadas foram realizadas, entretanto outras fontes poderiam ser consultadas por meio de busca manual, o que poderiam também contribuir para os objetivos do estudo, porém a pesquisa tornar-se-ia inviável no período disponível deste estudo.

Uma característica que pode ser julgada como limitação do REACT-M é a necessidade da presença do *Customer* em algumas atividades. Sabemos que em projetos reais o cliente nem sempre possui disponibilidade para participar de cerimônias como as que o REACT-M propõe. Outra limitação do REACT-M está ligada ao fato do mesmo estar fundamentado apenas nas abordagens ágeis encontradas nos estudos primários publicados em eventos ou periódicos científicos até a data de realização da RSL, excluindo-se algumas tendências sobre métodos ágeis e requisitos constantes na indústria de software atual, as quais não são ou ainda não foram publicadas por meio destes meios científicos.

Outra limitação enfrentada está ligada ao estudo de caso que foi executado apenas em um projeto específico dentro de ambiente acadêmico, pois não se experimentou a utilização do método em outros ambientes, apesar desta ser uma premissa do REACT-M, a possibilidade de ser escalável e aplicável em diferentes contextos de projetos de desenvolvimento de software, nos quais a equipe possui interesse de evoluir os requisitos do produto com adoção de abordagens ágeis. Além disso, outras estratégias poderiam ser utilizadas para a validação do método REACT-M, como *surveys* com especialistas em métodos ágeis ou experimentos com projetos de desenvolvimento de software. Contudo, o período disponível para este estudo foi insuficiente para realizar esta triangulação.

6.4 Trabalhos Futuros

Este estudo possui como tema uma área de grande abrangência, o que torna possível a sua expansão em diversas direções. Neste sentido, espera-se que esta dissertação seja utilizada por outros pesquisadores na evolução da Engenharia de Requisitos Ágil. O restante desta seção relata alguns dos tópicos do estudo que podem ser evoluídos em novos trabalhos.

6.4.1 Evolução da Revisão Sistemática

A RSL identificou diversas abordagens ágeis voltadas à Engenharia de Requisitos, porém outras abordagens ágeis podem não ter sido identificadas nos estudos primários, devido ao número de fontes pesquisadas ou até mesmo ao período de busca, já que existem eventos especializados na área possibilitando então que novas abordagens ágeis para requisitos sejam descobertas. Sendo assim, uma possível evolução no protocolo de revisão pode ajudar a encontrar evidências empíricas que possivelmente não tenham sido identificadas neste estudo.

6.4.2 Evolução da Abordagem Ágil proposta

Com o surgimento de novos estudos publicados a respeito da Engenharia de Requisitos Ágil, novas abordagens podem surgir para apoio às atividades de gerência de requisitos de software, o que pode tornar o método REACT-M propício a ser atualizado futuramente. Por exemplo, expandir o REACT-M a partir da realização de *surveys* com especialistas em métodos ágeis a fim de identificar novas tendências sobre a Agile-RE.

6.4.3 Utilização da abordagem em empresas que buscam maturidade

Considerando que o REACT-M está alinhado com as boas práticas constantes no processo de Gerência de Requisitos do MR-MPS-SW e na área de processo *Requirements Management* do CMMI-DEV, aplicar esta abordagem no contexto de uma empresa que busca implementar a melhoria do processo de software seria uma ótima oportunidade para validar sua fundamentação de forma prática em relação a algum destes modelos de qualidade.

REFERÊNCIAS BIBLIOGRÁFICAS

AGILE MANIFESTO. **Manifesto for Agile Software Development**. 2001. Disponível em: <http://agilemanifesto.org/>. Acesso em: Dezembro/2017.

BERTOLLO, G.. **Definição de Processos Em Um Ambiente de Desenvolvimento De Software.** Universidade Federal do Espírito Santo: Vitória, 2006.

BIOLCHINI, J.; MIAN, P.G.; NATALI, A.C.; TRAVASSOS, G.H. **Systematic Review in Software Enginnering: Relevance and Utility**. Relatório Técnico ES-679/05, PESC – COPPE/UFRJ, 2005.

CAO, L. C. L.; RAMESH, B. **Agile requirements engineering practices: An empirical study**. IEEE Software, 25(1), pp. 60–67, 2008.

COHN, M. **Desenvolvimento de Software com Scrum: Aplicando Métodos Ágeis com Sucesso**. Bookman: 1st Ed., ISBN-10: 8577808076, 2011.

COSTA, C. S. (2010): **Uma abordagem baseada em evidências para o gerenciamento de projetos no desenvolvimento distribuído de software**. Dissertação de Mestrado – Programa de Pós-Graduação em Ciência da Computação - Universidade Federal de Pernambuco, Recife.

COUNSELL, C. **Formulating questions and locating primary studies for inclusion in systematic reviews**. Ann Intern Med. v.1, n.127, p.380-7, 1997.

DANEVA, M.; INAYAT, I.; MORAES, L.; SALIM, S. **A Reflection on Agile Requirements Engineering: Solutions Brought and Challenges Posed**. XP 2015 Workshops, Helsinki, Finland, 2015.

DE MEDEIROS, J.F., RIBEIRO, J.L.D., Cortimiglia, M.N., **Success factors for environmentally sustainable product innovation: A systematic literature review**. J. Clean. Prod. 65, 76–86. 2014. doi:10.1016/j.jclepro.2013.08.035.

DEGRACE, P.; STAHL L. H. **Wicked problems, righteous solutions: a catalogue of modern so-ware engineering paradigms**. Nova Jersey: Yourdon Press, 1990.

DYBA, T.; DINGSOYR, T. **Empirical Studies of Agile Software Development: A Systematic Review**. Information and Software Technology, Butterworth-Heinemann Newton, MA, USA, v. 50, n. 9, pp. 833-859, 2008.

FALBO, R. A. (1998): **Integração de conhecimento em um ambiente de desenvolvimento de software**. Tese de Doutorado – Coordenação dos Programas de Pós-Gradução de Engenharia (COPPE) - Universidade Federal do Rio de Janeiro, Rio de Janeiro.

GARZA-REYES, J.A., **Green lean and the need for Six Sigma**. Int. J. Lean Six Sigma 6, 226–248, 2015. doi:10.1108/IJLSS-04-2014-0010.

HEIKKILÄ, V. T.; LASSENIUS, C.; DAMIAN, D.; PAASIVAARA, M. **A Mapping Study on Requirements Engineering in Agile Software Development**. 41 st Euromicro Conference on Software Engineering and Advanced Applications, 2015.

KÄPYAHO, M.; KAUPPINEN, M. **Agile requirements engineering with prototyping: A case study**. IEEE 23rd International Requirements Engineering Conference (RE). DOI: 10.1109/RE.2015.7320450. Publisher: IEEE. Ottawa, ON, Canada, Aug., 2015.

KITCHENHAM, B.; CHARTERS, S. **Guidelines for performing Systematic Literature Reviews in Software Engineering**. Keele University and Durham University Joint Report. EBSE 2007-001, Keele, 2007.

MAFRA, S.; BARCELOS, R. TRAVASSOS, G. **Aplicando uma metodologia Baseada em Evidência na Definição de Novas Tecnologias de Software**. In Proceedings of the 20th Brazilian Symposium on Software Engineering (SBES 2006), v. 1, pp. 239-254, 2006.

MEDEIROS, J. D. R. V.; ALVES, D. C. P.; et al. **Engenharia de requisitos em projetos ágeis: uma revisão sistemática da literatura**. Revista Principia Nº 26 Edição Especial, 2015.

NHSCRD - NHS Centre for Reviews and Dissemination. **Database of Abstracts of Reviews of Effectiveness**. In: The Cochrane Library, Issue 1. Oxford: Updated quarterly, 2003.

OLIVEIRA, S. R. B. **Processo de Software: Princípios, Ambientes e Mecanismos de Execução. Exame de Qualificação** - Universidade Federal de Pernambuco. Recife, 2006.

OLIVEIRA, S. R. B.; VASCONCELOS, A. M. L. **Qualidade, Gestão e Processos de Software**. 1th Ed., Editora UFPE, ISBN 978-85-415-0733-2, 2016.

PALOMINO, M.; DÁVILA, A.; MELENDEZ, K.; PESSOA, M. (2016): **Agile Practices Adoption in CMMI Organizations: A Systematic Literature Review**. In: MEJIA J., MUÑOZ M., ROCHA Á., SAN FELIU T., PEÑA A. Trends and Applications in Software Engineering. CIMPS. Advances in Intelligent Systems and Computing, vol 537. Springer, Cham. DOI: <https://doi.org/10.1007/978-3-319-48523-2_6. 2016>.

PARSONS, D.; RYU, H.; LAL, R. **The impact of methods and techniques on outcomes from agile software development projects**. In: McMASTER, Tom *et al* (org.). Organizational Dynamics of Technology-Based Innovation: Diversifying the Research Agenda, Springer, Boston, pp. 235–249, 2007.

PRESSMAN, R. S.; MAXIN, B. R. **Engenharia de Software: uma abordagem professional**. 8th Ed., McGraw-Hill. ISBN: 9788580555332. 2016.

QUSEF, A.; DE LUCIA, A. **Requirements Engineering in Agile Software Development**. Journal of Emerging Technologies in Web Intelligence, VOL. 2, NO. 3, August, 2010.

SABBAGH, R. **Srum, Gestão Ágil para Projetos de Sucesso**. Editora Casa do Código, 04101-300 – Vila Mariana – São Paulo – SP – Brasil, 2014.

SAMPAIO, R. F.; MANCINI, M. C. **Systematic review studies: a guide for careful synthesis of the scientific evidence.** Revista Brasileira de Fisioterapia, São Carlos, v.11, n.1, p. 77-82, jan./fev. 2007. doi: http://dx.doi.org/10.,1590/S1413-35552007000100013 . Disponível em: <http://www.scielo.br/scielo.php?pid=S1413-35552007000100013&script=sci_arttext>. Acesso em: 13 nov. 2013.

SANTOS, K. B. C. OLIVEIRA, S. R. B. **Uma Proposta de Método Ágil para apoio ao Processo de Desenvolvimento de Requisitos no Contexto de Projetos de Desenvolvimento de Software**. Programa de Pós-Graduação em Ciência da Computação - Universidade Federal do Pará, Pará, 2018.

SEHRISH A., SHAH S. A. A. **Impact and Challenges of Requirement Engineering in Agile Methodologies: A Systematic Review**. Bahria University Islamabad, Pakistan, 2017.

SCHWABER, K. **The Enterprise and Scrum. Redmond**: Microsoft Press, 2007.

SCHWABER, K.; BEEDLE, M. **Agile Software Development with Scrum**. 1th Edition, Prentice Hall, 2002.

SEI - Software Engineering Institute. **Capability Maturity Model Integration (CMMI) for Development, Version 1.3**. Carnegie Mellon, USA, 2010.

SCHÜTZ, G. R.; SANT'ANA, A. S. S.; SANTOS, S. G. **Política de periódicos nacionais em Educação Física para estudos de revisão sistemática.** Revista Brasileira de Cineantropometria do Desempenho Humano, Santa Catarina, v. 13, n. 4, p.313-319, 2011. doi: 10.5007/1980-0037.2011v13n4p313. Disponível em: <http://www.scielo.br/pdf/rbcdh/v13n4/11.pdf>. Acesso em: 13 nov. 2013.

SILVA, E. L.; MENEZES, E. M. **Metodologia da Pesquisa e Elaboração da Dissertação**. 4. ed. rev. Atual – Laboratório de Ensino a Distância da UFSC, 2005.

SILVA, F.Q.B.; SANTOS, A.C.C.; BORGES, C.C.; CARNEIRO, D.E.S. **Experimentação em Engenharia de Software: Glossário de Termos**. Proceedings of 7th Experimental Software Engineering Latin American Workshop (ESELAW'10), Goiânia, GO, Brazil (2010) 20-29.

SOFTEX – Associação para Promoção da Excelência do Software Brasileiro. **Melhoria do Processo de Software Brasileiro (MPS.BR) - Guia Geral,** 2016a.

SOFTEX – Associação para Promoção da Excelência do Software Brasileiro. **Guia de Implementação – Parte 1: Nível G**, 2016b.

SOMMERVILLE, I. **Software Engineering**. 10th Ed., Pearson Inc, Addison-Wesley. ISBN-10: 0-13-703515-2, 2019.

STANDISH GROUP. **Extreme Chaos Report**. The Standish Group International Inc. (2014). Disponível em: <*http://bit.ly/t1U1G* >. Acesso em: Abril/2017.

VACARI, I. **Um estudo empírico sobre a adoção de métodos ágeis para desenvolvimento de software em organizações públicas**. Dissertação de Mestrado. PUC-RS: Faculdade de Informática da Pontifícia Universidade Católica do Rio Grande do Sul, Porto Alegre, RS, Brasil, 2015.

VERSIONE. **The 12th annual The State of Agile Development**, 2017. Disponível em: <http://stateofagile.versionone.com>. Acesso em: Fevereiro/2018.

WAINER, J. **Métodos de pesquisa quantitativa e qualitativa para a Ciência da computação**. JAI 2007-Jornada de Atualização em Informática - Anais do XXVII Congresso da Sociedade Brasileira de Computação, 2007.

WEIGERS, K; BEATTY, J. **Software Requirements: best practices**. 3th Ed., Redmond: Microsoft Press. ISBN: 978-0-7356-7966-5, 2013.

APÊNDICE A – PROTOCOLO DA REVISÃO SISTEMÁTICA DA LITERATURA

Histórico de Revisões

Data	Versão	Descrição	Autor
10/06/18	0.1	Criação do documento de protocolo da Revisão sistemática da Literatura.	Bleno
16/06/18	0.2	Definição dos Critérios de Inclusão e Exclusão	Bleno
06/07/18	0.3	Definição dos critérios de avaliação da qualidade e estratégia de extração dos dados.	Bleno
10/07/18	1.0	Revisão e Validação do Protocolo	Sandro

Protocolo de Revisão

1. Contexto

O projeto SPIDER (acrônimo para S*oftware Process Improvement – DEvelopment and Research*) surgiu em 2009 com o objetivo de propor abordagens sistematizadas para apoiar a implementação de programas de melhoria do processo de software. Com relação à abordagem para sistematizar a implementação de processos, o Projeto SPIDER tem como possibilidades: Utilização de ferramentas já existentes no mercado; A adequação de ferramentas já existente no mercado; A criação de novas ferramentas. O Projeto SPIDER foca, ainda, na utilização de software livre, o que proporciona a liberdade: de executar o programa; de estudar como o programa funciona, e adaptá-lo para as suas necessidades; de redistribuir cópias; de modificar o programa; e de liberar

estas modificações (GNU, 2011). Utilizar software livre para a implementação de modelos de qualidade pode ocasionar em diminuição de custos e tempo de implementação (OLIVEIRA, 2010).

A Engenharia de Requisitos (ER) é uma área da Engenharia de Software de grande importância para o sucesso dos projetos de desenvolvimento de software. Utilizada em um contexto tradicional de desenvolvimento de software (modelos sequenciais, documentação rigorosa e extensa, cliente distante, equipe pouco colaborativa...) é denominada Engenharia de Requisitos Tradicional. Basicamente, ela é dívida em dois grupos de atividades relacionadas: Desenvolvimento de Requisitos e Gerência de Requisitos. Um desenvolvimento de requisitos criterioso é condição fundamental para o sucesso do projeto, pois os requisitos formam o alicerce para todo o ciclo do projeto, do desenvolvimento até a manutenção (SOFTEX, 2016). Segundo o SWEBoK (IEEE, 2014) e Wiegers *et al.* (2013).

A pesar de ter havido várias contribuições na área de ER, incorporando novas técnicas, ferramentas, métodos e metodologias de apoio às suas atividades, como exemplo os métodos ágeis, os quais proporcionaram diversos benefícios à área de desenvolvimento de software, ainda existem *gaps* e uma falta de visão consolidada e coerente sobre como esses métodos ágeis podem ser utilizados na ER, principalmente, no processo de Gerência de Requisitos. Dessa forma, surgiu a necessidade de estudar com mais detalhes a relação da ER no contexto dos métodos ágeis, a qual é conhecida como Engenharia de Requisitos Ágeis. Por exemplo: como seria a atividade de validação de requisitos no contexto dos métodos ágeis?

Portanto, esta Revisão Sistemática da Literatura (RSL) é parte de um projeto de dissertação de mestrado vinculado ao Projeto SPIDER com o objetivo de identificar quais as principais abordagens, existentes nos métodos ágeis, para apoio às atividades do processo de Engenharia de Requisitos, permitindo que os resultados desta RSL fundamentem a proposta de criação de um método ágil de apoio ao processo de Desenvolvimento de Requisitos.

2. Objetivos

Esta RSL terá como objetivo a identificação de abordagens existentes nos métodos ágeis para o Engenharia de Requisitos no contexto de projetos de desenvolvimento de software. Desta forma, têm-se a seguinte estrutura, conforme proposto em Santos (2010):

Analisar: relatos de experiência e publicações científicas através de um estudo baseado em revisão sistemática.

Com o propósito de: identificar abordagens existentes nos métodos ágeis para apoio ao processo de Desenvolvimento de Requisitos.

Com relação: a definição e uso de métodos, técnicas e práticas, no contexto dos métodos ágeis, empregados para a implantação e execução das atividades de Desenvolvimento de Requisitos em projetos de desenvolvimento de software.

Do ponto de vista: de pesquisadores e organizações desenvolvedoras/mantenedoras de software.

No contexto: acadêmico e industrial.

3. Referências

KITCHENHAM, B. (2007) "Guidelines for performing Systematic Literature Reviews in Software Engineering". Vol 2.3 EBSE Technical Report, EBSE-2007-01, 2007.

MAFRA, S., TRAVASSOS, G. (2005) "Técnicas de Leitura de Software: Uma Revisão Sistemática". XIX Simpósio Brasileiro de Engenharia de Software (SBES 2005).

SANTOS, G. S. (2010) "Revisão Sistemática, Mini-Curso". Simpósio Brasileiro de Qualidade de Software – SBQS 2010, Belém – PA.

4. Formulação da Pergunta

Com base no objetivo de investigação desta RSL a seguinte questão de pesquisa principal foi definida:

Quais são as abordagens existentes nas metodologias ágeis que apoiam as atividades de Gerência de Requisitos no contexto de projetos de desenvolvimento de software?

Por "abordagens", as quais se pretende investigar, entende-se como métodos, técnicas e práticas. A saber:
Método: um conjunto de técnicas, práticas e regras, organizado de forma sistemática e executado por meio de um passo-a-passo para alcançar um objetivo específico;
Prática: uma ação recomendada, a qual define o que pode ser feito para obter um resultado esperado;
Técnica: um roteiro específico de como realizar uma atividade.

4.1 Estrutura da Questão Principal

A questão levantada foi organizada conforme a estrutura *Population, Intervention, Context, Outcomes, Comparison* (PICOC), recomendada por Kitchenham (2007). Entretanto, apenas os itens População, Intervenção e Resultados foram considerados relevantes para a pesquisa. Tal restrição, segundo Santos (2010), caracteriza esta pesquisa como uma Revisão QUASI Sistemática da Literatura.
Nesse sentido, definiu-se a seguinte estrutura para a questão de pesquisa principal:

- **População (*P*):** Organizações de Software e Projetos de Software;
- **Intervenção (*I*):** Abordagens existentes nos métodos ágeis para apoiar as atividades de Desenvolvimento de Requisitos;
- **Resultados (*O*):** Métodos, técnicas e práticas ágeis para o desenvolvimento de requisitos de software.

4.2 Questões Secundárias

Um conjunto de questões secundárias referentes à questão principal foi estabelecido, questões essas para serem respondidas durante a fase de extração de informações. Tais

questões têm objetivo de esclarecer detalhes importantes que esta revisão procura identificar, para colaborar com o projeto onde esta se insere.

QS1 - Quais atividades de Gerência de Requisitos são apoiadas pelas abordagens encontradas?

QS2 - Caso a abordagem seja uma prática ou técnica, a mesma pertence a algum método ágil existente? Qual?

QS3 - Qual o contexto de aplicação da abordagem encontrada?

QS4 - Quais os ativos (papéis e artefatos) envolvidos nessas abordagens?

5. Escopo da Pesquisa

O escopo desta RSL é composto por critérios para seleção das fontes de pesquisa e restrições para garantir a viabilidade da mesma.

5.1 Critérios de Seleção de Fontes

Para seleção das fontes de pesquisa, foram definidos os seguintes critérios:

- Disponibilidade para consultas web.
- Disponibilidade para busca de artigos através do domínio da UFPA.
- Disponibilidade de artigos na íntegra através do domínio da UFPA ou a partir da utilização da *engine* de busca Google e/ou Google Scholar.
- Disponibilidade de artigos em inglês ou português.
- Relevância da fonte.

5.2 Restrições

Devido ao caráter da pesquisa, esta não deverá incorrer em ônus financeiro aos pesquisadores portanto apenas serão selecionadas as fontes que possibilitem consultas de forma gratuita ou a partir do acesso pelo domínio da UFPA. Serão apenas considerados os estudos obtidos através das fontes selecionadas e em conformidade com os critérios de inclusão e exclusão. A pesquisa estará restrita aos resultados publicados entre janeiro de 2001 até dezembro de 2015, contemplando, desta forma, um período de mais de 15 anos, dada a necessidade de identificar abordagens de apoio às atividades de Desenvolvimento de Requisitos no contexto dos métodos ágeis, uma vez que, os métodos ágeis se consolidaram a partir da criação do manifesto ágil em 2001.
Serão considerados apenas artigos em inglês ou português devido à proficiência dos autores nestes idiomas.

5.3 Métodos de busca nas Fontes

Os métodos de busca nas fontes podem ser definidos como: método de busca manual e método de busca automática.

5.3.1 Método de Busca Manual

A busca manual utiliza as fontes onde os estudos estão inseridos em anais de eventos e não disponíveis em um repositório digital e sim em um documento digital. Na busca manual, os anais das conferências são consultados para um determinado período

específico. Todos os estudos desses anais serão avaliados de acordo com as questões de pesquisa.

5.3.2 Método de Busca Automática

A pesquisa será realizada através de buscas automáticas em repositórios de estudos científicos na internet. Essas buscas são feitas por meio da "*string* de busca", uma composição de palavras-chave derivadas das questões de pesquisa e organizadas logicamente. Poderão ser necessárias eventuais buscas manuais por artigos que não sejam disponibilizados na íntegra nesses repositórios, ou seja, artigos que possuam versão contendo apenas resumo/abstract. Para estes casos, essa busca pode ser realizada por meio dos mecanismos de busca web Google (http://www.google.com.br/) e Google Scholar (http://scholar.google.com.br/).

5.4 Palavras-Chave e Sinônimos

A partir das questões de pesquisa, palavras-chave foram identificadas em acordo com a estrutura População, Intervenção e Resultados para a posterior formulação da *string* de busca. Segue a listagem de palavras-chave definidas:

- **População**
  - **Inglês:** *Software Development, Software Project, Software Organization, Software Enterprise, Software Company, Software Industry, Software Institute, Software Research Group, Software Technology Center;*
  - **Português:** Desenvolvimento de Software, Projeto de Software, Organização de Software, Empresa de Software, Indústria de Software, Instituição de Software, Grupo de Pesquisa de Software, Centros de Tecnologia de Software;
- **Intervenção**
  - **Inglês:** *Requirements, Feature, Needs, User Story, Requirements Engineering, Task, Sprint, Backlog, Stakeholders, Product Owner, Participants, Team, Management, Understanding, Commitment, Approval, Traceability, Bidirectional Traceability, Inconsistencies, alignment, Project Work, Change Management, Manage Change, Change Log, Impact Analysis, Product Requirements, Product Component Requirements, Agility, Agile, Scrum, Lean, Crystal, Kanban, XP, Extreme Programming, FDD, Feature-Driven Development, TDD, Test-Driven Development, BDD, Behaviour-Driven Development, ATDD, Acceptance Test-Driven Development, ASD, Adaptive Software Development;*
  - **Português:** Requisitos, Funcionalidade, Necessidade, História do Usuário, Engenharia de Requisitos, Tarefa, Fornecedor, Cliente, Stakeholders, Participantes, Equipe, Time, Backlog, Gerência de Requisitos, Fornecedor, Entendimento, Comprometimento, Rastreabilidade, Rastreabilidade Bidirecional, Identificar Inconsistências, Gerência de Mudanças, Histórico de Mudança, Análise de Impacto, Requisitos do Produto, Requisitos do Componente do Produto, Agilidade, Ágil, Scrum, Lean, Crystal, Kanban, XP, Programação Extrema, FDD, Desenvolvimento Orientado a Funcionalidade, TDD,

Desenvolvimento Orientado a Teste, BDD, Desenvolvimento Orientado a Comportamento, ATDD, Desenvolvimento Orientado a Teste de Aceitação, ASD, Desenvolvimento Adaptativo de Software;

- **Resultados**
  - **Inglês:** *Methodology, Method, Technique, Practice;*
  - **Português:** Metodologia, Método, Técnica, Prática.

5.5 String de Busca

Para elaboração da *string* de busca, as palavras-chave e sinônimos relacionados são agrupados por meio de operador <OR> e os conjuntos de termos são agrupados com o operador <AND>, de acordo com a estrutura PICO (ou PIO no caso desta pesquisa) conforme segue (Santos, 2010):

P <and> I <and> C <and> O

Conforme já especificado anteriormente, a Comparação (ou Controle) não se aplica ao contexto desta revisão. Tem-se, então, que o conjunto de palavras-chave referentes a Comparação (C) é vazio (C=Ø). Adicionalmente, as palavras chave da Intervenção são relevantes apenas no contexto da Rastreabilidade de Requisitos, portanto as palavras chave.

5.5.1 Para Q1

- *("Software" AND ("Project" OR "Development" OR "Organization" OR "Enterprise" OR "Company" OR "Industry" OR "Institute" OR "Research Group" OR "Technology Center")) AND (("Requirements" OR "Feature" OR "Needs" OR "User Story" OR "Backlog" OR "Task" OR "Sprint") AND ("Stakeholders" OR "Product Owner" OR "Participants" OR "Team" OR "Engineering" OR "Manage*" OR "Understanding" OR "Commitment" OR "approval" OR "Traceability" OR "Bidirectional Traceability" OR "Inconsistencies" OR "alignment" OR "Project Work" OR "Change" OR "Log" OR "Impact Analysis")) AND (("Agility" OR "Agile" OR "Scrum" OR "Lean" OR "Crystal" OR "Kanban" OR "XP" OR "Extreme Programming" OR "FDD" OR "Feature Driven Development" OR "TDD" OR "Test Driven Development" OR "BDD" OR "Behaviour Driven Development" OR "ATDD" OR "Acceptance Test Driven Development" OR "ASD" OR "Adaptive Software Development") AND ("Methodology" OR "Method" OR "Technique" OR "Practice"))*

5.6 Fontes de Busca

Com base nos critérios de seleção de fontes e nas restrições definidas neste protocolo, tem-se a seguinte listagem de fontes de busca:

- IEEEXplore Digital Library;
- ACM Digital Library;
- Ei Compendex;
- Scopus;
- ISI Web of Knowledge;
- Science Direct;

6. Tipo dos Estudos

Serão considerados estudos primários do tipo: experimentais (ou E*mpirical Studies*); teóricos (estudos conceituais baseados em um entendimento de uma área); e, relatos de experiência industrial. Na forma de artigos completos de periódicos ou conferências.

7. Idioma dos Artigos

Serão considerados artigos escritos em português e inglês, o primeiro por ser importante considerar pesquisas de âmbito nacional, dada a relevância do Programa MPS.BR para o estudo, e o segundo para expandir a abrangência da pesquisa, uma vez que é a língua definida como padrão na grande maioria dos periódicos e conferências internacionais.

8. Critérios de Inclusão e Exclusão dos Estudos

Os critérios de exclusão dos estudos são:

CE.1) Estudos que não estejam disponíveis livremente para consulta ou download (em versão completa) através das fontes de pesquisa ou através das ferramentas de busca Google (http://www.google.com.br/) e/ou Google Scholar (http://scholar.google.com.br/);

CE.2) Estudos que claramente não atendam as questões de pesquisa;

CE.3) Estudos repetidos (em mais de uma fonte de busca) terão apenas sua primeira ocorrência considerada;

CE.4) Estudos enquadrados como resumos, *keynote speeches*, cursos, tutoriais, workshops e afins;

CE.5) Estudos que não mencionem as palavras-chave da pesquisa no título, resumo ou nas palavras-chave do artigo, salvo trabalhos que abordem melhoria do processo de software, nos quais seja observada possibilidade do Gerência de Requisitos e métodos ágeis serem tratados ao longo do trabalho.

CE.6) Excluir se o estudo não estiver inserido no contexto de Projetos de Software, Indústria de Software ou Engenharia de Software.

CE.7) Excluir se o estudo não estiver apresentado em uma das linguagens aceitas (Inglês e Português);

Os critérios de inclusão dos estudos se baseiam em:

CI.1) Estudos que apresentem primária ou secundariamente abordagens, no contexto dos métodos ágeis, de apoio às atividades de Engenharia de Requisitos.

CI.2) Estudos que apresentem relatos de experiência da indústria, ou pesquisas de caráter experimental ou teórico, contanto que apresentem exemplos de aplicação, descrição de experimentos ou casos reais de uso de abordagens, no contexto dos métodos ágeis, para apoio às atividades de Engenharia de Requisitos.

9. Critérios de Qualidade dos Estudos Primários

Em adição aos critérios de inclusão e exclusão, é considerado importante avaliar a qualidade dos estudos primários (KITCHENHAM, 2007). Os critérios de qualidade que serão aplicados aos estudos primários foram adaptados de Costa (2010), uma vez que descreviam critérios abrangentes o suficiente para cobrir o escopo dos estudos a serem

considerados, com alterações para adequarem-se aos objetivos e questões de pesquisa desta RSL. Os critérios de qualidade são:

1. **Introdução/Planejamento**
   a. Os objetivos ou questões do estudo são claramente definidos (incluindo justificativas para a realização do estudo)?
   b. O tipo de estudo está definido claramente?
2. **Desenvolvimento**
   a. Existe uma clara descrição do contexto no qual a pesquisa foi realizada?
   b. O trabalho é bem/adequadamente referenciado (apresenta trabalhos relacionados ou semelhantes e baseia-se em modelos e teorias da literatura)?
3. **Conclusão**
   a. O estudo relata de forma clara e não ambígua os resultados?
   b. Os objetivos ou questões do estudo são alcançados?
4. **Critérios para a Questão de Investigação**
   a. O estudo lista primária ou secundariamente as metodologias ou métodos ou técnicas ou práticas ágeis para apoiar as atividades de Desenvolvimento de Requisitos?
   b. O estudo endereça explicitamente as abordagens ágeis para alguma atividade de Desenvolvimento de Requisitos?
5. **Critério Específico para estudos Experimentais**
   a. Existe um método ou um conjunto de métodos descrito para a realização do estudo?
6. **Critério Específico para estudos Teóricos**
   a. Existe um processo não tendencioso na escolha dos estudos?
7. **Critério Específico para Relato de Experiência Industrial**
   a. Existe uma descrição sobre a(s) organização(ões) onde foi conduzido o estudo?

10. Processo de Buscas nas Fontes

Para a execução desta revisão sistemática, o método de busca definido anteriormente seguirá as seguintes etapas:

10.1 Processo de Busca Automática

Serão realizadas as buscas em todas as fontes selecionadas, por meio de *strings* de busca, estudos claramente irrelevantes à pesquisa são descartados. Os artigos são catalogados de forma a estabelecer uma planilha com a lista, para cada pesquisador, de possíveis estudos primários;

11. Processo de Seleção dos Estudos Primários

Para a execução desta RSL, os seguintes recursos serão utilizados:

- Três pesquisadores (um aluno de mestrado e dois graduandos);
- Acesso às fontes de pesquisa por meio do domínio da Universidade Federal do Pará;
- Validações sobre documentos e procedimentos da realização da RSL através de reuniões com o coordenador do Projeto SPIDER e orientador do projeto de dissertação, onde esta pesquisa está inserida, o Prof. Dr. Sandro Ronaldo Bezerra Oliveira;

Durante a condução desta RSL, os estudos primários serão identificados conforme o seguinte processo:

1. A partir da leitura de título, palavras chave, resumo, introdução e conclusão, os artigos deverão ser avaliados quanto aos critérios de inclusão e exclusão, e o resultado deve ser registrado.
2. Os pesquisadores responsáveis pela seleção dos artigos devem entrar em consenso, quando necessário. Isso ocorre quando não há uma unanimidade na inclusão de um artigo e pelo menos um pesquisador decide não incluir.
3. Na fase de consenso, em caso de discordância sobre a inclusão de algum estudo, o mesmo deverá ser incluído.
4. Os estudos primários identificados serão posteriormente lidos em totalidade e então será aplicada a avaliação de qualidade e a estratégia de extração de dados, conforme descrito nas subseções seguintes.

12. Avaliação da Qualidade dos Estudos Primários

A qualidade de um artigo pode ser mensurada pela relevância e valor científico de seu conteúdo; também é considerado um critério de exclusão, a ser aplicado durante a condução da pesquisa. Desta forma, durante a análise dos estudos primários e coleta de resultados, serão aplicados os critérios de qualidade (definidos na seção 9), permitindo um processo adicional de validação dos estudos, de forma a identificar possíveis trabalhos que ainda devem ser desconsiderados da pesquisa e observar o grau de importância dos estudos individualmente para quaisquer comparações durante a síntese dos dados (KITCHENHAM, 2007).

Adicionalmente, a avaliação da qualidade pode servir como recomendação de estudos para futuros pesquisas, fornecendo informações a respeito da qualidade das informações de cada estudo avaliado (KITCHENHAM, 2007).

Artigos que, porventura, sejam excluídos por não se adequarem aos critérios de qualidade devem ser citados, juntamente com as razões para sua exclusão. Após este passo, os artigos incluídos na pesquisa passarão pela fase de extração de dados.

Os critérios (1) a (4) se aplicam a todos os estudos primários avaliados, enquanto os critérios (5) a (7) se aplicam especificamente aos respectivos tipos de estudos mencionados.

12.1 Processo de Avaliação de Qualidade

Os estudos primários selecionados são lidos em totalidade e então são avaliados quanto aos critérios de qualidade. Para avaliar o grau de adequação aos critérios de qualidade, será adotada a estratégia de avaliação proposta por Costa (2010), onde se utiliza a escala de *Likert*-5, permitindo respostas gradativas de 0 (discordo totalmente) à 4 (concordo totalmente). É importante dizer que a escala *Likert*-5 caracteriza-se por abranger cinco características: a validade da investigação, as ameaças à validade, a pertinência, a aplicabilidade e a consistência das evidências (SILVA *et al*., 2010). Para auxiliar a avaliação seguindo a escala de *Likert*-5 para cada critério de qualidade foram definidas escalas. Estes quadros são descritos abaixo, respectivamente:

Escala de *Likert*-5	
Concordo totalmente (4)	Deve ser concedido no caso em que o trabalho apresente no texto os critérios que atendam totalmente a questão.
Concordo parcialmente (3)	Deve ser concedido no caso em que o trabalho atenda parcialmente aos critérios da questão.

Neutro (2)	Deve ser concedido no caso em que o trabalho não deixe claro se atende ou não a questão;
Discordo parcialmente (1)	Deve ser concedido no caso em que os critérios contidos na questão não são atendidos pelo trabalho avaliado;
Discordo totalmente (0)	Deve ser concedido no caso em que não existe nada no trabalho que atenda aos critérios da questão.

Escala por Critério	
Critério	**Escala**
1a.	4 - Define e justifica o estudo claramente. 3 - Define claramente o estudo, porém a justificativa não é clara. 2 - Define claramente o estudo, mas não justifica. 1 - A definição dos objetivos do estudo não é clara. 0 - Não define o estudo.
1b.	4 – Define o tipo de estudo, referenciando na literatura a metodologia. 3 – Define o tipo de estudo, porém sem referenciar a metodologia. 2 – Não define o tipo de estudo. É possível inferir facilmente. 1 – Não define o tipo de estudo. É possível inferir com dificuldade. 0 – Não é possível inferir o tipo de estudo.
2a.	4 – Define claramente uma seção com o contexto da pesquisa. 3 – O contexto da pesquisa está incluído em uma seção não exclusiva. 2 – O contexto da pesquisa está disperso ao longo do texto. 1 – O contexto da pesquisa está disperso e é insubstancial. 0 – O contexto da pesquisa não é abordado.
2b.	4 – O texto apresenta uma seção de trabalhos relacionados. 3 – O texto apresenta trabalhos relacionados em uma seção não exclusiva. 2 – O texto apresenta trabalhos relacionados dispersos ao longo do texto. 1 – O texto não apresenta trabalhos relacionados, mas se apoia na literatura. 0 – O texto não apresenta trabalhos relacionados nem se apoia na literatura.
3a.	4 – Resultados são claramente apresentados na seção de conclusão. 3 – Resultados são claramente referenciados na seção de conclusão. 2 – Resultados apresentados na conclusão não são claros. 1 – Resultados referenciados na conclusão não são claros. 0 – Não são apresentados resultados.
3b.	4 – Os resultados estão totalmente aderentes ao objetivo do estudo. 3 – Os resultados estão aderentes ao objetivo do estudo, no entanto o autor faz ressalvas. 2 – Os resultados são parcialmente aderentes ao objetivo do estudo. 1 – Os resultados não estão aderentes ao objetivo do estudo. 0 – Não é alcançado nenhum resultado.
4a.	4 – Algum dos elementos é claramente descrito. 3 – Algum dos elementos é apresentado, mas não aprofundado. 2 – Algum dos elementos é avaliado, porém não descrito.

	1 – Algum dos elementos é citado indiretamente. 0 – Nenhum dos elementos é apresentado direta ou indiretamente.
4b.	4 – Endereça a abordagem ágil para alguma atividade de forma detalhada (o nome, o que representa e como se usa a abordagem). 3 – Endereça a abordagem ágil para alguma atividade de forma superficial (o nome e o que representa a abordagem). 2 – Endereça a abordagem ágil para alguma atividade de forma resumida (apenas o nome da abordagem). 1 – Não Endereça a abordagem ágil para alguma atividade. 0 – Não é apresentada nenhuma abordagem ágil.
5a.	4 – O método de experimento é definido e referenciado claramente. 3 – O método de experimento é definido claramente. 2 – O método de experimento é citado. 1 – O método de experimento não é citado, porém é possível inferir. 0 – Não é possível inferir o método de experimento.
6a.	4 – O texto descreve critérios para a escolha dos estudos. 3 – O texto não descreve critérios para a escolha dos estudos, porém apresenta estudos que discordam do estudo apresentado. 2 – O texto descreve apenas estudos aderentes ao estudo apresentado. 1 – O texto descreve estudos insuficientes. 0 – O texto não descreve estudos base.
7a.	4 – A área de atuação, tamanho e origem da organização são informados. 3 – Apenas duas das características do item 4 são informadas. 2 – Apenas uma das características do item 4 é informada. 1 – Nenhuma das características do item 4 é informada. 0 – O estudo não foi conduzido em uma ou mais organizações.

Deve-se então, dada à soma das pontuações, enquadrá-los em um dos cinco níveis de qualidade classificados por Beecham (2007) *apud* Costa (2010), tais como, apresentados no quadro abaixo:

Faixa de Notas	**Avaliação**
Excelente	>86%
Muito Boa	66%-85%
Boa	46%-65%
Média	26%-45%
Baixa	< 26%

13. Estratégia de Extração e Síntese de Informações

Cada publicação selecionada é avaliada com relação ao tipo de abordagem que ela utiliza. Esta aderência é registrada na tabela Avaliação da Qualidade.

13.1 Extração de resultados

Para cada estudo primário analisado durante a etapa de extração de resultados, será feito um resumo (já identificando o que é proposto: metodologia, método, técnica, prática) e serão observados os detalhes referentes as questões secundárias da questão (ver 4.2). Desta forma, será analisado:

Q1A – Qual o contexto de aplicação da abordagem encontrada?
A abrangência da abordagem (referente à questão de pesquisa Q1) será analisada em relação ao contexto de desenvolvimento onde foi encontrada a abordagem (acadêmico, industrial).

Q1B – Quais atividades de Gerência de Requisitos são apoiadas pelas abordagens encontradas?
Para cada abordagem ágil identificada nos estudos será analisado como a mesma atende alguma(s) atividade(s) do processo de Gerência de Requisitos.

Q1C – Caso a abordagem seja uma prática ou técnica, as mesmas pertencem a algum método existente? Qual?
Para cada abordagem ágil do tipo prática ou técnica será analisado se existe algum método ágil, as quais as mesmas pertencem.

Q1D – Quais os ativos (papéis, artefatos e procedimentos) envolvidos?
Para cada abordagem encontrada nos estudos, verificar quais os ativos envolvidos em sua execução.

13.2Sumarização dos Resultados

O principal gráfico gerado em relação à questão de pesquisa é o da quantidade de artigos que abordam uma determinada técnica ou método. Outros gráficos podem ser desenvolvidos ao longo da Extração dos Resultados, como, por exemplo:

- Quantidade de artigos selecionados por base de dados;
- Evolução ao longo do tempo da quantidade de estudos selecionados;
- Quantidade de estudos por tipo (Experimental, Teórico, Relato de Experiência);

APÊNDICE B – ESTUDOS PRIMÁRIOS SELECIONADOS

ID	Título do estudo primário
EP1	A brief report on working smarter with Agile software development
EP2	A case study: Agile development in the community laser-induced incandescence modeling environment (CLiiME)
EP3	A Case Study: Introducing eXtreme Programming in a US Government System Development Project
EP4	A cognitive requirement specification model
EP5	A conceptual model and process for client-driven agile requirements prioritization
EP6	A feasible user story tool for agile software development?
EP7	A genetic algorithm approach to release planning in agile environment
EP8	A Hybrid Approach of Requirement Engineering in Agile Software Development
EP9	A Hybrid Software Architecture Evaluation Method for FDD - An Agile Process Model
EP10	A Quality Attributes Evaluation Method for an Agile Approach
EP11	A semantic and collaborative platform for agile requirements evolution
EP12	A Soft-Structured Agile Framework for Larger Scale Systems Development
EP13	A test automation framework in POCT system using TDD techniques
EP14	A Usability-Pattern-Based Requirements-Analysis Method to Bridge the Gap between User Tasks and Application Features
EP15	Adaptive Agile Performance Modeling and Testing
EP16	Agile Adoption Story from NHN
EP17	Agile Method Tailoring in Distributed Enterprises: Product Owner Teams
EP18	Agile Requirements Engineering Practices: An Empirical Study
EP19	Agile requirements engineering with prototyping: A case study
EP20	Agile software development methodology for medium and large projects
EP21	An agile request for proposal (RFP) process
EP22	An Agile Technique for Agent Based Goal Refinement to Elicit Soft Goals in Goal Oriented Requirements Engineering
EP23	An approach to requirements elicitation and analysis using goal
EP24	An Empirical Study on the Requirements Engineering Practices for Agile Software Development
EP25	An evolutionary lifecycle model with Agile practices for software development at ABB
EP26	Application of Agile Requirement Engineering in Modest-Sized Information Systems Development
EP27	Are Use Cases Beneficial for Developers Using Agile Requirements?
EP28	AspOrAS: A requirements agile approach based on scenarios and aspects
EP29	Automated Acceptance Testing as an Agile Requirements Engineering Practice

EP30	Binding requirements and component architecture by using model-based test-driven development
EP31	Breaking the ice for agile development of embedded software: an industry experience report
EP32	Capturing, eliciting, predicting and prioritizing (CEPP) non-functional requirements metadata during the early stages of agile software development
EP33	Case studies in just-in-time requirements analysis
EP34	Comparing Agile Software Processes Based on the Software Development Project Requirements
EP35	Comparison between Adaptive Software Development and Feature Driven Development
EP36	Complementing Measurements and Real Options Concepts to Support Inter-iteration Decision-Making in Agile Projects
EP37	CPASA: Continuous Performance Assessment of Software Architecture
EP38	Do We Know Enough about Requirements Prioritization in Agile Projects: Insights from a Case Study
EP39	Dual Application Model for Agile Software Engineering
EP40	Effectiveness of scrum for offshore software development in Sri Lanka
EP41	Elicitation of Goals in Requirements Engineering Using Agile Methods
EP42	Enhancement approach for non-functional requirements analysis in Agile environment
EP43	Evaluating Test-Driven Development in an Industry-Sponsored Capstone Project
EP44	Experiences integrating sophisticated user experience design practices into agile processes
EP45	Extending Agile Processes with Creativity Techniques
EP46	Extreme programming modified: embrace requirements engineering practices
EP47	FDMD: Feature-Driven Methodology Development
EP48	FDRD: Feature driven reuse development process model
EP49	Forging high-quality User Stories: Towards a discipline for Agile Requirements
EP50	Goal Sketching: Towards Agile Requirements Engineering
EP51	Handling Uncertainty in Agile Requirement Prioritization and Scheduling Using Statistical Simulation
EP52	Improving the User Story Agile Technique Using the INVEST Criteria
EP53	Incorporating Security Requirements Engineering into the Dynamic Systems Development Method
EP54	Integrating Shadows in Model Driven Engineering for Agile Software Development
EP55	Management Information System Requirements Analysis Model Based on the Agile Development
EP56	Managing Agile Project Requirements with Storytest-Driven Development
EP57	Managing uncertainty of story-points in Agile software
EP58	Mind-mapping: An effective technique to facilitate requirements engineering in agile software development
EP59	Novel lightweight engineering artifacts for modeling non-functional requirements in agile processes
EP60	On agile performance requirements specification and testing
EP61	On requirements verification for model refinements
EP62	Piloting XP on four mission-critical projects

EP63	Prioritising user stories in agile environment
EP64	Quality Attribute Driven Agile Development
EP65	Requirements engineering and agile software development
EP66	Scrum + Engineering Practices: Experiences of Three Microsoft Teams
EP67	ScrumBut, But Does it Matter? A Mixed-Method Study of the Planning Process of a Multi-team Scrum Organization
EP68	SnapMind: A framework to support consistency and validation of model-based requirements in agile development
EP69	Socio-technical aspects of requirements-driven collaboration (RDC) in agile software development methods
EP70	Ssh! We are adding a process... [agile practices]
EP71	'State of the Art' in Using Agile Methods for Embedded Systems Development
EP72	Supporting the Dynamic Reprioritization of Requirements in Agile Development of Software Products
EP73	Technical Reviews in Agile Development: Case Mobile-D^TM
EP74	Testing in an agile product development environment: An industry experience report
EP75	The Agile Requirements Refinery: Applying SCRUM Principles to Software Product Management
EP76	The changing landscape of requirements engineering practices over the past decade
EP77	The Many Lives of an Agile Story: Design Processes, Design Products, and Understandings in a Large-Scale Agile Development Project
EP78	The NERV methodology: A lightweight process for addressing non-functional requirements in agile software development
EP79	The NORMAP Methodology: Lightweight Engineering of Non-functional Requirements for Agile Processes
EP80	The Research and Application of the Requirement Modeling Method on AM-RUP Requirement Process
EP81	The XP customer role in practice: three studies
EP82	There's never enough time: Doing requirements under resource constraints, and what requirements engineering can learn from agile development
EP83	Towards an aspect-oriented agile requirements approach
EP84	Transition to Agile Development - Rediscovery of Important Requirements Engineering Practices
EP85	User stories template for object-oriented applications
EP86	Using Agile Methods in Software Product Development: A Case Study
EP87	Using goal net to model user stories in agile software development
EP88	A Case Study for Prioritizing Features in Environments with Multiple Stakeholders
EP89	A Case Study on Benefits and Side-effects of Agile Practices in Large-scale Requirements Engineering
EP90	A Conceptual Model of Client-driven Agile Requirements Prioritization: Results of a Case Study
EP91	A Pilot Study of Comparative Customer Comprehension Between Extreme X-machine and Uml Models
EP92	A Reflection on Agile Requirements Engineering: Solutions Brought and Challenges Posed
EP93	Agile Modeling Method Engineering
EP94	An Agile Hypertext Design Methodology

EP95	An Agile Requirements Elicitation Approach Based on NFRs and Business Process Models for Micro-businesses
EP96	Communication Patterns of Agile Requirements Engineering
EP97	Extending XP Practices to Support Security Requirements Engineering
EP98	Generating Feature Model from Creative Requirements Using Model Driven Design
EP99	Hitting the Target: Adding Interaction Design to Agile Software Development
EP100	Integrating Requirements Engineering and User Experience Design in Product Life Cycle Management
EP101	Quantified Extreme Scenario Based Design Approach
EP102	Re-conceptualizing Requirements Engineering: Findings from a Large-scale, Agile Project
EP103	ScrumS: A Model for Safe Agile Development
EP104	The Benefits and Challenges of Executable Acceptance Testing
EP105	Towards an Evolutionary Framework for Agile Requirements Elicitation
EP106	Towards Continuous Validation of Customer Value
EP107	Understanding the Role of Requirements Artifacts in Kanban
EP108	A comparative study of software tools for user story management
EP109	A new agile process for Web development
EP110	A research and practice of agile unified requirement modeling
EP111	Agile requirements prioritization in large-scale outsourced system projects: An empirical study
EP112	Agile software development methodology for medium and large projects
EP113	An analysis of the agile method Scrum approach adapted in the process areas requirements management and requirements development of the CMMI
EP114	An approach to test-driven development of conceptual schemas
EP115	Automatically generating tests from natural language descriptions of software behavior
EP116	Business Language Driven Development: Joining business process models to automated tests
EP117	Creating user scenarios through user interaction diagrams by non-technical customers
EP118	From requirements to web applications in an agile model-driven approach
EP119	Implementing behavior driven development in an open source ERP
EP120	Little design up-front: A design science approach to integrating usability into agile requirements engineering
EP121	On requirement verification for evolving Statecharts specifications
EP122	Requirements engineering with agent-oriented models
EP123	Requirements specification as executable software design - A behavior perspective
EP124	SoBA: A tool support for story card based agile software development
EP125	Some findings concerning requirements in agile methodologies
EP126	Story card Maturity Model (SMM): A process improvement framework for agile requirements engineering practices
EP127	Study on combining model-driven engineering and scrum to produce web information systems
EP128	Test Driven domain modelling
EP129	The agile requirements refinery: Applying SCRUM principles to software product management

EP130	User scenarios through user interaction diagrams
EP131	Using i* models to enrich user stories
EP132	Adopting agile in an FDA regulated environment
EP133	Agile requirements engineering via paraconsistent reasoning
EP134	Applying XP to an agile-inexperienced software development team
EP135	Continuous and collaborative validation: A field study of requirements knowledge in agile
EP136	Delivering user stories for implementing logical software architectures by multiple scrum teams
EP137	Describing agile requirements development and communication using complex adaptive systems theory
EP138	Development of Complex Software with Agile Method
EP139	Engineering m-learning using agile user-centered design
EP140	Experimenting a requirements engineering process based on rational unified process (RUP) reaching capability maturity model integration (CMMI) maturity level 3 and considering the use of agile methods practices
EP141	Extreme requirements engineering (XRE)
EP142	Generating user stories in groups
EP143	Incorporation of usability evaluation methods in agile software model
EP144	Market-driven software project through agility: Requirements engineering perspective
EP145	Multi-case study of agile requirements engineering and the use of test cases as requirements
EP146	Requirements engineering in agile software development
EP147	Security and privacy behavior definition for behavior driven development
EP148	Supporting agile software development through active documentation
EP149	The projection of the specific practices of the third level of CMMI model in agile methods: Scrum, XP and Kanban
EP150	The Role of Requirements Engineering Practices in Agile Development: An Empirical Study
EP151	Mockup-Driven Development: Providing agile support for Model-Driven Web Engineering
EP152	A HCI technique for improving requirements elicitation
EP153	The role of physical artefacts in agile software development: Two complementary perspectives
EP154	Understanding agile software development practices using shared mental models theory
EP155	A Reference Method for User Story Requirements in Agile Systems Development
EP156	Agile techniques and frameworks based on the requirements for e-commerce applications
EP157	Aligning Requirements and Testing Working Together toward the Same Goal
EP158	An Agile Approach for Managing Requirements to Improve Learning and Adaptability
EP159	Application of a Hybrid Process Software Requirements Management
EP160	Applying Standard Independent Verification and Validation (IV&V) Techniques within an Agile Framework: Is there a Compatibility Issue?
EP161	Enhancing Requirements Engineering in Agile Methodologies by Agent-Oriented Goal Models

EP162	Lightweight Traceability for the Agile Architect
EP163	Requirement Change Taxonomy and Categorization in Agile Software Development
EP164	Requirements Engineering Challenges in Large-Scale Agile System Development
EP165	SCRUM Model for Agile Methodology
EP166	Towards Requirements Reuse by Implementing Traceability in Agile Development
EP167	An approach based on design practices to specify requirements in agile projects
EP168	Guidelines for utilizing change impact analysis when estimating effort in agile software development
EP169	Human Stories - A New Written Technique in Agile Software Requirements
EP170	Extracting conceptual models from user stories with Visual Narrator
EP171	Improving agile requirements: the Quality User Story framework and tool
EP172	Introducing Lean Canvas Model Adaptation in the Scrum Software Testing
EP173	Evaluating A Novel Agile Requirements Engineering Method: A Case Study
EP174	Impact and Challenges of Requirement Engineering in Agile Methodologies: A Systematic Review
EP175	A multi-case study of agile requirements engineering and the use of test cases as requirements
EP176	A requirements engineering techniques review in agile software development methods
EP177	Requirements elicitation based on inception deck and business processes models in scrum

Protocolo de Revisão

PRESSMAN, R. S.; MAXIN, B. R. **Engenharia de Software: uma abordagem professional**. 8th Ed., McGraw-Hill. ISBN: 9788580555332. 2016.

APÊNDICE C – FORMULÁRIO DE AVALIAÇÃO SWOT

Data: **Iteração avaliada:** **Participantes:**

Itens para avaliar: Ciclo de Vida Artefatos Papéis Cerimônias

Critérios	**S** (*Strengths*)	**W** (*Weaknesses*)	**O** (*Opportunities*)	**T** (*Threads*)
Eficiência				
Simplicidade				
Utilidade				
Aplicabilidade				
Confiabilidade				
Colaboração da Equipe				